RESUMEN SISTEMAS OPERATIVOS

1. INTRODUCCIÓN

1.1 ¿QUÉ ES UN SISTEMA OPERATIVO?

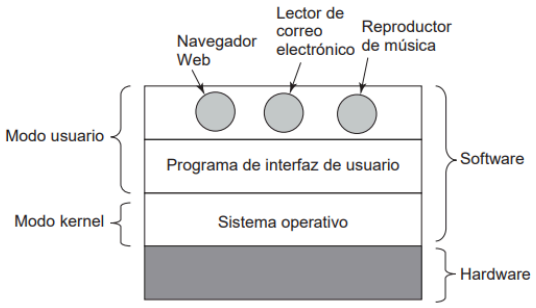


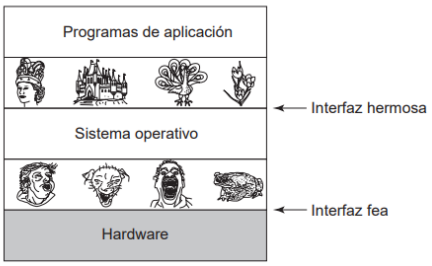
Figura 1-1. Ubicación del sistema operativo.

Es difícil definir qué es un sistema operativo aparte de decir que es el **software** que se ejecuta en **modo kernel** (además de que esto no siempre es cierto). Parte del problema es que los sistemas operativos realizan dos funciones básicas que no están relacionadas:

- **El sistema operativo como una máquina extendida**: La arquitectura de la mayoría de las computadoras a nivel de lenguaje máquina es primitiva y compleja de programar.

  La **abstracción** es la clave para lidiar con la **complejidad**. Una abstracción que casi cualquier usuario de computadora comprende es el archivo: es una pieza útil de información, como una fotografía digital, un mensaje de correo electrónico almacenado o una página Web. Es más fácil lidiar con fotografías, correos electrónicos y páginas Web que con los detalles de los discos.

El trabajo del sistema operativo es crear buenas abstracciones para después implementar y administrar los objetos abstractos entonces creados



- **El sistema operativo como administrador de recursos**:La administración de recursos incluye el **multiplexaje** (compartir) de recursos en dos formas distintas: en el **tiempo** y en el **espacio**.

En la perspectiva alterna, el trabajo del sistema operativo es proporcionar una **asignación ordenada y controlada** de los procesadores, memorias y dispositivos de E/S, **entre los diversos programas que compiten por estos recursos.**

Cuando un recurso se **multiplexa** en el **tiempo**, los distintos programas o usuarios **toman turnos para utilizarlo.**

El otro tipo de **multiplexaje** es en el **espacio**. En vez de que los clientes tomen turnos, cada uno obtiene **una parte del recurso.**

1.3 REVISIÓN DEL HARDWARE DE COMPUTADORA

La CPU, la memoria y los dispositivos de E/S están conectados mediante un **bus del**

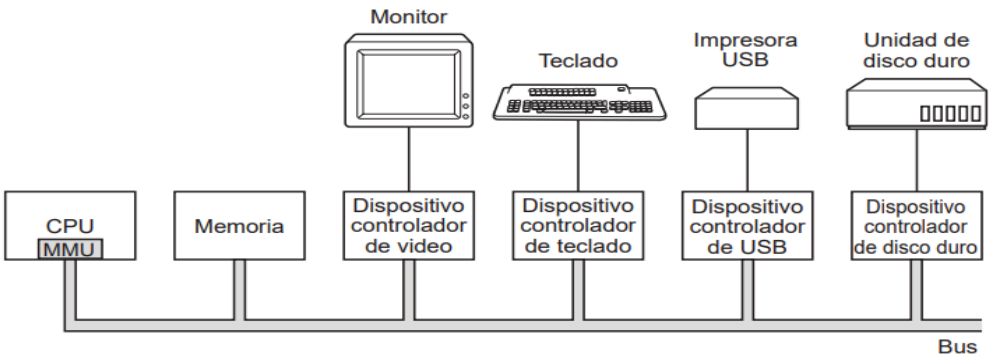


Figura 1-6. Algunos de los componentes de una computadora personal simple.

sistema y se comunican entre sí a través de este bus:

1.3.1 Procesadores

El "**cerebro**" de la computadora es la **CPU**, que obtiene las **instrucciones de la memoria** y las **ejecuta.** El ciclo básico de toda CPU es obtener la primera instrucción de memoria, decodificarla para determinar su tipo y operandos, ejecutarla y después obtener, decodificar y ejecutar las instrucciones subsiguientes. El ciclo se repite hasta que el programa termina.

Además de los registros generales utilizados para contener variables y resultados temporales, la mayoría de las computadoras tienen varios registros especiales que están visibles para el programador.

- **Contador de programa (Program Counter):** Contiene la dirección de memoria de la siguiente instrucción a obtener; se actualiza automáticamente para apuntar a la siguiente.
- **Apuntador de pila (Stack Pointer):** Apunta a la parte superior de la pila actual en memoria, la cual guarda los valores de los procedimientos activos (entrados pero no salidos).
- **PSW (Palabra de estado del programa):** Registro que contiene los bits de código de condición.

El sistema operativo debe estar al tanto de todos los registros.

La mayoría de las CPU, con excepción de las extremadamente simples que se utilizan en los sistemas integrados, tienen **dos modos: modo kernel y modo usuario**. Por lo general, un bit en el PSW controla el modo. Al operar en **modo kernel**, la CPU puede ejecutar **cualquier instrucció**n de su conjunto de instrucciones y utilizar **todas** las **características** del **hardware**. El sistema operativo opera en modo kernel, lo cual le da acceso al hardware completo.

En contraste, los **programas de usuario** operan en modo de usuario, el cual les permite ejecutar sólo un **subconjunto** de las **instrucciones** y les da acceso sólo a un subconjunto de las características.

Para obtener **servicios del sistema operativo**, un programa usuario debe lanzar una **llamada al sistema** (system call), la cual se atrapa en el kernel e invoca al sistema operativo. La instrucción **TRAP** cambia del **modo usuario** al **modo kernel** e inicia el sistema operativo.

Chips con multihilamiento y multinúcleo

Multihilamiento o **hiperhilamiento:**. Para una primera aproximación, lo que hace es permitir que la CPU contenga el estado de **dos hilos de ejecución** (threads) distintos y luego alterne entre uno y otro con una escala de tiempo en nanosegundos.

Memoria

Una memoria debe ser más rápida que la velocidad de ejecución de una instrucción, de manera que la memoria no detenga a la CPU.

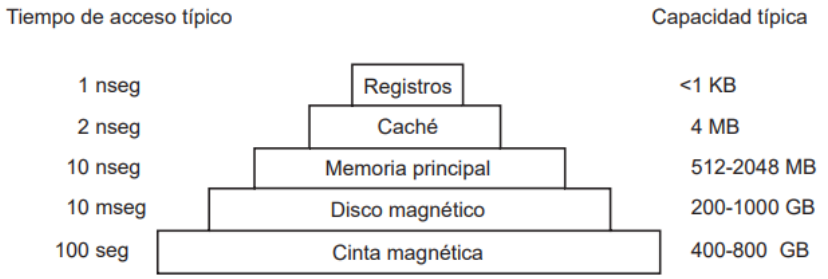


Figura 1-9. Una común jerarquía de memoria. Los números son sólo aproximaciones.

Registros: Compuestos del mismo material que la CPU y, por ende, tienen la misma rapidez.

Memoria caché: El hardware la controla de **manera parcial**. La memoria principal se divide en **líneas de caché**. Las líneas de caché que se utilizan con más frecuencia se mantienen en una caché de **alta velocidad,** ubicada **dentro** o muy **cerca** de la **CPU**

- Cuando el programa necesita leer una palabra de memoria, el hardware de la caché comprueba si la línea que se requiere se encuentra en la caché. Si es así (a lo cual se le conoce como **acierto de caché**) la petición de la caché **se cumple** y **no se envía una petición** de memoria a través del bus hacia la memoria principal.
- La memoria caché está **limitada en tamaño** debido a su alto costo.
- Las cachés son una idea tan útil que las CPUs modernas tienen dos de ellas. La **caché L1** o de primer nivel está siempre dentro de la CPU, y por lo

general alimenta las instrucciones decodificadas al motor de ejecución de la CPU.

- **Caché L2**, que contiene varios megabytes de palabras de memoria utilizadas recientemente. La diferencia entre las cachés L1 y L2 está en la velocidad. El acceso a la caché L1 se realiza sin ningún retraso, mientras que el acceso a la caché L2 requiere un retraso de uno o dos ciclos de reloj.

Memoria Principal:

También conocida como **RAM** (Random Access Memory, Memoria de Acceso Aleatorio).

- Además de la memoria principal, muchas computadoras tienen una pequeña cantidad de **memoria de acceso aleatorio no volátil**. A diferencia de la RAM, la memoria no volátil no pierde su contenido cuando se desconecta la energía.
  - La **ROM** (Read Only Memory, Memoria de sólo lectura) se programa en la fábrica y no puede modificarse después. Es rápida y económica.
  - La **EEPROM** (Electrically Erasable PROM, PROM eléctricamente borrable) y la memoria flash son también no volátiles, pero en contraste con la ROM se pueden borrar y volver a escribir datos en ellas.
  - **CMOS**, la cual es volátil. Muchas computadoras utilizan memoria CMOS para guardar la fecha y hora actuales

Discos

Un disco consiste en uno o más **platos** que giran a 5400, 7200 o 10,800 rpm. Un **brazo mecánico**, con un **punto de giro** colocado en una esquina, se mueve sobre los platos de manera similar al brazo de la aguja en un viejo tocadiscos.

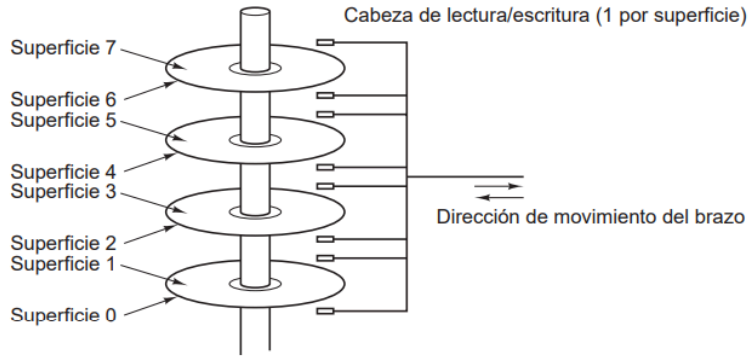


Figura 1-10. Estructura de una unidad de disco.

En cualquier posición dada del brazo, cada una de las cabezas puede leer una región anular conocida como **pista** (track). En conjunto, todas las pistas para una posición dada del brazo forman un **cilindro** (cylinder).

Muchas computadoras presentan un esquema conocido como **memoria virtual**, el cual permite la ejecución de **programas más grandes** que la **memoria física** al colocarlos en el **disco** y utilizar la memoria principal como un tipo de **caché** para las partes que se ejecutan con **más frecuencia**. Este esquema requiere la **reasignación de direcciones de memoria al instante**, para convertir la dirección que el programa generó en la dirección física en la RAM

en donde se encuentra la palabra. Esta asignación se realiza mediante una parte de la CPU conocida como **MMU** (Memory Management Unit, Unidad de Administración de Memoria)

Cintas

La última capa de la jerarquía en la memoria es la cinta magnética. Este medio se utiliza con frecuencia como r**espaldo para el almacenamiento en disco** y para contener conjuntos de datos muy extensos.

Dispositivos de E/S

Los dispositivos de E/S generalmente constan de dos partes: un dispositivo **controlador** y el **dispositivo en sí.** El dispositivo controlador es un **chip** o conjunto de chips que **controla físicamente** el **dispositivo**. Por ejemplo, acepta los comandos del sistema operativo para leer datos del dispositivo y los lleva a cabo.

Los dispositivos tienen **interfaces bastante simples**, debido a que no pueden hacer mucho y también para estandarizarlas. Esto último es necesario de manera que cualquier dispositivo controlador de disco IDE (Integrated Drive Electronics) significa Electrónica de unidades integradas.

El software que se comunica con un dispositivo controlador, que le proporciona comandos y acepta respuestas, se conoce como **driver** (controlador).Para utilizar el driver, se tiene que colocar en el sistema operativo de manera que pueda ejecutarse en **modo kernel**.

Las operaciones de entrada y salida se pueden realizar de tres maneras distintas.

1. **Un programa de usuario emite una llamada al sistema**, que el kernel posteriormente **traduce** en una llamada al procedimiento para el **driver apropiado**. Después el driver inicia la E/S y permanece en un ciclo estrecho, **sondeando** en forma continua al dispositivo para ver si ha terminado. Una vez terminada la E/S, el driver coloca los datos (si los hay) en donde se necesitan y regresa. Después el sistema operativo devuelve el control al llamador. A este método se le conoce como espera ocupada y tiene la desventaja de que mantiene ocupada la CPU sondeando al dispositivo hasta que éste termina.
2. El segundo método consiste en que el **driver inicie** el **dispositivo** y le **pida generar** una **interrupción** cuando **termine**. En este punto el driver regresa. Luego, el sistema operativo bloquea el programa llamador si es necesario y busca otro trabajo por hacer. Cuando el dispositivo controlador detecta el final de la transferencia, genera una interrupción para indicar que la operación se ha completado

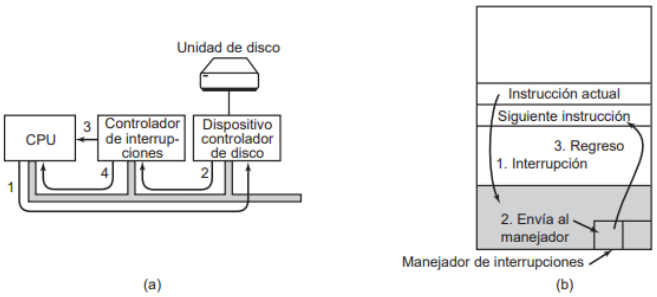


Figura 1-11. (a) Los pasos para iniciar un dispositivo de E/S y obtener una interrupción. (b) El procesamiento de interrupciones involucra tomar la interrupción, ejecutar el manejador de interrupciones y regresar al programa de usuario.

3. El tercer método para realizar operaciones de E/S hace uso de un **chip especial** llamado **DMA** (Direct Memory Access; Acceso directo a memoria) que puede controlar el **flujo de bits** entre la memoria y un dispositivo controlador sin la intervención constante de la CPU. La CPU configura el chip DMA, le indica cuántos bytes debe transferir, las direcciones de dispositivo y de memoria involucradas, la instrucción y deja que haga su trabajo. Cuando el chip DMA termina genera una interrupción, la cual se maneja de la manera antes descrita

Buses

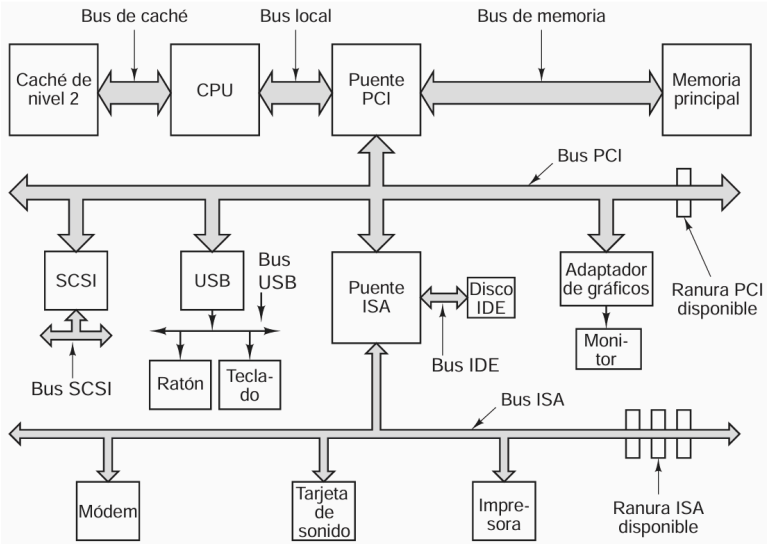


Figura 1-12. La estructura de un sistema Pentium extenso.

Los dos **buses principales** son el bus **ISA** (Industry Standard Architecture, Arquitectura estándar de la industria) de la IBM PC original y su sucesor, el bus **PCI** (Peripheral Component Interconnect, Interconexión de componentes periféricos).

El **USB** (Universal Serial Bus; Bus serial universal) se inventó para conectar a la computadora todos los **dispositivos de E/S lentos**, como el teclado y el ratón.

El bus **SCSI** (Small Computer System Interface, Interfaz para sistemas de cómputo pequeños) es un **bus de alto rendimiento**, diseñado para discos, escáneres y otros dispositivos veloces que necesitan de un ancho de banda considerable.

El sistema operativo tiene que saber qué dispositivos periféricos están conectados a la computadora y cómo configurarlos. Este requerimiento condujo a Intel y Microsoft a diseñar un sistema de PC conocido como **plug and play**. El sistema operativo tiene que saber qué dispositivos periféricos están conectados a la computadora y cómo configurarlos. Este requerimiento condujo a Intel y Microsoft a diseñar un sistema de PC conocido como plug and play. Antes de plug and play, cada tarjeta de E/S tenía un nivel de petición de interrupción fijo y direcciones fijas para sus registros de E/S. Lo cual presentaba un problema cuando dos dispositivos utilizaban la misma interrupción. La función de plug and play es permitir que el sistema recolecte automáticamente la información acerca de los dispositivos de E/S, asigne los niveles de interrupción y las direcciones de E/S de manera central, para que después indique a cada tarjeta cuáles son sus números.

Arranque de la computadora

En forma muy breve, el proceso de arranque del Pentium es el siguiente. Cada Pentium contiene una tarjeta madre (motherboard). En la tarjeta madre o padre hay un programa conocido como **BIOS** (Basic Input Output System, Sistema básico de entrada y salida) del sistema. El BIOS contiene software de E/S de bajo nivel, incluyendo procedimientos para leer el teclado, escribir en la pantalla y realizar operaciones de E/S de disco, entre otras cosas. Hoy en día está contenido en una RAM tipo flash que es no volátil pero el sistema operativo puede actualizarla cuando se encuentran errores en el BIOS.

Cuando se arranca la computadora, el BIOS inicia su ejecución. Primero hace **pruebas** para ver cuánta RAM hay instalada y si el teclado junto con otros dispositivos básicos están instalados y responden en forma correcta. Empieza explorando los **buses** ISA y PCI para detectar todos los dispositivos conectados a ellos. Comúnmente, algunos de estos dispositivos son heredados (es decir, se diseñaron antes de inventar la tecnología plug and play), además de tener valores fijos para los niveles de interrupciones y las direcciones de E/S (que posiblemente se establecen mediante interruptores o puentes en la tarjeta de E/S, pero que el sistema operativo no puede modificar). Estos dispositivos se registran; y los dispositivos plug and play también. Si los dispositivos presentes son distintos de los que había cuando el sistema se inició por última vez, se configuran los nuevos dispositivos

1.5 CONCEPTOS DE LOS SISTEMAS OPERATIVOS

Proceso: Un proceso es en esencia **un programa en ejecución.** Cada proceso tiene asociado un **espacio de direcciones**, una **lista de ubicaciones de memoria** que va desde algún mínimo (generalmente 0) hasta cierto valor máximo, donde el proceso puede leer y escribir información. El espacio de direcciones contiene el **programa ejecutable**, los **datos** del programa y su pila. También hay asociado a cada proceso un conjunto de recursos, que comúnmente incluye registros (el contador de programa y el apuntador de pila, entre ellos), una lista de archivos abiertos, alarmas pendientes, listas de procesos relacionados y toda la demás información necesaria para ejecutar el programa. En esencia, un proceso es un recipiente que guarda toda la información necesaria para ejecutar un programa

Las **llamadas al sistema** de administración de procesos clave son las que se encargan de la creación y la terminación de los procesos. Considere un ejemplo común. Un proceso llamado intérprete de comandos o shell lee comandos de una terminal. El usuario acaba de

escribir un comando, solicitando la compilación de un programa. El shell debe entonces crear un proceso para ejecutar el compilador. Cuando ese proceso ha terminado la compilación, ejecuta una llamada al sistema para terminarse a sí mismo.

Si un proceso puede crear uno o más procesos aparte (conocidos como procesos hijos) y estos procesos a su vez pueden crear procesos hijos, llegamos rápidamente la estructura de **árbol de procesos.**

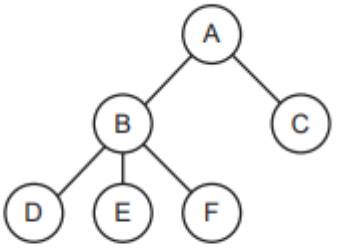


Figura 1-13. Un árbol de proceso. El proceso *A* creó dos proceso hijos, *B* y *C*. El proceso *B* creó tres procesos hijos, *D*, *E* y *F*.

Espacios de direcciones: Cada computadora tiene cierta memoria principal que utiliza para mantener los programas en ejecución. En un sistema operativo muy simple sólo hay un programa a la vez en la memoria.

Archivos:Existen dos tipos de archivos especiales: archivos especiales de bloque y archivos especiales de carácter.

Entrada/Salida:

Protección
El shell
Memoria virtual

1.6 Llamadas al Sistema

• **Definición:** Son la interfaz que permite a los programas de usuario solicitar servicios al sistema operativo.

• **Funcionamiento:** Aunque parecen llamadas a procedimientos normales, utilizan una instrucción **TRAP** para cambiar la CPU del modo usuario al modo kernel y transferir el control al sistema operativo para ejecutar la tarea.

• **POSIX (UNIX):** Se relacionan directamente con los procedimientos de biblioteca (relación uno a uno). Incluyen llamadas para la administración de procesos (`fork`, `exec`), archivos (`read`, `write`) y directorios.

• **Win32 API (Windows):** Es una interfaz con miles de funciones que está **desacoplada** de las verdaderas llamadas al sistema. Son procedimientos de biblioteca que pueden o no invocar al kernel, lo que permite a Microsoft cambiar el kernel sin afectar la compatibilidad de los programas.

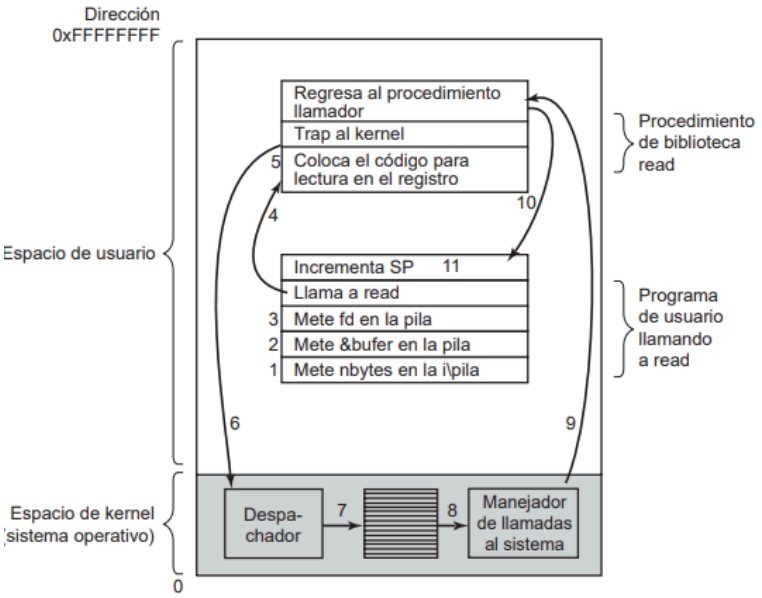


Figura 1-17. Los 11 pasos para realizar la llamada al sistema `read(fd, bufer, nbytes)`.

UNIX	Win32	Descripción
fork	CreateProcess	Crea un nuevo proceso
waitpid	WaitForSingleObject	Puede esperar a que un proceso termine
execve	(ninguno)	CreateProces = fork + execve
exit	ExitProcess	Termina la ejecución
open	CreateFile	Crea un archivo o abre uno existente
close	CloseHandle	Cierra un archivo
read	ReadFile	Lee datos de un archivo
write	WriteFile	Escribe datos en un archivo
lseek	SetFilePointer	Desplaza el apuntador del archivo
stat	GetFileAttributesEx	Obtiene varios atributos de un archivo
mkdir	CreateDirectory	Crea un nuevo directorio
rmdir	RemoveDirectory	Elimina un directorio vacío
link	(ninguno)	Win32 no soporta los enlaces
unlink	DeleteFile	Destruye un archivo existente
mount	(ninguno)	Win32 no soporta el montaje
umount	(ninguno)	Win32 no soporta el montaje
chdir	SetCurrentDirectory	Cambia el directorio de trabajo actual
chmod	(ninguno)	Win32 no soporta la seguridad (aunque NT sí)
kill	(ninguno)	Win32 no soporta las señales
time	GetLocalTime	Obtiene la hora actual

Figura 1-23. Las llamadas a la API Win32 que apenas corresponden a las llamadas de UNIX de la figura 1-18.

1.7 Estructura de un sistema operativo

- • **Sistemas monolíticos:** Es la organización más común. Todo el sistema operativo se ejecuta como **un solo programa en modo kernel**. El sistema operativo se escribe como una **colección de procedimientos**, enlazados entre sí en un solo programa binario

ejecutable extenso. Su estructura básica consta de un programa principal que invoca **procedimientos de servicio**, los cuales a su vez se ayudan de **procedimientos utilitarios**,

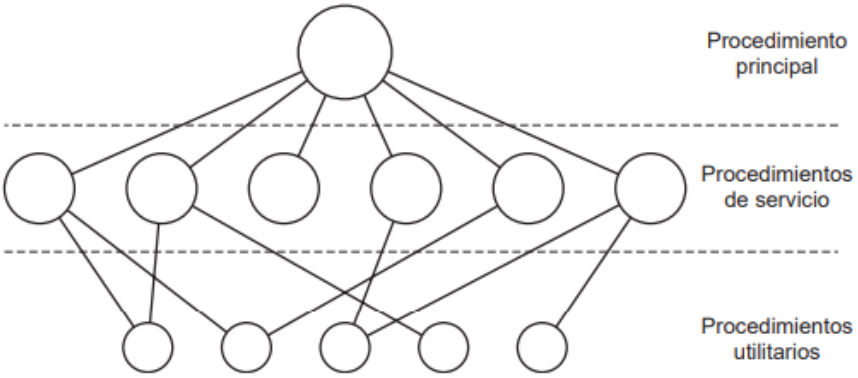


Figura 1-24. Un modelo de estructuración simple para un sistema monolítico.

- • **Sistemas de capas:** El sistema operativo se organiza como una **jerarquía de capas**, cada una construida encima de la que tiene abajo. El primer ejemplo fue el **sistema THE**, que tenía seis capas (desde la asignación del procesador hasta el operador). Una generalización de esto fue el sistema MULTICS, descrito como una serie de **anillos concéntricos**,,.
- • **Microkernels:** Su idea básica es lograr una **alta confiabilidad** al dividir el sistema operativo en **módulos pequeños y bien definidos**. Sólo uno de ellos (el **microkernel**) se ejecuta en modo kernel y el resto se ejecuta como **procesos de usuario** ordinarios sin poder relativamente (como los drivers de dispositivos). Un ejemplo es **MINIX 3**. A menudo se busca colocar el **mecanismo** en el kernel, pero dejar la **directiva** (política) en los procesos de usuario,,.
- **Modelo cliente-servidor:** Es una ligera variación de la idea del microkernel que diferencia dos clases de procesos: los **servidores** (proporcionan cierto servicio) y los **clientes** (utilizan estos servicios). La comunicación se lleva a cabo comúnmente mediante el **paso de mensajes**.
- **Máquinas virtuales:**
    - **Monitor de máquina virtual (VM/370):** Separa las funciones de multiprogramación y de máquina extendida. El monitor se ejecuta en el hardware y realiza la multiprogramación, proporcionando varias **máquinas virtuales** a la siguiente capa, las cuales son **copias exactas** del hardware real.
    - **Hipervisores:** El monitor de máquina virtual tradicional se conoce como **hipervisor de tipo 1** (se ejecuta en el hardware). Un **hipervisor de tipo 2** se ejecuta como un programa de usuario (por ejemplo, en Windows o Linux) e "interpreta" el conjunto de instrucciones,.
    - **JVM:** La Máquina virtual de Java es una arquitectura de computadora que utiliza un intérprete para ejecutar código byte.
    - **Exokernels:** Es una estrategia para **particionar** la máquina, proporcionando a cada usuario un subconjunto de los recursos. En la capa inferior (modo kernel) se ejecuta el **exokernel**, cuyo trabajo es asignar recursos a las máquinas virtuales y comprobar los intentos de utilizarlos.

UNIDAD 2 - PROCESOS E HILOS

El concepto más importante en cualquier sistema operativo es el de **proceso, una abstracción de un programa en ejecución.**
Los procesos son una de las abstracciones más antiguas e importantes que proporcionan los sistemas operativos: proporcionan la capacidad de operar (pseudo) concurrentemente, incluso cuando hay sólo una CPU disponible. Convierten una CPU en varias CPU virtuales.

2.1 PROCESOS

Todas las computadoras modernas ofrecen **varias cosas al mismo tiempo.** En cualquier sistema de multiprogramación, la CPU **conmuta de un proceso a otro con rapidez,** ejecutando cada uno durante décimas o centésimas de milisegundos: hablando en sentido estricto, en cualquier instante la CPU está ejecutando sólo un proceso, y en el transcurso de 1 segundo podría trabajar en varios de ellos, dando la apariencia de un **paralelismo** (o pseudoparalelismo, para distinguirlo del verdadero paralelismo de hardware de los sistemas multiprocesadores con dos o más CPUs que comparten la misma memoria física).
Un proceso no es más que una instancia de un programa en ejecución, incluyendo los valores actuales del contador de programa, los registros y las variables. Esta **conmutación** rápida de un **proceso a otro** se conoce como **multiprogramación**.

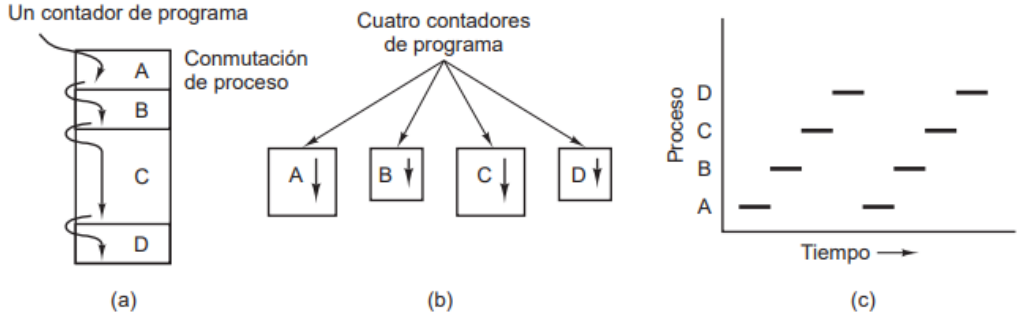


Figura 2-1. (a) Multiprogramación de cuatro programas. (b) Modelo conceptual de cuatro procesos secuenciales independientes. (c) Sólo hay un programa activo a la vez.

La **diferencia** entre un proceso y un programa es sutil pero crucial. La idea clave es que un **proceso es una actividad de cierto tipo:** tiene un programa, una entrada, una salida y un estado. Varios procesos pueden compartir un solo procesador mediante el uso de un algoritmo de planificación.

Creación de un proceso

Hay cuatro eventos principales que provocan la creación de procesos:

1. El arranque del sistema.
2. La ejecución, desde un proceso, de una llamada al sistema para creación de procesos.
3. Una petición de usuario para crear un proceso.
4. El inicio de un trabajo por lotes.

Demonios: Los procesos que permanecen en segundo plano para manejar ciertas actividades como correo electrónico, páginas Web, noticias, impresiones, etcétera. En Unix podemos usar ps para listar los procesos en ejecución.

Además de los procesos que se crean al arranque, posteriormente se pueden crear otros. A menudo, un **proceso en ejecución** emitirá **llamadas al sistema** para **crear** uno o más

procesos nuevos, para que le ayuden a realizar su trabajo. En especial, es útil crear procesos cuando el trabajo a realizar se puede formular fácilmente en términos de varios procesos interactivos relacionados entre sí, pero independientes
En los sistemas interactivos, los usuarios pueden iniciar un programa escribiendo un **comando** o haciendo (doble) **clic** en un icono
En los sistemas de procesamiento por lotes que se encuentran en las mainframes grandes. Aquí los usuarios pueden enviar trabajos de procesamiento por lotes al sistema (posiblemente en forma remota). Cuando el sistema operativo decide que tiene los **recursos** para **ejecutar otro trabajo**, crea un proceso y ejecuta el siguiente trabajo de la cola de entrada.
En UNIX sólo hay una llamada al sistema para crear un proceso: **fork**. Esta llamada crea un **clon exacto del proceso que hizo la llamada.** Después de fork, los dos procesos (padre e hijo) tienen la **misma imagen de memoria**, las mismas **cadenas de entorno** y los mismos **archivos abiertos.** Eso es todo. Por lo general, el proceso hijo ejecuta después a **execve** o una llamada al sistema similar para **cambiar su imagen de memoria** y ejecutar un nuevo programa. Por el contrario, en Windows una sola llamada a una función de Win32 (**CreateProcess**) maneja la creación de procesos y carga el programa correcto en el nuevo proceso.

2.1.3 Terminación de procesos

1. Salida normal (voluntaria).
2. Salida por error (voluntaria).
3. Error fatal (involuntaria).
4. Eliminado por otro proceso (involuntaria)

La mayoría de los procesos terminan debido a que han **concluido su trabajo**. Cuando un compilador ha compilado el programa que recibe, ejecuta una llamada al sistema para indicar al sistema operativo que ha terminado. Esta llamada es exit en UNIX y ExitProcess en Windows. Los programas orientados a pantalla también admiten la terminación voluntaria.
La segunda razón de terminación es que el proceso descubra un **error**. Por ejemplo, si un usuario escribe el comando para ejecutar foo.c y no existe dicho archivo, el compilador simplemente termina.
La tercera razón de terminación es un **error fatal** producido por el **proceso**, a menudo debido a un error en el programa. Algunos ejemplos incluyen ejecutar una instrucción ilegal, hacer referencia a una parte de memoria no existente o la división entre cero.
La cuarta razón por la que un proceso podría terminar es que **ejecute una llamada al sistema** que indique al sistema operativo que **elimine otros procesos.** En UNIX esta llamada es kill. La función correspondiente en Win32 es TerminateProcess

2.1.4 Jerarquías de procesos

En algunos sistemas, cuando un proceso crea otro, **el proceso padre y el proceso hijo** continúan **asociados** en ciertas formas. El proceso hijo puede crear por sí mismo más procesos, formando una jerarquía de procesos.

En **UNIX**, un proceso y todos sus hijos y descendientes forman un **grupo de procesos**. Cuando un usuario envía una **señal desde el teclado**, ésta se envía a todos los procesos

del grupo asociado al teclado. Cada proceso puede **atrapar la señal, ignorarla o ser eliminado**.

Al iniciar la computadora, UNIX crea un proceso especial llamado **init**. Este proceso lee cuántas terminales hay y usa **fork** para crear un proceso por cada una. Luego, estos procesos esperan un inicio de sesión y, si es exitoso, ejecutan un **shell** que puede crear más procesos. Así, **todos los procesos del sistema forman un solo árbol**, con **init en la raíz**.

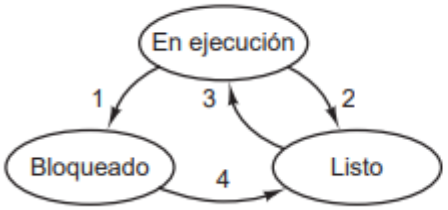
En **Windows**, no existe una jerarquía real de procesos: **todos los procesos son iguales**. Al crear un proceso, el padre recibe un **token (manejador)** para controlar al hijo, pero puede pasarlo a otros procesos, lo que **rompe la jerarquía**. En cambio, en UNIX **los procesos no pueden desheredar a sus hijos**.

Estados de un proceso

Cuando un proceso se bloquea, lo hace debido a que por lógica no puede continuar, comúnmente porque está esperando una entrada que todavía no está disponible. También es posible que un proceso, que esté listo en concepto y pueda ejecutarse, se detenga debido a que el sistema operativo ha decidido asignar la CPU a otro proceso por cierto tiempo

Los los tres estados en los que se puede encontrar un proceso son:

1. **En ejecución** (en realidad está usando la CPU en ese instante).
2. **Listo** (ejecutable; se detuvo temporalmente para dejar que se ejecute otro proceso).
3. **Bloqueado** (no puede ejecutarse sino hasta que ocurra cierto evento externo)



1. El proceso se bloquea para recibir entrada
2. El planificador selecciona otro proceso
3. El planificador selecciona este proceso
4. La entrada ya está disponible

Hay **cuatro transiciones** entre los **tres estados de un proceso**:

- La **transición 1** ocurre cuando un proceso no puede continuar y pasa a **bloqueado**, ya sea por una llamada al sistema o automáticamente (como en UNIX al leer sin datos disponibles).
- Las **transiciones 2 y 3** las maneja el **planificador**:
  la **2** cuando un proceso cede la CPU, y la **3** cuando vuelve a ejecutarse. La planificación busca **eficiencia y equidad**.
- La **transición 4** sucede cuando ocurre el **evento esperado**; el proceso pasa a **listo** o se ejecuta si la CPU está libre.

El **modelo de procesos** permite pensar el sistema como procesos que se bloquean y se desbloquean, con el **planificador** como nivel más bajo del sistema operativo.

Implementación de los procesos

Para implementar el modelo de procesos, el sistema operativo mantiene una tabla (un

arreglo de estructuras) llamada **tabla de procesos**, con sólo una **entrada por** cada **proceso** (algunos autores llaman a estas entradas bloques de control de procesos). Esta entrada contiene **información importante acerca del estado del proceso**, incluyendo su contador de programa, apuntador de pila, asignación de memoria, estado de sus archivos abiertos, información de contabilidad y planificación, etc.

Administración de procesos	Administración de memoria	Administración de archivos
Registros	Apuntador a la información del segmento de texto	Directorio raíz
Contador del programa	Apuntador a la información del segmento de datos	Directorio de trabajo
Palabra de estado del programa	Apuntador a la información del segmento de pila	Descripciones de archivos
Apuntador de la pila		ID de usuario
Estado del proceso		ID de grupo
Prioridad		
Parámetros de planificación		
ID del proceso		
Proceso padre		
Grupo de procesos		
Señales		
Tiempo de inicio del proceso		
Tiempo utilizado de la CPU		
Tiempo de la CPU utilizado por el hijo		
Hora de la siguiente alarma		

Figura 2-4. Algunos de los campos de una entrada típica en la tabla de procesos.

La **ilusión de varios procesos secuenciales** en una o varias CPU se mantiene mediante las **interrupciones**.
Cada clase de **E/S** tiene una ubicación asociada en memoria llamada **vector de interrupción**, que contiene la **dirección del procedimiento de servicio de interrupciones**.

1. El hardware mete el contador del programa a la pila, etc.
2. El hardware carga el nuevo contador de programa del vector de interrupciones.
3. Procedimiento en lenguaje ensamblador guarda los registros.
4. Procedimiento en lenguaje ensamblador establece la nueva pila.
5. El servicio de interrupciones de C se ejecuta (por lo general lee y guarda la entrada en el búfer).
6. El planificador decide qué proceso se va a ejecutar a continuación.
7. Procedimiento en C regresa al código de ensamblador.
8. Procedimiento en lenguaje ensamblador inicia el nuevo proceso actual.

Figura 2-5. Esqueleto de lo que hace el nivel más bajo del sistema operativo cuando ocurre una interrupción.

HILOS

En los sistemas operativos tradicionales, cada proceso tiene **un espacio** de **direcciones** y **un solo hilo de control**. De hecho, ésa es casi la definición de un proceso. Sin embargo, con frecuencia hay situaciones en las que es conveniente tener **varios hilos** de control en el mismo espacio de direcciones que se ejecuta en cuasi-paralelo
Uso de hilos

Los **hilos** son “miniprocesos” dentro de un proceso. Se usan porque muchas aplicaciones realizan **varias actividades a la vez**, algunas de las cuales pueden **bloquearse**. Al dividir una aplicación en varios hilos secuenciales que se ejecutan en **cuasi-paralelo**, el **modelo de programación se simplifica**.

A diferencia de los procesos, los **hilos comparten el mismo espacio de direcciones y los mismos datos**, lo cual es esencial para ciertas aplicaciones. Además, los hilos son **más**

ligeros, se **crean y destruyen más rápido** que los procesos, y permiten **traslapar cálculo y E/S**, mejorando el rendimiento cuando hay operaciones de disco.

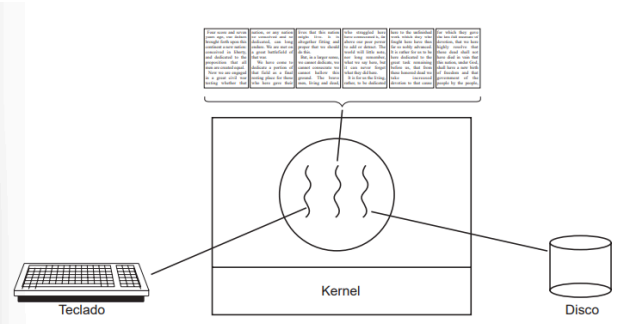


Figura 2-7. Un procesador de palabras con tres hilos.

En sistemas con **varias CPU**, los hilos permiten un paralelismo **real**.

Ejemplo:

- En un **procesador de palabras**, un hilo interactúa con el usuario, otro vuelve a dar formato en segundo plano y otro realiza **respaldos automáticos**, manteniendo buena respuesta.
- En un **servidor Web**, un hilo despachador recibe solicitudes y las asigna a **hilos trabajadores**, que comparten una **caché** y se bloquean si deben acceder al disco.

Los hilos permiten pensar los programas como **actividades paralelas simples**, algo que no sería posible usando procesos separados porque **no compartirían memoria**.

Un **servidor Web sin hilos** puede funcionar con **un solo hilo**, procesando una petición a la vez. Mientras espera al disco, la **CPU queda inactiva**, se procesan menos peticiones por segundo y el **rendimiento es bajo**.

Si no hay hilos, pero se quiere alto rendimiento, se puede usar **E/S sin bloqueo**. El servidor guarda el **estado de cada petición en una tabla** y reacciona a eventos (peticiones nuevas o respuestas del disco). Este diseño es una **máquina de estados finitos**, logra paralelismo, pero es **difícil de programar**.

Los **hilos** combinan lo mejor: **paralelismo** con **llamadas al sistema con bloqueo**, lo que **simplifica la programación** y **mejora el rendimiento**.

Comparación:

- **Hilos**: paralelismo + llamadas con bloqueo
- **Un solo hilo**: sin paralelismo + llamadas con bloqueo
- **Máquina de estados finitos**: paralelismo + llamadas sin bloqueo e interrupciones

Los hilos también son útiles para **procesar grandes volúmenes de datos**, separando **entrada, procesamiento y salida** en distintos hilos, permitiendo que todo ocurra al mismo tiempo sin dejar la CPU inactiva.

Modelo clásico de hilos

El **modelo clásico de hilos** separa dos conceptos del proceso: **agrupamiento de recursos** y **ejecución**.
El **proceso** agrupa recursos (espacio de direcciones, archivos abiertos, señales, procesos hijos, etc.), mientras que el **hilo** es la unidad de **ejecución**, con su **contador de programa, registros y pila**.

Los **hilos permiten varias ejecuciones dentro del mismo proceso**, compartiendo el **mismo espacio de direcciones y recursos**. Por eso se los llama **procesos ligeros**. A diferencia de varios procesos, los hilos **cooperan** y no tienen protección entre ellos.

Un proceso puede tener **uno o varios hilos**:

- Varios procesos → cada uno con su propio espacio de direcciones.
- Un proceso multihilo → varios hilos **comparten memoria**.

En una sola CPU, los hilos **se turnan** para ejecutarse, dando la **ilusión de paralelismo**. Cada hilo tiene su **propia pila**, necesaria porque cada uno tiene un historial de ejecución distinto.

Los hilos pueden estar en los mismos estados que los procesos: **ejecución, bloqueado, listo o terminado**.
Se crean con llamadas como **thread_create**, terminan con **thread_exit**, pueden esperar a otros con **thread_join** y ceder la CPU con **thread_yield**.

Aunque son muy útiles, los hilos **complican la programación**:

- Problemas con **fork** en procesos con varios hilos.
- Conflictos por **datos compartidos** (archivos, memoria, recursos).

Por eso, los programas multihilo requieren un diseño **cuidadoso** para funcionar correctamente.

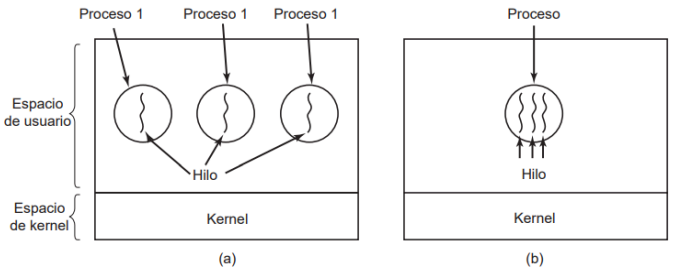


Figura 2-11. (a) Tres procesos, cada uno con un hilo. (b) Un proceso con tres hilos.

Elementos por proceso	Elementos por hilo
Espacio de direcciones	Contador de programa
Variables globales	Registros
Archivos abiertos	Pila
Procesos hijos	Estado
Alarmas pendientes	
Señales y manejadores de señales	
Información contable	

Al igual que un proceso tradicional (es decir, un proceso con sólo un hilo), un hilo puede estar en uno de varios **estados**: en ejecución, bloqueado, listo o terminado.
Implementación de hilos en el espacio de usuario

Hay **dos formas principales de implementar hilos**: en **espacio de usuario** o en el **kernel** (también existe un modelo híbrido).

- En los **hilos de nivel usuario**, el kernel **no sabe que existen** y ve sólo procesos con un hilo. Los hilos se implementan mediante una **biblioteca** y un **sistema en tiempo de ejecución**, que mantiene una **tabla de hilos** por proceso.
  La **ventaja principal** es el **alto rendimiento**: la creación, planificación y conmutación de hilos es **muy rápida**, sin traps al kernel. Además, cada proceso puede usar su **propio algoritmo de planificación** y escalan mejor cuando hay muchos hilos.

Las **desventajas** son importantes:

- Una **llamada al sistema con bloqueo** bloquea **todo el proceso**, no sólo el hilo.
- Los **fallos de página** también bloquean a todos los hilos.
- No hay **interrupciones de reloj**, por lo que un hilo puede quedarse con la CPU si no la cede voluntariamente.
- Para evitar bloqueos se necesitan **llamadas sin bloqueo**, `select` y **envolturas**, lo que vuelve el código **complejo e ineficiente**.

En resumen:
Los **hilos de nivel usuario** son **rápidos**, pero **difíciles de usar correctamente**, y funcionan mal en aplicaciones que hacen **muchas llamadas al sistema**, que son justamente donde más se desean los hilos.

Implementación de hilos en el kernel

- En los **hilos de nivel kernel**, el **kernel conoce y administra los hilos**. Mantiene una **tabla global de hilos**, con su estado, registros y demás información. Crear o destruir hilos requiere **llamadas al sistema**, lo que implica **mayor costo** que en hilos de nivel usuario.

Ventajas principales:

- Si un hilo se **bloquea** o ocurre un **fallo de página**, el kernel puede ejecutar **otro hilo del mismo proceso o de otro proceso**.
- No se necesitan llamadas al sistema **sin bloqueo** ni envolturas.

- Funcionan bien en aplicaciones con **muchas llamadas al sistema**.

Desventaja principal:

- **Alta sobrecarga**: crear, destruir y manejar hilos es más lento porque todo pasa por el kernel. Para reducir esto, algunos sistemas **reciclan hilos**.

Los hilos de kernel **no resuelven todos los problemas**, como:

- Qué hacer con los hilos al usar **fork**.
- Cómo manejar las **señales**, que se envían a procesos y no a hilos.

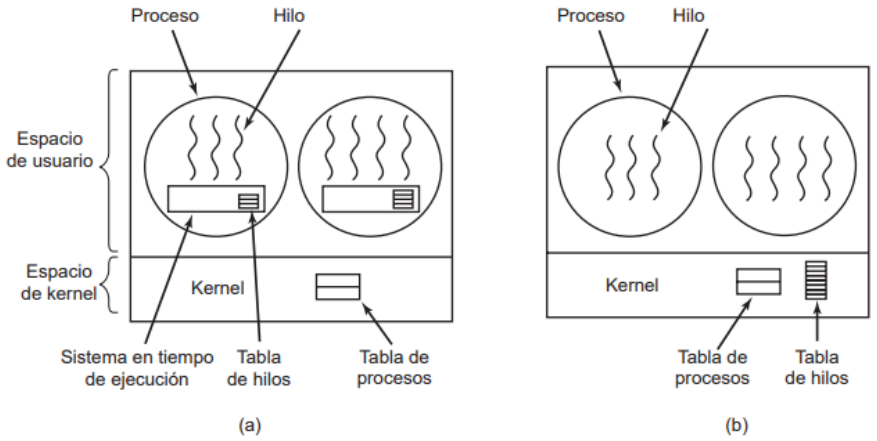


Figura 2-16. (a) Un paquete de hilos de nivel usuario. (b) Un paquete de hilos administrado por el kernel.

Implementaciones híbridas:

- Combinan **hilos de kernel** con **hilos de usuario**.
  El kernel planifica los hilos de kernel, y **cada hilo de kernel puede ejecutar varios hilos de usuario multiplexados**.
  Este modelo ofrece **máxima flexibilidad**, combinando rendimiento y control.

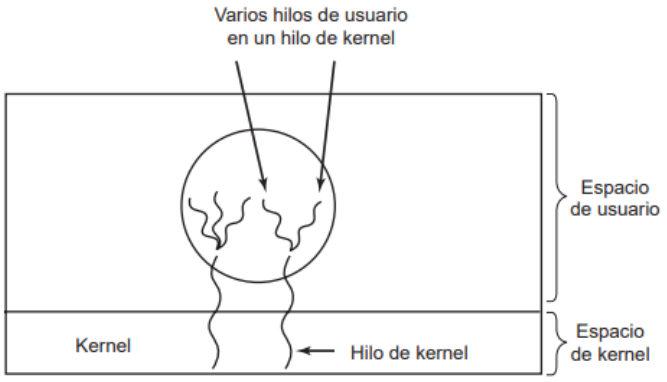


Figura 2-17. Multiplexaje de hilos del nivel usuario sobre hilos del nivel kernel.

Activaciones del planificador

Las **activaciones del planificador** buscan combinar lo mejor de los **hilos de kernel** y los **hilos de nivel usuario**: mantener el **paralelismo real y el manejo correcto de bloqueos**, pero con **menor sobrecarga y mayor eficiencia**.

La idea central es que el **kernel asigna procesadores virtuales** a cada proceso y el **sistema en tiempo de ejecución (en espacio de usuario)** decide qué hilos ejecutar en ellos.
Cuando un hilo se **bloquea** (llamada al sistema o fallo de página), el kernel **notifica al sistema en tiempo de ejecución** mediante una **llamada ascendente (upcall)**, permitiéndole **replanificar otro hilo** sin bloquear todo el proceso.

Cuando el evento que causó el bloqueo se resuelve, el kernel vuelve a avisar con otra **upcall**, y el sistema decide **reanudar el hilo o dejarlo listo**.
Esto evita transiciones innecesarias entre **usuario y kernel**, mejorando el rendimiento.

La **principal crítica** es que las **llamadas ascendentes rompen el modelo clásico por capas**, ya que el kernel llama a código de nivel superior.

Hilos emergentes

Las **activaciones del planificador** buscan combinar lo mejor de los **hilos de kernel** y los **hilos de nivel usuario**:
mantener el **paralelismo real y el manejo correcto de bloqueos**, pero con **menor sobrecarga y mayor eficiencia**.

La idea central es que el **kernel asigna procesadores virtuales** a cada proceso y el **sistema en tiempo de ejecución (en espacio de usuario)** decide qué hilos ejecutar en ellos.
Cuando un hilo se **bloquea** (llamada al sistema o fallo de página), el kernel **notifica al sistema en tiempo de ejecución** mediante una **llamada ascendente (upcall)**, permitiéndole **replanificar otro hilo** sin bloquear todo el proceso.

Cuando el evento que causó el bloqueo se resuelve, el kernel vuelve a avisar con otra **upcall**, y el sistema decide **reanudar el hilo o dejarlo listo**.
Esto evita transiciones innecesarias entre **usuario y kernel**, mejorando el rendimiento.

La **principal crítica** es que las **llamadas ascendentes rompen el modelo clásico por capas**, ya que el kernel llama a código de nivel superior.

Los **hilos emergentes** son otra técnica usada en sistemas distribuidos:
La llegada de un mensaje **crea automáticamente un nuevo hilo** para procesarlo. Como el hilo empieza desde cero, se **reduce la latencia**.
Pueden ejecutarse en **kernel** (más rápido y potente, pero más riesgoso) o en **usuario** (más seguro).

Los **hilos emergentes** son otra técnica usada en sistemas distribuidos:
La llegada de un mensaje **crea automáticamente un nuevo hilo** para procesarlo. Como el hilo empieza desde cero, se **reduce la latencia**.

Pueden ejecutarse en **kernel** (más rápido y potente, pero más riesgoso) o en **usuario** (más seguro).

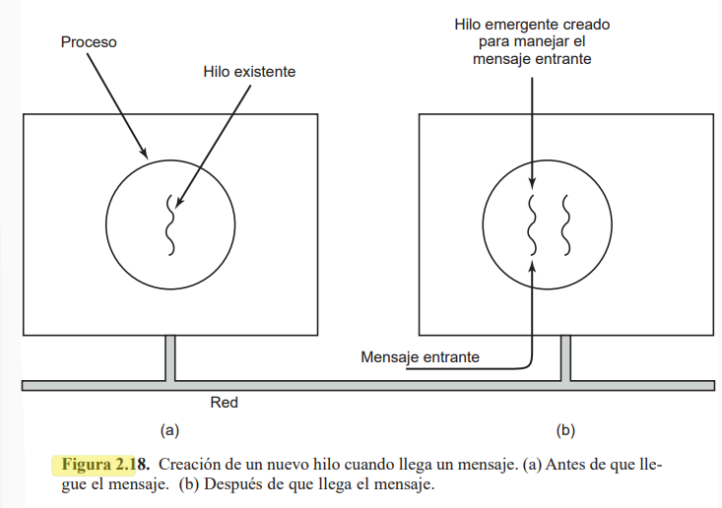
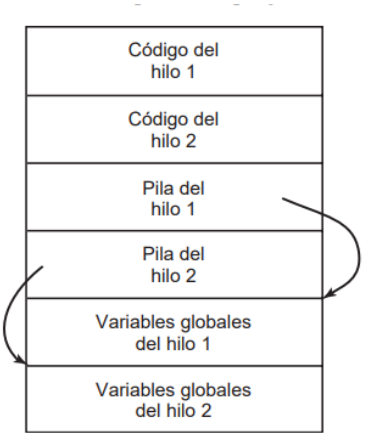


Figura 2.18. Creación de un nuevo hilo cuando llega un mensaje. (a) Antes de que llegue el mensaje. (b) Después de que llega el mensaje.

Convertir un programa de **un solo hilo a multihilo es difícil**.
Los principales problemas son:



- **Variables globales** (como `errno`) generan **conflictos entre hilos**.
- Muchas **bibliotecas no son reentrantes** y fallan con varios hilos.
- La **gestión de señales** se vuelve ambigua y compleja.
- La **administración de múltiples pilas** no está bien soportada si el kernel no conoce los hilos.

Conclusión:
Agregar hilos **no es automático**; requiere **rediseñar bibliotecas, semántica de llamadas al sistema y manejo de recursos**, manteniendo compatibilidad con programas existentes.

Comunicación entre procesos

Con frecuencia, los **procesos necesitan comunicarse con otros procesos**. Un ejemplo típico es una **canalización del shell**, donde la salida de un proceso se pasa al siguiente.
Esta necesidad se denomina **comunicación entre procesos (IPC)** y se busca que sea **bien estructurada**, sin usar interrupciones.

Existen **tres cuestiones fundamentales** en la IPC:

1. **Cómo un proceso pasa información a otro**.

2. **Cómo evitar que los procesos se interpongan entre sí** al usar recursos compartidos.
3. **Cómo asegurar la secuencia correcta** cuando hay dependencias (por ejemplo, que un proceso espere a que otro produzca datos).

Estas tres cuestiones se analizarán en las secciones siguientes.

Aunque se estudian en el contexto de los **procesos**, **las dos últimas también se aplican a los hilos**.
La primera (paso de información) es más sencilla en hilos porque **comparten el espacio de direcciones**, pero **la exclusión mutua y la sincronización** presentan los mismos problemas y requieren las mismas soluciones.

Condiciones de carrera

En algunos sistemas operativos, los procesos que cooperan pueden **compartir almacenamiento**, ya sea en **memoria principal** o en **archivos compartidos**.
La ubicación no cambia la naturaleza del problema.

Ejemplo: **spooler de impresión**.

- Un proceso coloca el nombre del archivo a imprimir en un directorio compartido.
- Un demonio de impresión revisa ese directorio y procesa los archivos.

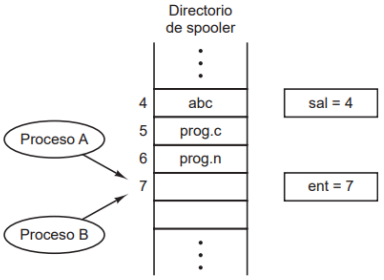


Figura 2-21. Dos procesos desean acceder a la memoria compartida al mismo tiempo.

Supóngase que:

- `sal` apunta al **siguiente archivo a imprimir**.
- `ent` apunta a la **siguiente ranura libre** del directorio.

Si **dos procesos acceden simultáneamente** a `ent`, ambos pueden creer que la misma ranura está libre y **sobrescribir información**, provocando que **uno de los trabajos se pierda**, sin que el sistema detecte el error.

Este tipo de situación, donde:

- dos o más procesos **leen o escriben datos compartidos**, y
- el resultado **depende del orden y el momento de ejecución**,

se denomina **condición de carrera**.

Las condiciones de carrera son **difíciles de depurar** porque la mayoría de las veces el programa funciona bien, pero **ocasionalmente falla de manera inexplicable**.

Regiones críticas

Para evitar condiciones de carrera, es necesario impedir que **más de un proceso acceda al mismo tiempo** a los datos compartidos.
Esto se logra mediante **exclusión mutua**.

La **región crítica** (o **sección crítica**) es la parte del programa donde un proceso:

- accede a **memoria compartida**,
- archivos compartidos,
- o cualquier recurso que pueda producir condiciones de carrera.

Si se garantiza que **nunca haya dos procesos simultáneamente en sus regiones críticas**, se evitan las condiciones de carrera.

Sin embargo, una solución correcta debe cumplir **cuatro condiciones**:

1. **No puede haber dos procesos al mismo tiempo en sus regiones críticas.**
2. **No deben hacerse suposiciones sobre velocidades ni cantidad de CPUs.**
3. **Un proceso fuera de su región crítica no debe bloquear a otros.**
4. **Ningún proceso debe esperar indefinidamente para entrar a su región crítica.**

El comportamiento deseado es:

- Un proceso entra a su región crítica.
- Otro proceso que intenta entrar **debe esperar**.
- Cuando el primero sale, el segundo **entra inmediatamente**.

De esta forma, los procesos **cooperan correctamente**, sin interferencias ni bloqueos injustos.

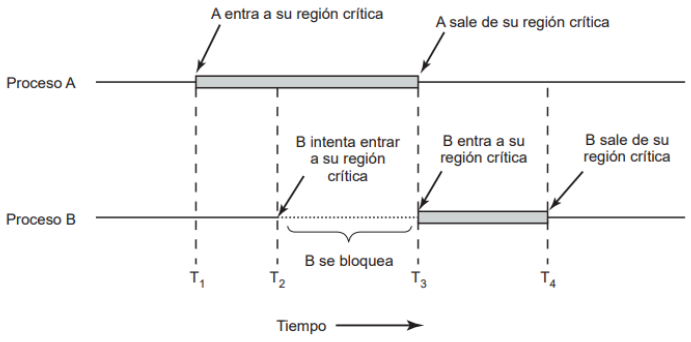


Figura 2-22. Exclusión mutua mediante el uso de regiones críticas.

Exclusión mutua con espera ocupada

En esta sección se analizan **propuestas para lograr exclusión mutua**, de modo que **mientras un proceso esté en su región crítica**, ningún otro proceso pueda entrar y provocar **condiciones de carrera**.

Las soluciones presentadas utilizan **espera ocupada** (busy waiting), es decir, procesos que **esperan activamente** hasta poder entrar.

Deshabilitando interrupciones

En un sistema con **un solo procesador**, la solución más simple es que un proceso **deshabilite las interrupciones** al entrar a su región crítica y las **rehabilite al salir**.

- Sin interrupciones, **no hay conmutación de procesos**, porque la CPU sólo cambia de proceso por una interrupción.
- Así, el proceso puede **examinar y actualizar memoria compartida sin interferencias**.

Problemas:

- **No es seguro** permitir que procesos de usuario deshabiliten interrupciones: si no las reactivan, el sistema puede quedar inutilizado.
- En **sistemas multiprocesador**, deshabilitar interrupciones **sólo afecta a una CPU**; las demás siguen ejecutándose y pueden acceder a la memoria compartida.
- Por eso, **no es una solución general válida** para procesos de usuario.

Conclusión:

- Deshabilitar interrupciones es **útil dentro del kernel**, para proteger estructuras internas.
- **No es apropiado como mecanismo general de exclusión mutua**, y su utilidad disminuye aún más con los **sistemas multinúcleo**.

Variables de candado

Se propone una solución puramente de **software** usando una **variable compartida llamada candado**:

- `candado = 0` → nadie está en la región crítica
- `candado = 1` → algún proceso está en la región crítica

Un proceso:

1. Verifica el candado.
2. Si es 0, lo pone en 1 y entra.
3. Si es 1, espera hasta que vuelva a 0.

Problema:

Esta solución tiene **exactamente el mismo error que el spooler**:

- Dos procesos pueden leer el candado como 0 **al mismo tiempo**.
- Ambos lo fijan en 1 y **entran simultáneamente** a la región crítica.

Verificar el candado más de una vez **no resuelve el problema**, porque la condición de carrera puede ocurrir entre verificaciones.

Conclusión:

- **No garantiza exclusión mutua**.
- No es una solución válida.

Alternancia estricta

Se introduce una variable compartida `turno`, que indica **qué proceso puede entrar** a la región crítica.

- Cada proceso espera hasta que `turno` tenga su valor.
- Al salir de la región crítica, cambia `turno` para ceder el acceso al otro proceso.
- El proceso que no tiene el turno **espera activamente** (espera ocupada).

La **espera ocupada** consiste en evaluar continuamente una variable hasta que cambie.
Un candado que usa este método se llama **candado de giro**.

Problema principal:

Aunque **evita condiciones de carrera**, **viola la condición 3** de las regiones críticas:

> *Un proceso fuera de su región crítica no debe bloquear a otros.*

Si un proceso es lento o está en su región no crítica, el otro proceso puede quedar **bloqueado innecesariamente**, aunque nadie esté usando la región crítica.

Consecuencia:

- Los procesos deben **alternarse estrictamente**, incluso cuando no es necesario.
- Un proceso no puede entrar dos veces seguidas a su región crítica.
- Esto provoca **ineficiencia y bloqueos injustificados**.

Conclusión:

- Aunque es correcta desde el punto de vista de exclusión mutua,
  no es una solución práctica ni aceptable, porque **bloquea procesos sin motivo**.

Solución de Peterson

La **solución de Peterson** es un **algoritmo de software** para lograr **exclusión mutua sin alternancia estricta**.

Fue propuesta por **G. L. Peterson (1981)** y reemplazó a la solución más compleja de **Dekker**.

Utiliza:

- una variable **turno**, y
- un arreglo **interesado[]** que indica si un proceso quiere entrar a la región crítica.

Antes de entrar a la **región crítica**, cada proceso:

1. Indica que está interesado.
2. Establece el turno.
3. Espera mientras el otro proceso esté interesado **y** sea su turno.

Al salir de la región crítica, el proceso:

- marca que **ya no está interesado**.

Propiedades importantes:

- Garantiza **exclusión mutua**.
- No requiere **alternancia estricta**.
- Funciona incluso si ambos procesos intentan entrar **casi simultáneamente**.
- Utiliza **espera ocupada**.

Cuando ambos procesos quieren entrar:

- El último en escribir `turno` cede el acceso al otro.
- Uno entra a la región crítica y el otro espera.

La instrucción TSL

La instrucción **TSL (Test and Set Lock)** es una **instrucción de hardware** que:

- lee el valor de una variable compartida (candado),
- la guarda en un registro,
- y luego **la fija en un valor distinto de cero**,
  todo de manera **indivisible (atómica)**.

Durante la ejecución de TSL:

- se **bloquea el bus de memoria**,
- ningún otro procesador puede acceder a esa variable.

Esto es **muy distinto** de deshabilitar interrupciones:

- deshabilitar interrupciones **no evita** que otros procesadores accedan a la memoria.

Uso de TSL:

- Si `candado = 0`, el proceso lo fija en 1 y entra a la región crítica.
- Si `candado ≠ 0`, el proceso **espera ocupadamente** hasta que se libere.

Al salir de la región crítica:

- el proceso vuelve a poner `candado = 0`.

Características:

- Solución **simple y directa**.
- Requiere **espera ocupada**.
- La exclusión mutua depende de que los procesos **llamen correctamente** a entrar y salir de la región crítica.

La instrucción XCHG

La instrucción **XCHG** intercambia de forma **atómica**:

- el contenido de un registro,
- con una palabra de memoria (el candado).

Funciona de manera equivalente a **TSL**:

- si el valor previo del candado era 0, el proceso entra,
- si no, sigue esperando.

Nota importante:

- Las CPUs **Intel x86** utilizan **XCHG** para sincronización de bajo nivel.

2.3.4 Dormir y despertar

Las soluciones anteriores (**Peterson, TSL, XCHG**) son correctas, pero **todas usan espera ocupada**, lo cual:

- desperdicia CPU,
- puede causar efectos inesperados.

Inversión de prioridades

Si un proceso de **alta prioridad** entra en espera ocupada mientras uno de **baja prioridad** está en la región crítica:

- el proceso de baja prioridad **nunca se ejecuta**,
- el de alta prioridad **espera indefinidamente**.

Esto se conoce como **inversión de prioridades**.

Primitivas sleep y wakeup

Para evitar la espera ocupada, se introducen primitivas que **bloquean procesos**:

- **sleep**: el proceso se bloquea (queda inactivo).
- **wakeup**: despierta a un proceso bloqueado.

Estas primitivas permiten que un proceso **no consuma CPU** mientras espera.

El problema del productor–consumidor

Dos procesos comparten un **búfer limitado**:

- el **productor** inserta elementos,
- el **consumidor** los retira.

Reglas:

- Si el búfer está **lleno**, el productor se duerme.
- Si el búfer está **vacío**, el consumidor se duerme.
- Cada uno despierta al otro cuando corresponde.

Para controlar el búfer se usa una variable compartida **cuenta**.

Condición de carrera en sleep/wakeup

El acceso a **cuenta** no está protegido, lo que produce una **condición de carrera fatal**.

Puede ocurrir que:

- el productor envíe un **wakeup** cuando el consumidor **todavía no está dormido**,
- la señal se **pierda**,
- luego ambos procesos se duerman **para siempre**.

Problema central:

> Una señal de despertar enviada a un proceso que aún no duerme **se pierde**.

Bit de espera de despertar

Una solución parcial es agregar un **bit de espera de despertar**:

- si llega un wakeup y el proceso no está dormido, se guarda la señal.
- cuando el proceso intenta dormir, consume ese bit y **no se duerme**.

Limitación:

- Funciona en casos simples.
- **No escala** bien con múltiples procesos.
- El problema fundamental **sigue existiendo**.

2.3.5 Semáforos

En 1965, **E. W. Dijkstra** propuso una solución general al problema de la **pérdida de señales de despertar** introduciendo una nueva variable: el **semáforo**.

Un **semáforo** es una **variable entera** que:

- vale **0** si no hay señales de despertar almacenadas,
- vale un **número positivo** si hay una o más señales pendientes.

Operaciones sobre semáforos

Dijkstra definió dos operaciones:

- **down** (generalización de *sleep*)
- **up** (generalización de *wakeup*)

Operación down

- Si el valor del semáforo es mayor que 0, lo decrementa y continúa.
- Si el valor es 0, el proceso se **bloquea**.
- La comprobación, modificación y bloqueo se realizan como una **acción atómica indivisible**.

Operación up

- Incrementa el valor del semáforo.
- Si hay procesos bloqueados, **despierta uno** de ellos.
- También es una operación **atómica**.
- Ningún proceso se bloquea al ejecutar up.

La **atomicidad** es esencial para evitar **condiciones de carrera**.

Semáforos y productor–consumidor

Los semáforos resuelven el problema del productor–consumidor **sin pérdida de señales**.

Se utilizan **tres semáforos**:

- **llenas**: cuenta ranuras ocupadas
- **vacías**: cuenta ranuras libres
- **mutex**: asegura exclusión mutua

Valores iniciales:

- llenas = 0
- vacías = N
- mutex = 1

El semáforo **mutex** es un **semáforo binario** y se usa para **exclusión mutua**.
Los semáforos **llenas** y **vacías** se usan para **sincronización**.

Dos usos distintos del semáforo

1. **Exclusión mutua** → mutex
2. **Sincronización de eventos** → llenas y vacías

Semáforos e interrupciones

En sistemas con E/S:

- Cada dispositivo puede tener un **semáforo asociado**.
- El proceso realiza **down** y se bloquea.
- El manejador de interrupciones realiza **up** y despierta al proceso.

Así, las interrupciones quedan **ocultas** mediante semáforos.

2.3.6 Mutexes

Un **mutex** es una versión **simplificada** del semáforo:

- sólo sirve para **exclusión mutua**,
- **no cuenta señales**.

Un mutex puede estar:

- **abierto** (0)
- **cerrado** (≠ 0)

Operaciones

- **mutex_lock**: adquiere el mutex o se bloquea.
- **mutex_unlock**: libera el mutex y despierta a un hilo.

Los mutexes son:

- simples,
- eficientes,
- ideales para **hilos de nivel usuario**.

Diferencia clave con espera ocupada

En hilos de usuario:

- no hay reloj que fuerce el cambio de hilo
- la espera ocupada puede bloquear el sistema.

Por eso, **mutex_lock** llama a **thread_yield**:

- cede la CPU a otro hilo,
- evita espera ocupada.

Todo ocurre **en espacio de usuario**, sin llamadas al kernel.

Mutexes y memoria compartida

Para que mutexes, semáforos o Peterson funcionen entre procesos:

- debe existir **memoria compartida**.

Opciones:

1. estructuras en el **kernel** accesibles por llamadas al sistema,
2. **segmentos de memoria compartida** entre procesos,
3. archivos compartidos (peor caso).

Aunque procesos compartan memoria, **no son hilos**:

- los procesos siguen teniendo recursos separados.

Mutexes en Pthreads

Pthreads ofrece mutexes con las siguientes operaciones:

- `pthread_mutex_init`
- `pthread_mutex_destroy`
- `pthread_mutex_lock`
- `pthread_mutex_trylock`
- `pthread_mutex_unlock`

Los mutexes **no son obligatorios**:

- el programador debe usarlos correctamente.

Variables de condición (Pthreads)

Las **variables de condición** permiten que un hilo:

- se bloquee hasta que **una condición se cumpla**.

Operaciones principales:

- **pthread_cond_wait**
- **pthread_cond_signal**
- **pthread_cond_broadcast**

Características clave:

- siempre se usan **junto con mutexes**,
- `pthread_cond_wait` libera y recupera el mutex **en forma atómica**,
- **no tienen memoria**: si no hay hilos esperando, la señal se pierde.

2.4 PLANIFICACIÓN

Cuando una computadora se multiprograma, existen **varios procesos o hilos en estado listo** que compiten por la CPU.
Si hay **una sola CPU**, es necesario decidir **cuál se ejecuta a continuación**.

- El **planificador de procesos** es la parte del sistema operativo que toma esa decisión.

- El **algoritmo de planificación** es el criterio que utiliza el planificador para decidir.

Muchas de las cuestiones de planificación aplican tanto a **procesos como a hilos**.
Cuando el kernel administra hilos, la planificación suele hacerse **por hilo**, sin importar demasiado a qué proceso pertenece.

En esta sección se analizan primero los conceptos generales y luego los **algoritmos concretos**, distinguiendo distintos tipos de sistemas.

Un buen planificador **impacta directamente** en:

- rendimiento del sistema
- tiempo de respuesta
- satisfacción del usuario

Computadoras personales

En las PCs modernas:

- Normalmente hay **un solo proceso activo**
- La CPU **no suele ser un recurso escaso**
- La planificación **no es crítica** en la mayoría de los casos

Existen excepciones (video, multimedia, cómputo intensivo), pero **no son la regla**.

Servidores

En **servidores en red**, la situación cambia:

- Muchos procesos compiten constantemente por la CPU
- La planificación vuelve a ser **muy importante**
- Se priorizan procesos que atienden usuarios frente a tareas en segundo plano

Costo de la conmutación de procesos

Cambiar de proceso **no es gratis**:

- cambio de modo usuario → kernel
- guardado de registros y estado
- cambio de mapas de memoria
- posible invalidación de caché

Demasiadas conmutaciones degradan el rendimiento, por lo que el planificador debe ser cuidadoso.

Comportamiento de un proceso

Los procesos **alternan** entre:

- **ráfagas de CPU**
- **esperas por E/S**

Esto da lugar a dos tipos de procesos:

Procesos limitados a cálculos (CPU-bound)

- Ráfagas de CPU **largas**
- Esperas de E/S **poco frecuentes**

Procesos limitados a E/S (I/O-bound)

- Ráfagas de CPU **cortas**
- Esperas de E/S **frecuentes**

Lo importante es la **duración de la ráfaga de CPU**, no la de la E/S.

A medida que las CPUs son más rápidas que los discos:

- los procesos tienden a ser **más limitados a E/S**
- planificarlos bien es cada vez más importante

Cuándo planificar procesos

El planificador debe tomar decisiones en varias situaciones:

1. **Creación de un proceso**
    - decidir si se ejecuta el padre o el hijo
2. **Terminación de un proceso**
    - elegir otro proceso listo o uno inactivo
3. **Bloqueo de un proceso**
    - por E/S, semáforos, etc.
4. **Interrupciones de E/S**
    - decidir si ejecutar el proceso desbloqueado o continuar con el actual
5. **Interrupciones de reloj**
    - permiten o no la apropiación

Planificación no apropiativa

- El proceso se ejecuta **hasta que se bloquea o libera la CPU**
- No se lo interrumpe por tiempo
- Más simple, menos sobrecarga

Planificación apropiativa

- Cada proceso tiene un **tiempo máximo**
- Si se agota, se suspende
- Requiere interrupciones de reloj

Categorías de los algoritmos de planificación

Según el tipo de sistema:

1. Procesamiento por lotes

- No hay usuarios esperando
- Se aceptan algoritmos no apropiativos
- Se busca eficiencia global

2. Sistemas interactivos

- Usuarios esperan respuestas rápidas
- La apropiación es **esencial**
- Incluye servidores

3. Sistemas de tiempo real

- Existen **tiempos límite**
- Importa cumplir plazos y la predictibilidad
- No se ejecutan programas arbitrarios

Metas de los algoritmos de planificación

En todos los sistemas

- **Equidad**: procesos similares deben recibir servicio similar
- **Aplicación de políticas**: cumplir prioridades del sistema
- **Balance**: mantener CPU y E/S ocupados

Procesamiento por lotes

- **Rendimiento**: trabajos por hora
- **Tiempo de retorno**: desde envío hasta terminación
- **Utilización de la CPU** (métrica secundaria)

Sistemas interactivos

- **Tiempo de respuesta**
- **Proporcionalidad** (cumplir expectativas del usuario)

Sistemas de tiempo real

- **Cumplir plazos**
- **Predictibilidad**

2.4.2 Planificación en sistemas de procesamiento por lotes

Primero en entrar, primero en ser atendido (FCFS)

- Algoritmo **no apropiativo**
- Se ejecutan los procesos **en orden de llegada**
- Implementación simple: una cola

Ventaja

- Fácil de entender
- Equitativo en sentido formal

Desventaja

- Perjudica procesos limitados a E/S
- Un proceso largo puede retrasar muchos cortos

Trabajo más corto primero (SJF)

- Algoritmo **no apropiativo**
- Supone conocer el tiempo de ejecución
- Ejecuta primero el trabajo más corto

Resultado

- Minimiza el **tiempo promedio de retorno**

- Es óptimo **solo si todos los trabajos llegan juntos**

Menor tiempo restante a continuación (SRTN)

- Versión **apropiativa** de SJF
- Siempre ejecuta el proceso con **menor tiempo restante**
- Un proceso nuevo puede desplazar al actual

Ventaja

- Excelente tiempo de respuesta para trabajos cortos
- Penaliza menos a procesos interactivos



Figura 2-41. Planificación de turno circular. (a) La lista de procesos ejecutables. (b) La lista de procesos ejecutables una vez que *B* utiliza su quántum.

:

2.4.3 Planificación en sistemas interactivos

Los **sistemas interactivos** (PCs, servidores, sistemas multiusuario) necesitan **tiempos de respuesta rápidos**. El objetivo principal no es maximizar el rendimiento total, sino **responder rápido a acciones cortas del usuario**.

Planificación por turno circular (Round-Robin)

Es uno de los algoritmos **más usados, simples y equitativos**.

- A cada proceso se le asigna un **intervalo de tiempo fijo**, llamado **quántum**.
- El proceso se ejecuta **hasta que:
  - termina,
  - se bloquea,
  - o se agota su quántum**.
- Si se agota el quántum, el proceso se **interrumpe** y se coloca **al final de la cola** de procesos listos.

Idea clave: todos los procesos obtienen CPU por turnos.

El problema del tamaño del quántum

- **Quántum muy corto**
  → muchas conmutaciones de proceso
  → mucho tiempo perdido en sobrecarga administrativa
  → baja eficiencia de la CPU

- **Quántum muy largo**
  → mala respuesta a procesos interactivos cortos
  → un proceso puede esperar varios segundos antes de ejecutarse

Conclusión:
Un quántum **intermedio (20–50 ms)** suele ser una solución razonable.

Planificación por prioridad

No todos los procesos tienen la misma importancia.

- A cada proceso se le asigna una **prioridad**.
- **Siempre se ejecuta el proceso listo con mayor prioridad**.

Ejemplos:

- Un proceso que reproduce video en tiempo real tiene más prioridad que uno que envía correos en segundo plano.
- En sistemas multiusuario, distintos usuarios pueden tener prioridades distintas.

Evitar monopolios de CPU

Para evitar que un proceso de alta prioridad se ejecute indefinidamente:

- Se puede **reducir su prioridad con el tiempo**, o
- Limitar el tiempo máximo que puede ejecutarse (quántum).

Prioridades estáticas y dinámicas

- **Estáticas**: asignadas desde el inicio (ej.: jerarquías, costos, niveles).
- **Dinámicas**: cambian según el comportamiento del proceso.

Ejemplo importante:

- Los procesos **limitados a E/S** deben recibir la CPU rápido para iniciar su próxima operación.
- Una política simple es asignar prioridad proporcional a **1/f**, donde *f* es la fracción del quántum utilizado.

Múltiples colas de prioridad

Se agrupan procesos en **clases de prioridad**.

- El planificador:
  - ejecuta primero todos los procesos de la **clase más alta**,
  - usando **turno circular dentro de cada clase**.

- Las clases de menor prioridad **no se ejecutan** mientras haya procesos en clases superiores.

Problema:
Si no se reajustan prioridades, los procesos de baja prioridad pueden **morir de hambre** (starvation).

Históricamente:

- Sistemas como **CTSS** y **XDS 940** usaron múltiples colas para:
    - favorecer procesos interactivos,
    - reducir el costo de conmutación,
    - penalizar procesos muy largos.

El proceso más corto a continuación

Busca aplicar la idea de **trabajo más corto primero** a sistemas interactivos.

- Cada comando interactivo se considera un "trabajo".
- Se ejecuta el proceso con **menor tiempo estimado de ejecución**.

Como el tiempo real no se conoce:

- Se estima usando **promedios ponderados** de ejecuciones anteriores.
- Técnica conocida como **envejecimiento (aging)**.

Muy importante:

- Se usa el comportamiento pasado para predecir el futuro.
- Favorece comandos cortos y mejora el tiempo de respuesta.

Planificación garantizada

El sistema hace una **promesa explícita**:

> "Si hay n procesos activos, cada uno recibirá aproximadamente 1/n del tiempo de CPU."

- El sistema mide:
    - cuánto CPU recibió cada proceso,
    - cuánto debería haber recibido.
- Se ejecuta el proceso que **menos ha recibido en proporción a lo que le corresponde**.

Planificación por sorteo (lotería)

Cada proceso recibe **boletos de lotería**.

- En cada decisión de planificación:
    - se sortea un boleto,
    - el proceso ganador obtiene CPU.

Características clave:

- Más boletos → más probabilidad de ejecutarse.
- Fácil de entender y de implementar.
- Muy flexible.

Ventajas importantes:

- Alta capacidad de respuesta.
- Procesos pueden **transferir boletos** (cliente-servidor).
- Ideal para repartir CPU en proporciones exactas (ej.: video).

Planificación por partes equitativas

La planificación se hace **por usuario**, no por proceso.

- Cada usuario recibe una fracción de la CPU.
- Sus procesos se planifican dentro de ese límite.

Ejemplo:

- Usuario A tiene 4 procesos.
- Usuario B tiene 1 proceso.
- Ambos reciben 50% de la CPU.

Evita que un usuario "gane" CPU sólo por crear más procesos.

2.4.4 Planificación en sistemas de tiempo real

En los **sistemas de tiempo real**, **el tiempo es crítico**.

- Obtener la respuesta correcta **fuera de tiempo** puede ser tan grave como no obtenerla.
- Ejemplos: reproductores de audio, control industrial, monitoreo médico, robots.

Tipos de tiempo real

- **Tiempo real duro:** los tiempos límite **deben cumplirse siempre**.
- **Tiempo real suave:** fallar ocasionalmente es tolerable, pero degrada la calidad.

Eventos periódicos y aperiódicos

- **Periódicos**: ocurren a intervalos regulares.
- **Aperiódicos**: ocurren de forma impredecible.

Un sistema es **planificable** si la suma de las cargas cumple:

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Donde:

- Ci: tiempo de CPU requerido por evento
- Pi: período del evento

Si no se cumple, **no es posible garantizar los tiempos límite**.

Planificación estática y dinámica

- **Estática**:
    - decisiones tomadas antes de ejecutar el sistema,
    - requiere información perfecta previa.
- **Dinámica**:
    - decisiones tomadas en tiempo de ejecución,
    - más flexible, menos predecible.

2.5 Problemas clásicos de comunicación entre procesos (IPC)

Esta sección presenta **problemas modelo** usados para estudiar **sincronización y exclusión mutua**. No son ejercicios artificiales: representan situaciones reales de competencia por recursos y acceso a datos compartidos.

2.5.1 El problema de los filósofos comelones

Situación

- Cinco filósofos se sientan alrededor de una mesa.
- Cada uno alterna entre **pensar** y **comer**.
- Para comer necesita **dos tenedores**.
- Hay **un tenedor entre cada par de filósofos**.

Objetivo

Permitir que los filósofos coman y piensen:

- sin interbloqueo,
- sin inanición,
- permitiendo el mayor paralelismo posible.

Problemas de las soluciones simples

- Si todos toman un tenedor al mismo tiempo → **interbloqueo**.
- Si sueltan y reintentan todos juntos → **inanición**.

- Usar un mutex global evita errores, pero **sólo permite que coma uno a la vez**, desperdiciando recursos.

Idea de la solución correcta

- Cada filósofo tiene un **estado**: pensando, hambriento o comiendo.
- Un filósofo **sólo puede comer si sus dos vecinos no están comiendo**.
- Se usan:
    - un mutex para proteger el estado global,
    - un semáforo por filósofo para bloquearlo si no puede comer.

Resultado

- No hay interbloqueo.
- No hay inanición.
- Pueden comer varios filósofos al mismo tiempo si no son vecinos.

2.5.2 El problema de los lectores y escritores

Situación

Modela el acceso concurrente a una base de datos:

- **Lectores** sólo leen.
- **Escritores** modifican datos.

Reglas

- Varios lectores pueden leer al mismo tiempo.
- Si un escritor escribe, nadie más puede acceder.

Solución clásica

- Se permite que múltiples lectores entren juntos.
- El primer lector bloquea la base de datos.
- El último lector la libera.
- El escritor necesita acceso exclusivo.

Problema

- Si llegan lectores constantemente, un escritor puede quedar bloqueado indefinidamente → **inanición de escritores**.

Alternativa

- Dar prioridad a escritores.
- Evita inanición, pero reduce concurrencia y rendimiento.

UNIDAD 3 - ADMINISTRACIÓN DE MEMORIA

La **memoria principal (RAM)** es un recurso crítico que debe administrarse con cuidado. Aunque la memoria creció muchísimo con los años, **los programas crecen todavía más rápido**.
El sistema operativo crea **abstracciones de memoria** y las administra para:

- asignar memoria a los procesos,
- evitar interferencias,
- y usarla de forma eficiente.w

Las computadoras no tienen una sola memoria ideal, sino una **jerarquía de memoria**:

- caché: muy rápida, pequeña y volátil,
- memoria principal: más grande, velocidad media, volátil,
- disco: grande, lento y no volátil,
- Almacenamiento removible.

El **administrador de memoria** es la parte del sistema operativo que:

- lleva registro de qué memoria está en uso,
- asigna memoria a los procesos,
- la libera cuando ya no se necesita.

En este capítulo se estudia **la memoria principal**, desde esquemas simples hasta sofisticados.

3.1 Sin abstracción de memoria

Modelo

No existe ninguna abstracción: cada programa ve **directamente la memoria física**.

Una instrucción como:

```
MOV REGISTRO1, 1000
```

accede **a la dirección física 1000** de la RAM.

El programador ve la memoria como:

- direcciones desde 0 hasta un máximo,
- cada dirección corresponde a una celda de memoria.

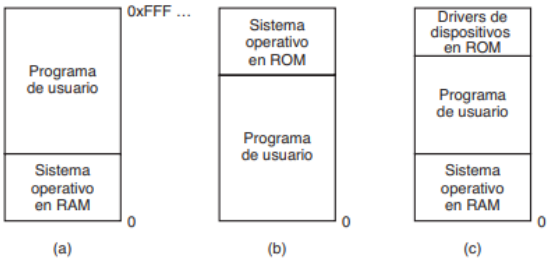


Figura 3-1. Tres formas simples de organizar la memoria con un sistema operativo y un proceso de usuario. También existen otras posibilidades.

Consecuencia principal

No pueden ejecutarse **dos programas al mismo tiempo** en memoria.

Si un programa escribe en una dirección:

- puede sobrescribir datos del otro,
- ambos programas fallan.

Ejecución típica

- El sistema carga **un solo programa** desde disco a memoria.
- Lo ejecuta.
- Cuando termina, carga otro programa **sobrescribiendo el anterior**.

¿Se puede tener paralelismo?

De forma muy limitada:

- usando **múltiples hilos dentro de un mismo proceso**, ya que comparten memoria.
- no sirve para ejecutar programas independientes.

Multiprogramación sin abstracción

Dos alternativas históricas:

Intercambio (swapping)

- Se guarda todo el contenido de memoria en disco.
- Se carga otro programa.
- Nunca hay más de un programa en memoria a la vez.

Protección por hardware (IBM 360)

- Memoria dividida en bloques con **llaves de protección**.
- Cada proceso sólo puede acceder a bloques con su llave.
- El sistema operativo controla las llaves.

Esto evita que los procesos se dañen entre sí, **pero no resuelve otro problema clave**.

Problema de reubicación

Los programas usan **direcciones absolutas**.

Si un programa fue escrito suponiendo que empieza en la dirección 0:

- al cargarse en otra dirección,
- los saltos (JMP) apuntan a lugares incorrectos,
- el programa falla.

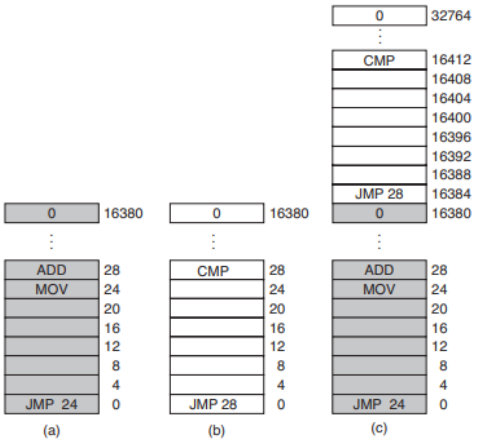


Figura 3-2. Ilustración del problema de reubicación. (a) Un programa de 16 KB. (b) Otro programa de 16 KB. (c) Los dos programas cargados consecutivamente en la memoria.

Solución parcial: reubicación estática

- Al cargar el programa, se suma una constante a todas las direcciones.
- Requiere:
  - saber qué palabras son direcciones y cuáles constantes,
  - modificar el programa al cargarse,
  - menor velocidad de carga.

No es una solución general ni elegante.

3.2 UNA ABSTRACCIÓN DE MEMORIA: ESPACIOS DE DIRECCIONES

Permitir que los procesos accedan directamente a la **memoria física** trae dos problemas graves.
Primero, un programa podría modificar la memoria del sistema operativo y hacer que el sistema falle.
Segundo, se vuelve muy difícil ejecutar **varios programas al mismo tiempo** sin que se interfieran.

Por eso, el sistema operativo introduce una **abstracción de memoria**, del mismo modo que el proceso es una abstracción de la CPU.

3.2.1 La noción de espacio de direcciones

Para que varios programas puedan coexistir en memoria sin interferirse, hay que resolver dos problemas: **protección** y **reubicación**.

La solución es el **espacio de direcciones**.
Un espacio de direcciones es el conjunto de direcciones que puede usar un proceso. Cada proceso ve **su propia memoria**, independiente de los demás. Así, la dirección 28 de un proceso no tiene relación con la dirección 28 de otro.

De esta manera:

- los procesos no pueden acceder a memoria ajena,
- cada programa se ejecuta como si fuera el único en la máquina.

Espacio de direcciones:

○ Crea un tipo de memoria abstracta para que los programas vivan ahí.

○ Un espacio de direcciones (address space) es el conjunto de direcciones que puede utilizar un proceso para direccionar la memoria.

○ Cada proceso tiene su propio espacio de direcciones, independiente de los que pertenecen a otros procesos

Registros base y límite

Una forma simple de implementar espacios de direcciones es usando **registros base y límite**.

Cuando un proceso se ejecuta:

- el registro **base** indica dónde empieza el proceso en memoria física,
- el registro **límite** indica su tamaño.

Cada vez que el proceso accede a memoria, la CPU suma automáticamente la dirección lógica del proceso con el valor del registro base. Además, verifica que la dirección no exceda el límite.
Si lo hace, se produce un error y el acceso se bloquea.

Así, el proceso siempre accede sólo a su región de memoria, aunque internamente se use memoria física real.

La desventaja es que cada acceso requiere una suma y una comparación, lo que introduce cierta sobrecarga.

3.2.2 Intercambio

En la práctica, la memoria RAM no alcanza para mantener todos los procesos al mismo tiempo.
La solución más simple es el **intercambio**.

El sistema operativo:

- carga un proceso completo en memoria,
- lo ejecuta un tiempo,
- lo guarda en disco,
- trae otro proceso.

Cuando un proceso vuelve a memoria, puede hacerlo en otra ubicación, por lo que necesita **reubicación**, que puede hacerse con registros base y límite.

Mover procesos genera huecos en la memoria. Unir esos huecos se llama **compactación**, pero casi no se usa porque consume mucho tiempo de CPU.

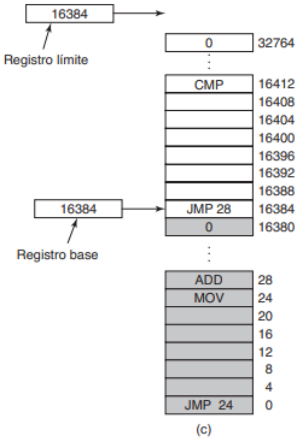


Figura 3-3. Se pueden utilizar registros base y límite para dar a cada proceso un espacio de direcciones separado.

La memoria física puede no ser lo suficientemente grande para almacenar todos los procesos:

○ Intercambio:

■ Copia programas desde la RAM al disco y trae programas del disco a la RAM.

■ Cuando el proceso finaliza su tiempo de ejecución, éste puede volver al disco o mantenerse en la RAM.

○ Memoria virtual: permite que los programas se ejecuten incluso cuando solo se encuentran en forma parcial en la memoria.

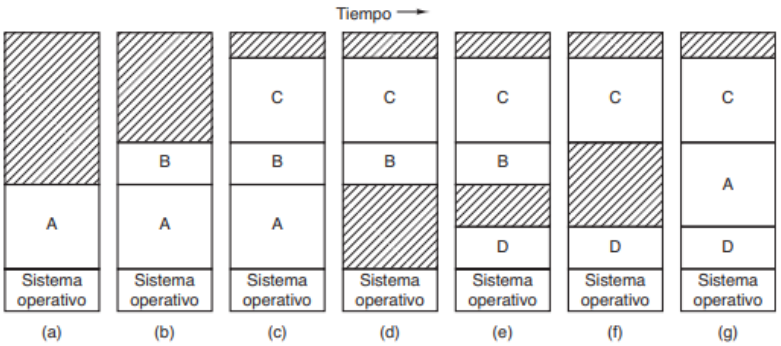


Figura 3-4. La asignación de la memoria cambia a medida que llegan procesos a la memoria y salen de ésta. Las regiones sombreadas son la memoria sin usar.

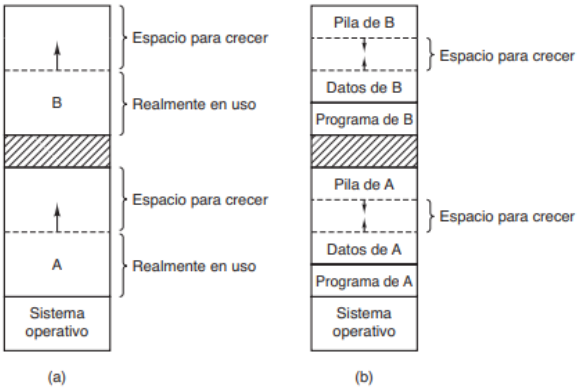


Figura 3-5. (a) Asignación de espacio para un segmento de datos en crecimiento. (b) Asignación de espacio para una pila en crecimiento y un segmento de datos en crecimiento.

Crecimiento de los procesos

Los procesos pueden crecer mientras se ejecutan, por ejemplo:

- el **heap** crece hacia arriba,
- la **pila** crece hacia abajo.

Si no hay espacio suficiente:

- el proceso debe moverse,
- intercambiarse,
- o suspenderse.

3.2.3 Administración de memoria libre

El sistema operativo debe llevar registro de qué partes de la memoria están libres.

Una opción es el **mapa de bits**, donde cada unidad de memoria tiene un bit que indica si está libre u ocupada. Es simple, pero buscar espacios grandes es lento.

Otra opción es usar **listas ligadas** de procesos y huecos. Esto facilita fusionar huecos cuando un proceso termina y permite aplicar distintos algoritmos de asignación.

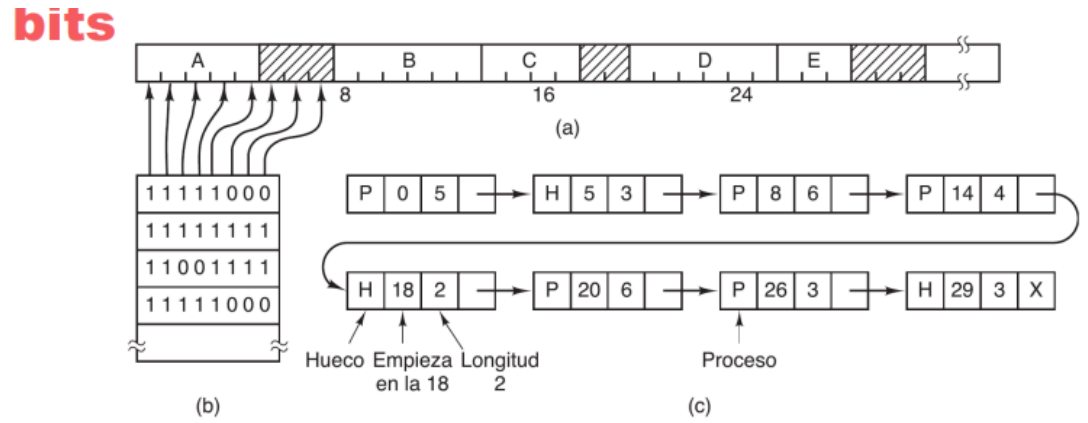


Figura 3-6. (a) Una parte de la memoria con cinco procesos y tres huecos. Las marcas de graduación muestran las unidades de asignación de memoria. Las regiones sombreadas (0 en el mapa de bits) están libres. **(b)** El mapa de bits correspondiente. **(c)** La misma información en forma de lista.

Algoritmos de asignación (idea general)

- **Primer ajuste**: usa el primer hueco suficientemente grande. Es rápido.
- **Mejor ajuste**: busca el hueco más pequeño posible. Produce más fragmentación.
- **Peor ajuste**: usa el hueco más grande. No funciona bien en la práctica.
- **Ajuste rápido**: mantiene listas por tamaños frecuentes. Es muy veloz, pero fragmenta.

3.3 MEMORIA VIRTUAL

El crecimiento del software hace imposible que todos los programas quepan completos en memoria.
El **intercambio** tampoco es viable, porque mover procesos enteros entre memoria y disco es demasiado lento.

La solución es la **memoria virtual**: permitir que los programas se ejecuten aunque **sólo una parte de ellos esté en memoria**.

Esta memoria virtual nace por la necesidad de que los programas sean más grandes que la memoria, por lo que, se dividen los programas en pequeñas partes (Sobrepuestos/ Overlays). De esta forma ahora cuando se arranca un programa, se carga del Administrador de Sobrepuestos para ejecutar el bloque 0 del programa, luego el 1, y así. La Memoria Virtual conforma una parte del Disco, haciendo los intercambios desde ahí, guardando los sobrepuestos.

Cada proceso tiene su **propio espacio de direcciones**, dividido en partes pequeñas. Sólo las partes necesarias están en RAM; el resto permanece en disco.

Idea central de la memoria virtual

Un programa cree que toda su memoria está disponible, pero en realidad:

- algunas partes están en memoria física,
- otras están en disco,
- El sistema operativo mueve partes automáticamente según se necesiten.

Esto elimina la necesidad de que el programador gestione la memoria manualmente.

3.3.1 Paginación

La técnica más usada para implementar memoria virtual es la **paginación**.

El espacio de direcciones virtuales se divide en bloques de tamaño fijo llamados **páginas**.
La memoria física se divide en bloques del mismo tamaño llamados **marcos de página**.

Las páginas pueden estar:

- asociadas a un marco en memoria,
- o no estar en memoria (en disco).

Direcciones virtuales y MMU

Los programas generan **direcciones virtuales**.
Estas direcciones no van directamente a la memoria.

Primero pasan por la **MMU (Unidad de Administración de Memoria)**, que:

- traduce la dirección virtual a una dirección física,
- o detecta que la página no está en memoria.

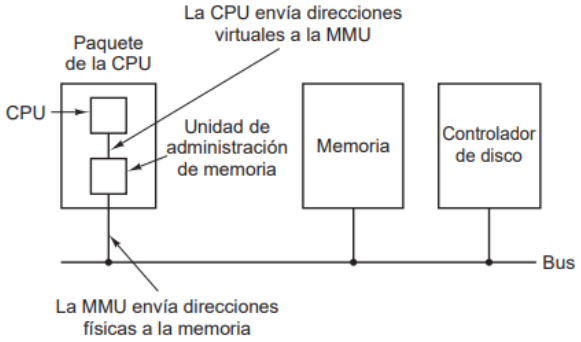


Figura 3-8. La posición y función de la MMU. Aquí la MMU se muestra como parte del chip de CPU, debido a que es común esta configuración en la actualidad. Sin embargo, lógicamente podría ser un chip separado y lo era hace años.

Si la página no está en memoria, ocurre un **fallo de página**.

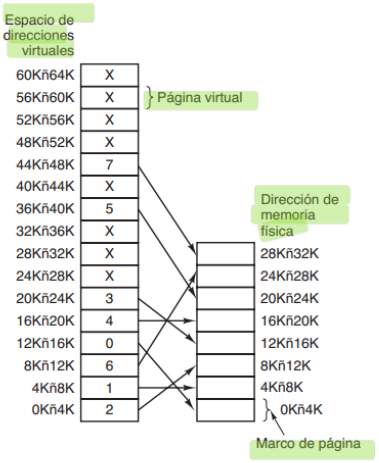


Figura 3-9. La relación entre las direcciones virtuales y las direcciones de memoria física está dada por la tabla de páginas. Cada página empieza en un múltiplo de 4096 y termina 4095 direcciones más arriba, por lo que de 4 K a 8 K en realidad significa de 4096 a 8191 y de 8 K a 12 K significa de 8192 a 12287.

Fallo de página

Cuando una página no está en memoria:

1. la CPU genera un trap al sistema operativo,
2. el sistema operativo elige un marco para liberar,
3. guarda la página vieja en disco si fue modificada,
4. trae la página necesaria desde disco,
5. reintenta la instrucción.

Este proceso es transparente para el programa.

División de la dirección virtual

Una dirección virtual se divide en dos partes:

- **número de página**, que identifica la página virtual,
- **desplazamiento**, que indica la posición dentro de la página.

El número de página se usa como índice en la **tabla de páginas**.

La asociación de direcciones virtuales a direcciones físicas se puede resumir de la siguiente manera:

○ La dirección virtual se divide en un número de página virtual (bits de mayororden) y en un desplazamiento (bits de menor orden).

○ Por ejemplo, con una dirección de 16 bits y un tamaño de página de 4 KB, los 4 bits superiores podrían especificar una de las 16 páginas virtuales y los 12 bits inferiores podrían entonces especificar el desplazamiento de bytes (0 a 4095)dentro de la página seleccionada.

○ Sin embargo, también es posible una división con 3, 5 u otro número de bits para la página.

○ Las distintas divisiones implican diferentes tamaños de página.

3.3.2 Tablas de páginas

La **tabla de páginas** asocia cada página virtual con:

- un marco de página físico,
- o indica que la página no está presente.

Su función es traducir direcciones virtuales en direcciones físicas.

El número de página virtual se utiliza como índice en la tabla de páginas para buscar la entrada para esa página virtual.
En la entrada en la tabla de páginas, se encuentra el número de marco de página (si lo hay). El número del marco de página se adjunta al extremo de mayor orden del desplazamiento, reemplazando el número de página virtual, para formar una dirección física que se pueda enviar a la memoria.

El propósito de la tabla de páginas es asociar páginas virtuales a los marcos de página.

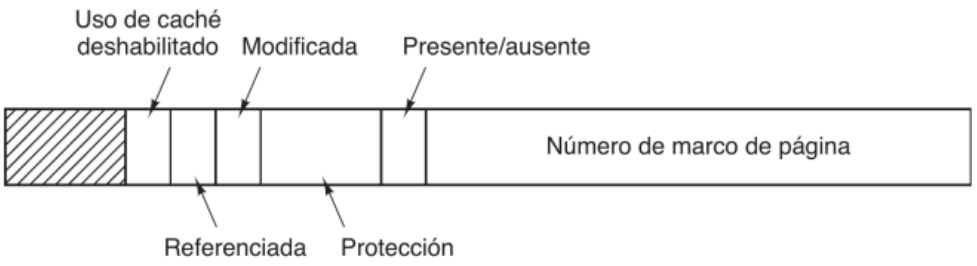
Entrada de la tabla de páginas

Cada entrada contiene información clave:

- número de marco de página,
- bit de presente/ausente,
- bits de protección (lectura, escritura, ejecución),
- bit de modificada,
- bit de referenciada,
- control de uso de caché.

Estos bits permiten al sistema operativo:

- proteger memoria,
- decidir qué página reemplazar,
- evitar escrituras innecesarias al disco.



3.3.3 Aceleración de la paginación

En cualquier sistema con **memoria virtual paginada** aparecen dos problemas fundamentales:

1. **La traducción de direcciones virtuales a físicas debe ser muy rápida**, porque ocurre en **cada referencia a memoria**.
2. **Las tablas de páginas pueden ser enormes**, porque los espacios de direcciones virtuales modernos son muy grandes.

Si la tabla de páginas estuviera sólo en memoria principal, cada acceso a memoria requeriría primero acceder a la tabla de páginas y luego a la memoria real. Esto **duplica las referencias a memoria** y reduce drásticamente el rendimiento. Por eso, este esquema puro no es aceptable en la práctica.

Búfer de traducción adelantada (TLB)

Para resolver el problema de la velocidad, se introduce el **TLB (Translation Lookaside Buffer)**.
El TLB es una **memoria pequeña y muy rápida**, ubicada dentro de la **MMU**, que guarda **un subconjunto de las entradas de la tabla de páginas** que se usan con mayor frecuencia.

Cada entrada del TLB contiene:

- número de página virtual
- número de marco de página
- bits de protección
- bit de modificada
- bit de válida

Válida	Página virtual	Modificada	Protección	Marco de página
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Figura 3-12. Un TLB para acelerar la paginación.

El funcionamiento es el siguiente:

- Cuando la CPU genera una dirección virtual, la MMU **primero consulta el TLB**.
- Si la página virtual está en el TLB y los permisos son correctos, se obtiene directamente la dirección física **sin acceder a la tabla de páginas**.
- Si la página no está en el TLB, ocurre un **fallo de TLB**:
    - Se consulta la tabla de páginas en memoria.

    - Se reemplaza una entrada del TLB por la nueva.
    - Se continúa la ejecución.

Gracias al principio de **localidad**, la mayoría de las referencias a memoria se resuelven en el TLB, evitando accesos costosos a la tabla de páginas.

Fallos suaves y fallos duros

Es fundamental distinguir dos tipos de fallos:

- **Fallo suave**: la página no está en el TLB pero sí está en memoria.
  Sólo se actualiza el TLB. Es rápido (nanosegundos).
- **Fallo duro**: la página no está en memoria.
  El sistema operativo debe traerla desde disco. Es extremadamente lento (milisegundos).

Un fallo duro es **varios órdenes de magnitud más lento** que un fallo suave, por lo que el objetivo principal del sistema es minimizar los fallos duros.

Administración del TLB mediante hardware y software

En algunas arquitecturas, el **hardware** maneja automáticamente el TLB.
En muchas arquitecturas RISC modernas, el **sistema operativo** administra el TLB mediante software.

En este último caso:

- Un fallo de TLB genera un trap al sistema operativo.
- El sistema busca la página, actualiza el TLB y reinicia la instrucción.
- Este mecanismo es viable porque los fallos de TLB son frecuentes pero baratos, y el TLB suele ser suficientemente grande.

3.3.4 Tablas de páginas para memorias extensas

Aunque el TLB acelera la traducción, **no resuelve el tamaño enorme** de las tablas de páginas.

Tablas de páginas multinivel

La paginación multinivel es un método típico en arquitecturas de 32 bits que busca no mantener todas las tablas de páginas en memoria, sino solo las necesarias.

Para esto, divide la tabla de páginas en varios niveles.
La dirección virtual se divide en índices de cada nivel y un desplazamiento.

Cada nivel apunta al siguiente solo si es necesario, por lo que solo se crean y cargan en memoria las tablas que se utilizan.

Si una tabla no está presente, se produce un fallo de página y el sistema operativo debe intervenir.

Consiste en evitar mantener todas las tablas de páginas todo el tiempo, y en especial las que no se necesitan. La entrada que se coloca en la tabla de primer nivel, lleva a la de segundo nivel. Aunque el espacio de direcciones contiene muchas palabras, solo pocas tablas son requeridas. Los bits de Presente/Ausente en las páginas de nivel superior obligan a que se produzca un fallo de página. En este caso el SO tiene que actuar. El problema es que es propensa a quedarse sin memoria gracias a la arquitectura de 64 bits

La idea es **no mantener toda la tabla en memoria**.

La dirección virtual se divide en:

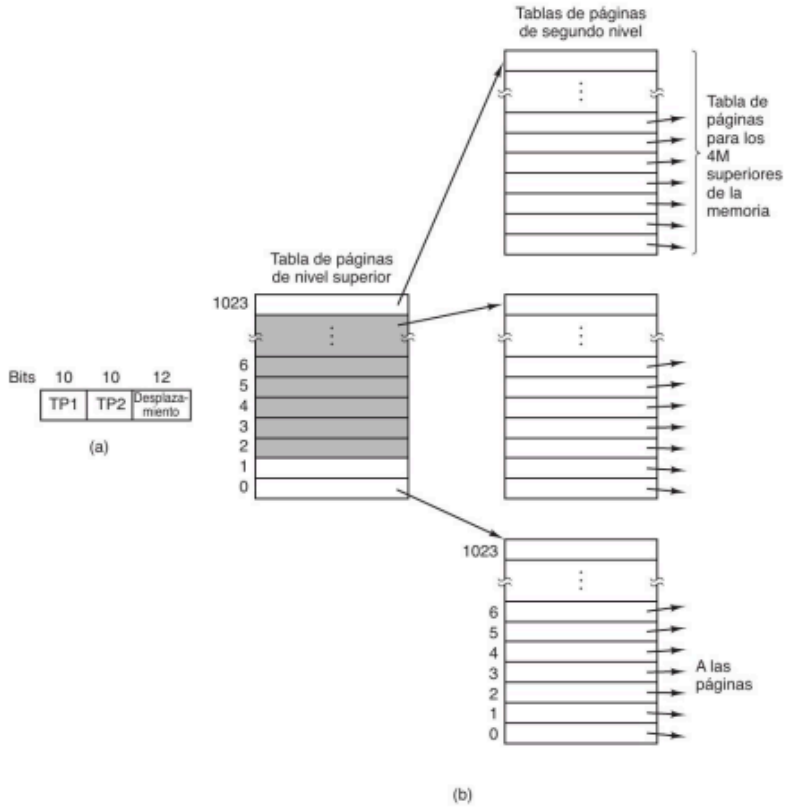
- varios índices (TP1, TP2, …),
- un desplazamiento.

Cada nivel apunta al siguiente **sólo si se usa**.

Ventaja:

- sólo existen en memoria las tablas necesarias,
- Las grandes regiones no usadas no consumen memoria.

Es el método típico en sistemas de **32 bits**.



Tablas de páginas invertidas

Las tablas de páginas invertidas mantienen una entrada por marco físico en lugar de por página virtual, reduciendo el tamaño de la tabla, pero haciendo más compleja la traducción de direcciones, por lo que se utilizan TLB y tablas hash para acelerar las búsquedas.

Hay una entrada por cada marco de página en la memoria real. Ahorran grandes cantidades de espacio, aunque tienen una seria desventaja: la traducción. La solución es usar el TLB. Si el TLB contiene las páginas de uso frecuente, la traducción puede ser igual de rápido que con las tablas regulares. Sin embargo, ya sabemos que pasa si ocurre un fallo en el TLB. Por lo que se incorpora una tabla Hash. Con todas las páginas virtuales con el mismo hash encadenadas en conjunto. Si la tabla tiene tantas ranuras como páginas físicas, la cadena promedio tendrá solo una entrada.

En sistemas de **64 bits**, incluso las tablas multinivel se vuelven gigantes.

La solución:

- una entrada **por marco físico**, no por página virtual.

Cada entrada dice:

- qué proceso,
- qué página virtual ocupa ese marco.

Ventaja:

- la tabla es pequeña.

Desventaja:

- traducir direcciones es más complejo.

Por eso:

- se usa **TLB** para los accesos frecuentes,
- En fallos de TLB se busca en la tabla invertida usando **hash**.

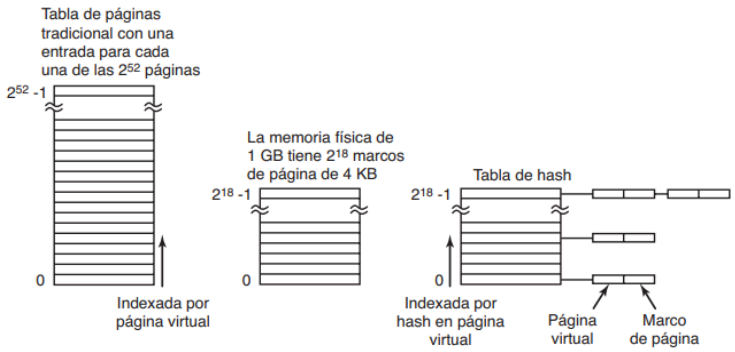


Figura 3-14. Comparación de una tabla de páginas tradicional con una tabla de páginas invertida.

Idea final para fijar

- La paginación **necesita aceleración** para ser usable.
- El **TLB** es clave para el rendimiento.
- Para memorias grandes:
    - 32 bits → tablas multinivel
    - 64 bits → tablas invertidas + TLB

3.4 Algoritmos de reemplazo de páginas

Cuando ocurre un **fallo de página**, el sistema operativo debe **desalojar una página de la memoria** para hacer espacio a la nueva.
Si la página fue **modificada**, debe escribirse de vuelta al disco; si no, simplemente se descarta.

Elegir cualquier página al azar funciona, pero **el rendimiento mejora mucho** si se elimina una página **que no se usa con frecuencia**, ya que eliminar una página muy usada provoca que tenga que traerse de nuevo rápidamente.

Este problema aparece también en **memorias caché** y en **cachés de servidores web**: cuando el espacio es limitado, hay que decidir **qué descartar**.

Una decisión clave es si la página a desalojar debe pertenecer **al mismo proceso** o puede pertenecer **a otro proceso**. Ambos enfoques existen.

3.4.1 Algoritmo óptimo

El **algoritmo óptimo** elimina la página **que se va a usar más lejos en el futuro**.
De este modo, el siguiente fallo de página se retrasa lo máximo posible.

El problema es que **no se puede implementar**, porque el sistema operativo **no conoce el futuro**.
Sólo se usa como **referencia teórica** para comparar otros algoritmos.

3.4.2 NRU (No usadas recientemente)

Este algoritmo usa dos bits por página:

- **R (referenciada)**
- **M (modificada)**

Las páginas se clasifican en cuatro clases:

- Clase 0: no referenciada, no modificada
- Clase 1: no referenciada, modificada
- Clase 2: referenciada, no modificada

- Clase 3: referenciada, modificada

El algoritmo **elige al azar una página de la clase con menor número disponible**.

La idea es simple:
Es mejor eliminar una página **no usada recientemente**, y mejor aún si **no está modificada**.

Es **simple**, **barato**, pero **bastante burdo**.

3.4.3 FIFO (Primera en entrar, primera en salir)

FIFO elimina la página **que llegó primero a memoria**.

Es fácil de implementar, pero **muy malo**:
una página puede haber llegado primero y **seguir siendo muy usada**.

Por eso, FIFO casi no se usa en sistemas reales.

3.4.4 Segunda oportunidad

Es una mejora de FIFO.

Antes de eliminar la página más antigua, se revisa su bit **R**:

- Si **R = 0**, se elimina.
- Si **R = 1**, se borra R y la página se mueve al final de la lista.

Así, una página muy usada no se elimina de inmediato.
Si todas las páginas tienen R = 1, el algoritmo **degenera en FIFO**, pero **siempre termina**.

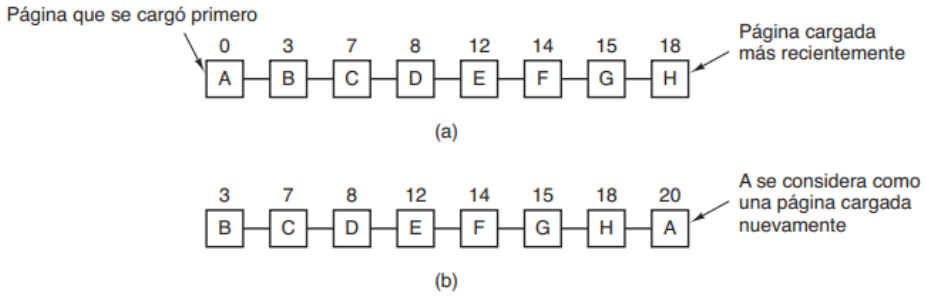


Figura 3-15. Operación del algoritmo de la segunda oportunidad. (a) Páginas ordenadas con base en FIFO. (b) Lista de las páginas si ocurre un fallo de página en el tiempo 20 y *A* tiene su bit *R* activado. Los números encima de las páginas son sus tiempos de carga.

3.4.5 Algoritmo del reloj

Es una implementación más eficiente de segunda oportunidad.

Las páginas se organizan en una **lista circular**, y una **manecilla** apunta a la candidata a eliminar.

- Si **R = 0**, se elimina.
- Si **R = 1**, se borra R y se avanza la manecilla.

Hace lo mismo que en la segunda oportunidad, pero **sin mover páginas en memoria**, por eso es más eficiente.

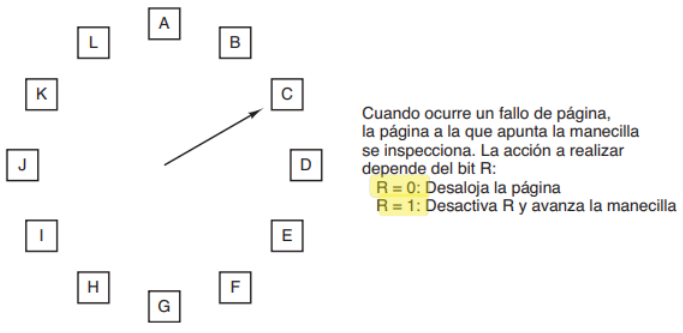


Figura 3-16. El algoritmo de reemplazo de páginas en reloj.

3.4.6 LRU (Menos usadas recientemente)

LRU elimina la página **que no se ha usado durante más tiempo**.

Se basa en la idea de **localidad**:
las páginas usadas recientemente probablemente se vuelvan a usar.

Es una **excelente aproximación al algoritmo óptimo**, pero **difícil de implementar exactamente**, porque requeriría actualizar información **en cada referencia a memoria**.

3.4.7 Simulación de LRU en software

Como LRU exacto es caro, se usan aproximaciones:

NFU (No utilizadas frecuentemente)

Cada página tiene un contador que se incrementa según el bit R en cada pulso de reloj.

Problema: **no olvida el pasado**, páginas usadas hace mucho pueden quedar protegidas para siempre.

Envejecimiento (aging)

Es una mejora de NFU:

- Los contadores se desplazan a la derecha.
- El bit R se inserta en el bit más significativo.

Así, **las referencias antiguas pierden peso** con el tiempo.

Es una **muy buena aproximación a LRU**, eficiente y realista.

3.4.8 Conjunto de trabajo

Los procesos no usan todas sus páginas al mismo tiempo, sino un **conjunto pequeño** durante cada fase de ejecución.
A este conjunto se lo llama **conjunto de trabajo**.

Si el conjunto de trabajo está en memoria, el proceso corre bien.
Si no entra en memoria, aparece el **thrashing**: muchos fallos de página y ejecución lentísima.

El **modelo del conjunto de trabajo** intenta mantener en memoria **las páginas usadas en los últimos τ segundos de tiempo virtual**.

El algoritmo elimina páginas **que no están en el conjunto de trabajo**.

Es conceptualmente muy bueno, pero **costoso de implementar**.

3.4.9 WSClock

WSClock combina:

- algoritmo del **reloj**
- información del **conjunto de trabajo**
- bits **R y M**

Funciona así, en esencia:

- Si R = 1, se borra y se avanza.
- Si R = 0 y la página es vieja y limpia, se elimina.
- Si es vieja pero sucia, se programa la escritura y se sigue buscando.

Es **eficiente**, **realista** y **muy usado en la práctica**.

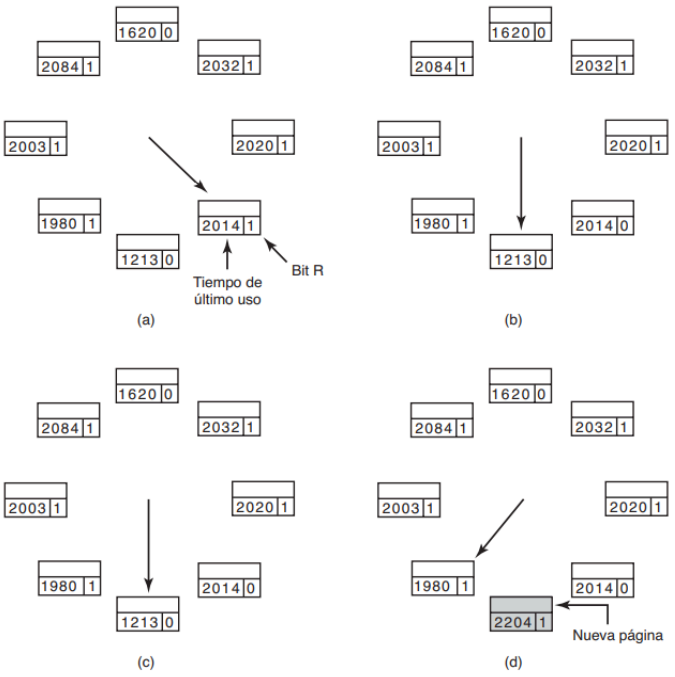


Figura 3-21. Operación del algoritmo WSClock. (a) y (b) dan un ejemplo de lo que ocurre cuando $R = 1$. (c) y (d) dan un ejemplo de cuando $R = 0$.

Algoritmo	Comentario
Óptimo	No se puede implementar, pero es útil como punto de comparación
NRU (No usadas recientemente)	Una aproximación muy burda del LRU
FIFO (primera en entrar, primera en salir)	Podría descartar páginas importantes
Segunda oportunidad	Gran mejora sobre FIFO
Reloj	Realista
LRU (menos usadas recientemente)	Excelente, pero difícil de implementar con exactitud
NFU (no utilizadas frecuentemente)	Aproximación a LRU bastante burda
Envejecimiento	Algoritmo eficiente que se aproxima bien a LRU
Conjunto de trabajo	Muy costoso de implementar
WSClock	Algoritmo eficientemente bueno

3.6.2 Fallo de página

Secuencia esencial:

1. El hardware hace un trap al kernel, guardando el contador del programa en la pila
2. Se inicia una rutina en código ensamblador para guardar los registros generales e información volátil
3. El sistema operativo descubre que ha ocurrido un fallo de página y trata de descubrir cuál página virtual se necesita.
4. Una vez que se conoce la dirección virtual que produjo el fallo, el sistema comprueba si esta dirección es válida y si la protección es consistente con el acceso.
5. Si el marco de página seleccionado está sucio, la página se planifica para transferirla al disco y se realiza una conmutación de contexto
6. Tan pronto como el marco de página esté limpio, el sistema operativo busca la dirección de disco en donde se encuentra la página necesaria, y planifica una operación de disco para llevarla a memoria.
7. Cuando la interrupción de disco indica que la página ha llegado, las tablas de páginas se actualizan para reflejar su posición y el marco se marca como en estado normal.
8. La instrucción fallida se respalda al estado en que tenía cuando empezó, y el contador de programa se restablece para apuntar a esa instrucción.
9. El proceso fallido se planifica y el sistema operativo regresa a la rutina (en lenguaje ensamblador) que lo llamó.
10. Esta rutina recarga los registros y demás información de estado, regresando al espacio de usuario para continuar la ejecución, como si no hubiera ocurrido el fallo.

trap → identificar página → validar → elegir marco → escribir si está sucia → leer página → actualizar tablas → **reintentar instrucción**.

3.7 Segmentación

Hasta ahora, la memoria virtual era **unidimensional**: un **solo espacio de direcciones** lineal de 0 a un máximo.

El problema es que muchos programas trabajan con **estructuras lógicas distintas** (tablas, pilas, código) que **crecen y se reducen de forma independiente**.

Ejemplo clásico: un **compilador**, que maneja texto fuente, tabla de símbolos, constantes, árbol sintáctico y pila.

En una memoria unidimensional, estas estructuras compiten por un único espacio y **pueden chocar entre sí**, desperdiciando memoria o complicando la gestión.

La **segmentación** resuelve esto dividiendo la memoria en **múltiples espacios de direcciones independientes**, llamados **segmentos**.

Estos segmentos, a contrario de las páginas, varían en longitud a su gusto. Estas longitudes pueden cambiar durante la ejecución de un proceso.

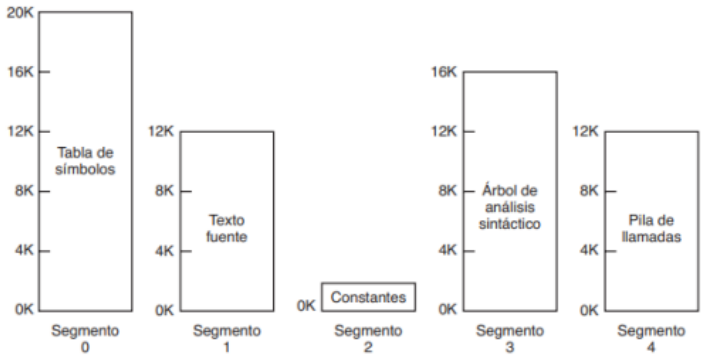


Figura 3-32. Una memoria segmentada permite que cada tabla crezca o se reduzca de manera independiente a las otras tablas.

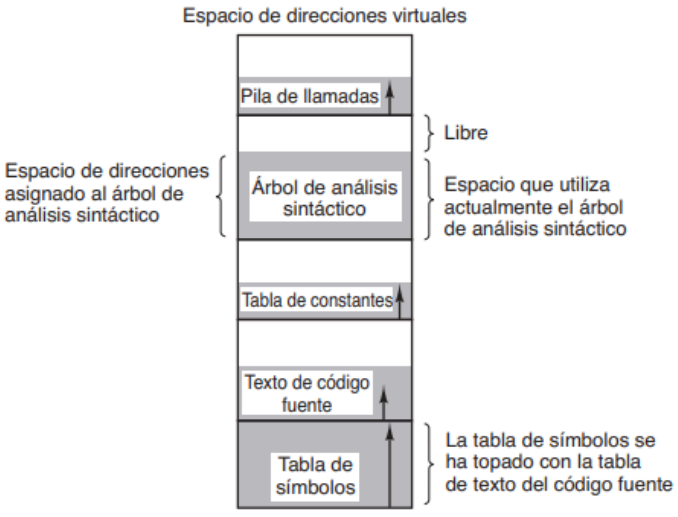


Figura 3-31. En un espacio de direcciones unidimensional con tablas que aumentan de tamaño, una tabla puede toparse con otra.

Idea central de la segmentación

Un **segmento** es una entidad **lógica**, conocida por el programador (procedimiento, arreglo, pila, datos).

Cada segmento:

- empieza en 0,
- tiene **longitud variable**,
- puede **crecer o reducirse independientemente**,
- tiene **protección propia**.

Una dirección ya no es un solo número, sino un par:
(número de segmento, dirección dentro del segmento).

Esto elimina la necesidad de que el programador administre manualmente estructuras que cambian de tamaño.

Ventajas clave de la segmentación

1. **Crecimiento independiente**
   Un segmento puede crecer sin afectar a los demás. No hay choques entre tablas o pilas.
2. **Modularidad y vinculación**
   Si cada procedimiento está en su propio segmento, recompilar uno **no cambia las direcciones de los demás**.
   Esto simplifica enormemente la vinculación de programas grandes.
3. **Compartición natural**
   Segmentos completos (por ejemplo, bibliotecas) pueden **compartirse entre procesos**.
4. **Protección semántica**
   Cada segmento puede tener protección acorde a su contenido:
   - código: sólo ejecución,
   - datos: lectura/escritura,
   - pila: no ejecutable.

5. Esto **detecta errores de programación** de forma natural.

La protección es **natural en segmentación** porque el contenido del segmento es lógico y conocido.
En paginación, el contenido de una página es accidental.

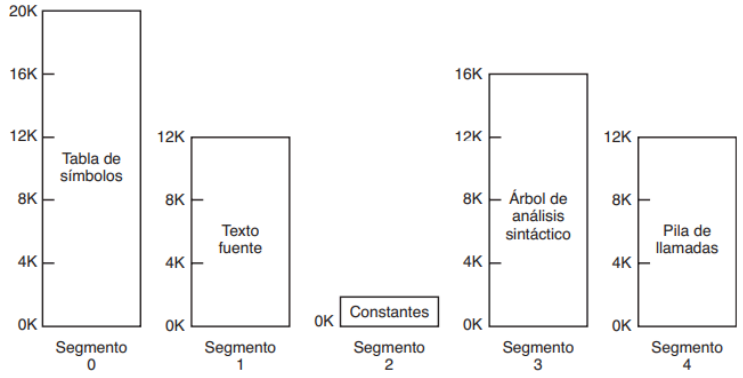


Figura 3-32. Una memoria segmentada permite que cada tabla crezca o se reduzca de manera independiente a las otras tablas.

Consideración	Paginación	Segmentación
¿Necesita el programador estar consciente de que se está utilizando esta técnica?	No	Sí
¿Cuántos espacios de direcciones lineales hay?	1	Muchos
¿Puede el espacio de direcciones total exceder al tamaño de la memoria física?	Sí	Sí
¿Pueden los procedimientos y los datos diferenciarse y protegerse por separado?	No	Sí
¿Pueden las tablas cuyo tamaño fluctúa acomodarse con facilidad?	No	Sí
¿Se facilita la compartición de procedimientos entre usuarios?	No	Sí
¿Por qué se inventó esta técnica?	Para obtener un gran espacio de direcciones lineal sin tener que comprar más memoria física	Para permitir a los programas y datos dividirse en espacios de direcciones lógicamente independientes, ayudando a la compartición y la protección

Figura 3-33. Comparación de la paginación y la segmentación.

Segmentación pura: problema

Los segmentos **no tienen tamaño fijo**.
Al cargarlos y desalojarlos se generan **huecos en la memoria**.

Esto produce **fragmentación externa** (efecto de tablero de ajedrez).

Solución posible: **compactación**, moviendo segmentos para unir huecos, pero es costosa.

3.7.2 Segmentación con paginación: MULTICS

Cada **programa posee una tabla de segmentos**, con un descriptor. Esta tabla, por el tamaño, se crea también como su propio segmento y se página. Contiene la información si está en Memoria. El descriptor posee el tamaño del segmento. La dirección del segmento en el disco no está en el descriptor, sino que en otra tabla utilizada por el manejador de fallos de segmento.

MULTICS combina **segmentación + paginación**.

Cada segmento:

- es un espacio de direcciones lógico,
- pero internamente está **paginado**.

Esto permite:

- no tener el segmento completo en memoria,
- mantener las ventajas lógicas de la segmentación.

Cada proceso tiene una **tabla de segmentos**.
Cada descriptor de segmento indica:

- tamaño,
- protección,
- si está en memoria,
- apuntador a su **tabla de páginas**.

Una dirección MULTICS tiene tres partes:
segmento → página → desplazamiento.

El hardware usa un **TLB** para acelerar el acceso y evitar recorrer tablas en cada instrucción.

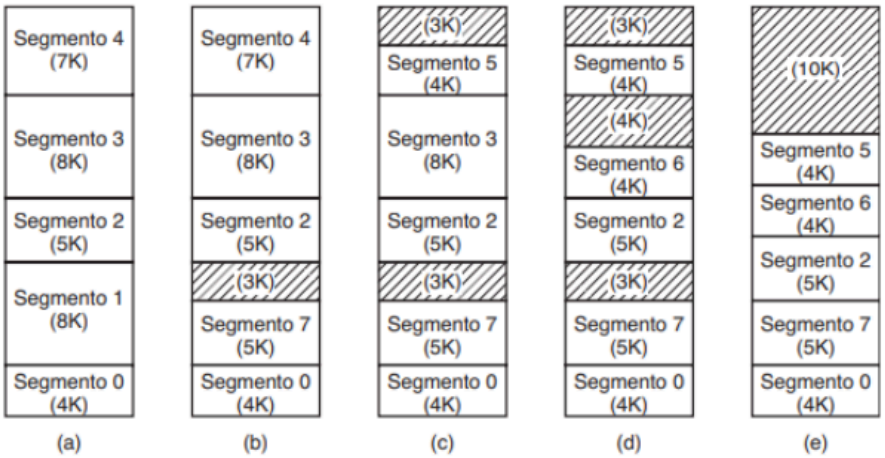


Figura 3-34. (a)-(d) Desarrollo del efecto de tablero de ajedrez. (e) Eliminación del efecto de tablero de ajedrez mediante la compactación.

Cada segmento es un espacio de direcciones virtuales ordinario y se pagina.

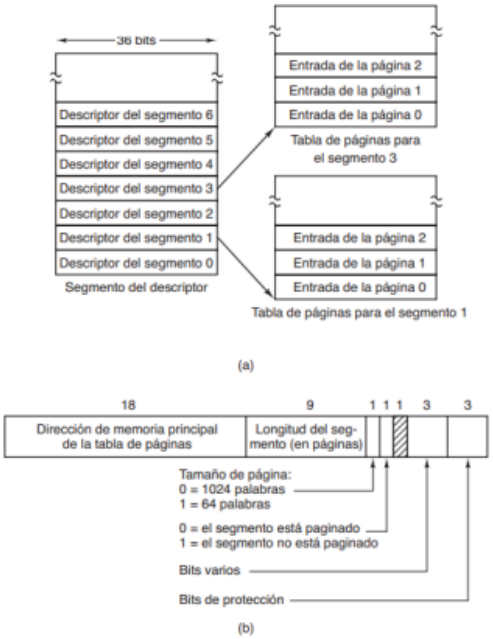


Figura 3-35. La memoria virtual de MULTICS. (a) El segmento del descriptor apunta a las tablas de páginas. (b) Un descriptor de segmento. Los números son las longitudes de los campos.

3.7.3 Segmentación con paginación: Intel Pentium

El Pentium adopta una idea similar a MULTICS.

LDT (Tabla de Descriptores Locales) y GDT (Tabla de descriptores globales). Cada programa tiene su propia LDT pero una única GDT compartida por todos los programas en la computadora. Cada selector es un número de 16 bits.

Al momento que se carga, el descriptor obtiene la GDT o LDT y se almacena en registros. Si el segmento no existe, o se paginó para sacarlo, se produce un Trap. Después el hardware utiliza el límite para ver que no se salga del segmento, en caso contrario, Trap. Si pasa, suma el campo Base.

Si se desactiva la paginación, se utiliza aun sistema de segmentación pura. En caso contrario, la dirección lineal se interpreta como una virtual y se asigna usando tablas de página. Cada programa tiene un directorio de páginas de 1024 entradas de 32 bits.

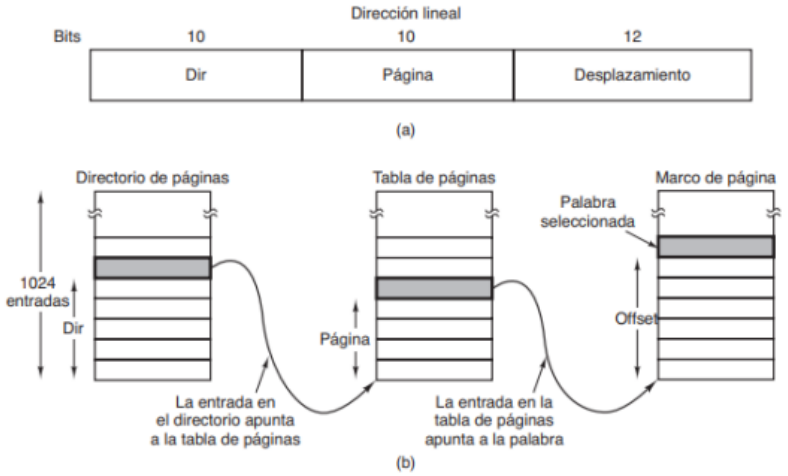


Figura 3-42. Asociación de una dirección lineal a una dirección física.

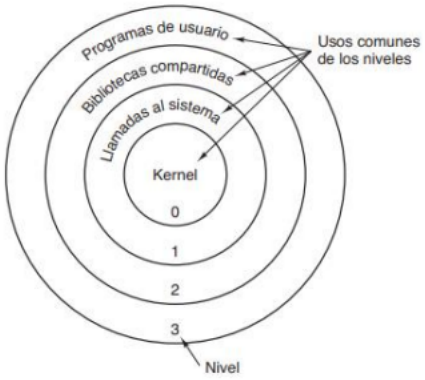


Figura 3-43. La protección en el Pentium.

Tiene:

- **segmentación** (LDT y GDT),
- **paginación** (dos niveles),
- **TLB** para acelerar traducciones.

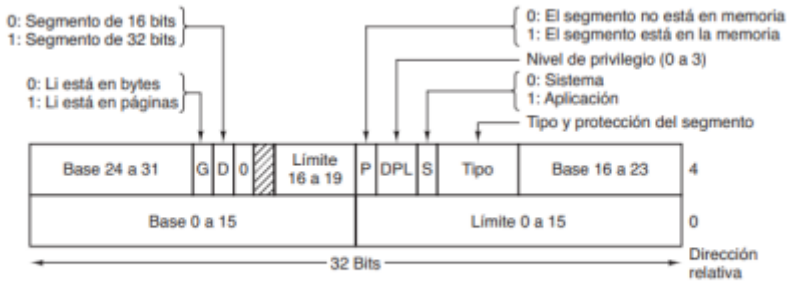


Figura 3-40. Descriptor del segmento de código del Pentium. Los segmentos de datos difieren un poco.

Primero:

1. El selector identifica el segmento.

2. El descriptor aporta **base + límite** → dirección lineal.

Luego:
3. La dirección lineal se traduce mediante **paginación** a dirección física.

Si la paginación está deshabilitada → **segmentación pura**.
Si todos los segmentos tienen base 0 y límite máximo → **paginación pura**.

Por eso:

> todos los sistemas operativos modernos en Pentium usan paginación y **simulan** segmentación mínima.

Protección en Pentium

Hay **cuatro niveles de protección (0 a 3)**:

- 0: kernel,
- 1: llamadas al sistema,
- 2: bibliotecas,
- 3: usuario.

Los saltos entre niveles se hacen mediante **compuertas de llamada**, no direcciones directas.
Este modelo proviene directamente de MULTICS.

Unidad 4 SISTEMAS DE ARCHIVOS

Los procesos necesitan **almacenar y recuperar información**.
Guardar datos sólo en el espacio de direcciones de un proceso **no alcanza**, por tres razones fundamentales:

1. **El espacio de direcciones es limitado**: muchas aplicaciones requieren almacenar cantidades muy grandes de información.
2. **La información se pierde al terminar el proceso**: para aplicaciones como bases de datos, bancos o contabilidad, esto es inaceptable.
3. **Varios procesos deben acceder a la información al mismo tiempo**: la información no puede pertenecer a un solo proceso.

De estos problemas surgen **tres requerimientos esenciales del almacenamiento a largo plazo**:

- almacenar grandes cantidades de información,
- que la información sea **persistente** (sobrevive a la terminación del proceso),
- permitir **acceso concurrente** de múltiples procesos.

El disco y el problema real

A nivel físico, un disco puede verse como una **secuencia lineal de bloques de tamaño fijo**, con dos operaciones básicas:

- leer el bloque *k*,
- escribir el bloque *k*.

Aunque con esto sería posible almacenar información, **no es práctico** para sistemas reales. Surgen preguntas clave:

- ¿cómo se encuentra la información?,
- ¿cómo se protege de otros usuarios?,
- ¿cómo se sabe qué bloques están libres?

La abstracción fundamental: el archivo

Así como el sistema operativo creó:

- la abstracción de **proceso** a partir del procesador,
- la abstracción de **espacio de direcciones** a partir de la memoria física,

crea ahora la abstracción de **archivo** para resolver el almacenamiento a largo plazo.

Un **archivo** es una **unidad lógica de información**, independiente de cualquier proceso. La información en los archivos es **persistente**: sólo desaparece si su propietario la elimina explícitamente.

El conjunto de mecanismos que crean, nombran, protegen y administran archivos se denomina **sistema de archivos**, y es uno de los **conceptos centrales de los sistemas operativos**, junto con procesos y memoria virtual.

4.1 Archivos

Aquí el enfoque es **desde el punto de vista del usuario**: cómo se ven y cómo se usan.

4.1.1 Nomenclatura de archivos

La **nomenclatura** es clave: un archivo se accede siempre **por su nombre**.

Características generales:

- los nombres son cadenas de caracteres,
- pueden incluir letras, dígitos y caracteres especiales,
- muchos sistemas permiten nombres de hasta **255 caracteres**.

La **extensión** suele indicar el tipo de archivo (texto, imagen, programa, etc.).

- En **UNIX**, las extensiones son **convenciones**, no reglas impuestas.

- En **Windows**, las extensiones **tienen significado para el sistema operativo**: el sistema asocia extensiones con programas, y al hacer doble clic se ejecuta el programa correspondiente.

Las extensiones ayudan a los programas a decidir **cómo tratar un archivo**, por ejemplo:

4.1.2 Estructura de archivos

Los sistemas de archivos pueden ver un archivo de distintas maneras.

1. Secuencia de bytes

El archivo es sólo una **secuencia de bytes sin estructura**.
El sistema operativo **no impone significado**.

- Máxima flexibilidad.
- El significado lo define el programa.
- Es el modelo usado por **UNIX y Windows**.

2. Secuencia de registros

El archivo es una **secuencia de registros de longitud fija**, con estructura interna.

- Las operaciones leen o escriben **registros completos**.
- Fue común en sistemas **mainframe** (tarjetas perforadas, impresoras).
- Hoy **no se usa** en sistemas de propósito general.

3. Árbol de registros

El archivo es un **árbol de registros**, cada uno con una **llave**.

- Se accede por llave, no por posición.
- El sistema operativo decide dónde colocar los registros.
- Usado en **sistemas comerciales grandes**, no en UNIX/Windows.

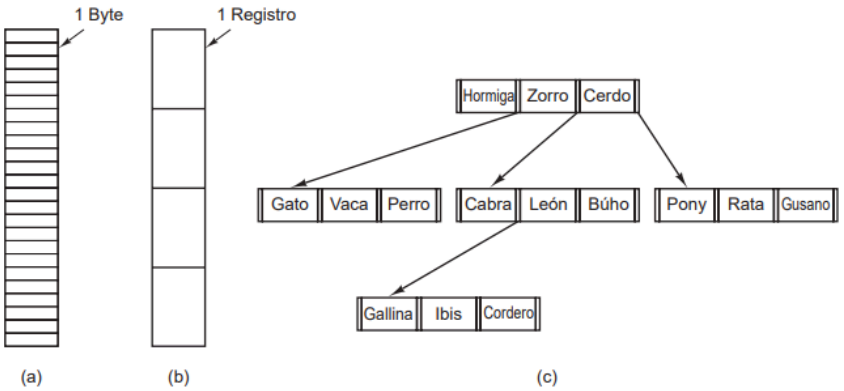


Figura 4-2. Tres tipos de archivos. (a) Secuencia de bytes. (b) Secuencia de registros. (c) Árbol.

4.1.3 Tipos de archivos

Muchos sistemas operativos soportan **varios tipos de archivos**.

En **UNIX y Windows** existen:

- **archivos regulares**
- **directorios**

UNIX además tiene:

- **archivos especiales de caracteres**, para dispositivos de E/S en serie (terminales, impresoras, redes),
- **archivos especiales de bloques**, para modelar discos.

En este capítulo, el foco está en los **archivos regulares**, que contienen información del usuario.

Archivos ASCII y binarios

Los archivos regulares suelen ser de dos tipos:

Archivos ASCII
Son archivos de texto formados por líneas.

- Cada línea termina con retorno de carro, avance de línea o ambos, según el sistema.
- Las líneas no necesitan tener la misma longitud.
- Se pueden mostrar, imprimir y editar directamente.
- Facilitan la conexión entre programas, por ejemplo en canalizaciones de shell.

Su gran ventaja es la **portabilidad y legibilidad**.

Archivos binarios
No son archivos ASCII.

- Al mostrarlos producen caracteres incomprensibles.
- Tienen una **estructura interna conocida por los programas** que los usan.

Un ejemplo típico es el **archivo ejecutable**: aunque es una secuencia de bytes, el sistema operativo exige un formato específico para poder ejecutarlo.
Este formato incluye secciones como encabezado, texto, datos y tablas auxiliares.

Algunos sistemas operativos reconocen varios tipos de archivos binarios, otros sólo el ejecutable.

Un sistema con archivos fuertemente tipificados puede proteger al usuario, pero también **limitarlo**, ya que impide usos no previstos por los diseñadores del sistema.

4.1.4 Acceso a archivos

Los primeros sistemas sólo permitían **acceso secuencial**:

- se lee el archivo desde el principio hasta el final,
- no se puede saltar posiciones intermedias.

Esto era adecuado para **cintas magnéticas**.

Con los discos aparece el **acceso aleatorio**:

- se puede leer cualquier parte del archivo sin leer lo anterior,
- Es esencial para bases de datos y sistemas de reservaciones.

Hay dos formas de indicar la posición de lectura:

- especificarla en cada operación `read`,
- usar una operación `seek` para fijar la posición actual y luego leer secuencialmente.

UNIX y Windows usan este segundo enfoque.

4.1.5 Atributos de archivos

Además del nombre y los datos, cada archivo tiene **atributos** (metadatos).

Estos describen:

- **protección**: quién puede acceder y cómo,
- **propietario y creador**,
- **banderas** que controlan el comportamiento del archivo,
- **fechas** de creación, acceso y modificación,
- **tamaño actual** (y en algunos sistemas, tamaño máximo).

Las **banderas** permiten, por ejemplo:

- marcar un archivo como sólo lectura,
- ocultarlo en los listados,
- indicar si necesita respaldo,
- eliminarlo automáticamente al terminar un proceso.

Los atributos de tiempo son fundamentales para tareas como decidir si un archivo fuente debe recopilarse.

4.1.6 Operaciones de archivos

Los archivos existen para **almacenar información** y permitir que se **recupere posteriormente**.
Para eso, los sistemas operativos ofrecen un conjunto de **llamadas al sistema** que permiten crear, usar y administrar archivos.

Aunque los detalles varían entre sistemas, las **operaciones fundamentales** son esencialmente las mismas.

Creación y eliminación

Create
Crea un archivo vacío, sin datos.
Su objetivo no es almacenar información todavía, sino **anunciar la llegada del archivo** y **establecer sus atributos iniciales** (por ejemplo, protección).

Delete
Elimina un archivo que ya no se necesita.
Esta operación **libera el espacio en disco** que el archivo ocupaba.
Todos los sistemas operativos tienen una llamada explícita para esta función.

Apertura y cierre

Open
Antes de usar un archivo, un proceso debe abrirlo.
Al hacerlo, el sistema operativo:

- lleva los **atributos del archivo** a memoria principal,
- carga la **lista de direcciones de disco** del archivo.

Esto permite que las operaciones posteriores sean rápidas.

Close
Cuando el archivo ya no se necesita, se cierra.

- se liberan las entradas internas usadas por el sistema,
- se fuerza la escritura del último bloque, incluso si no está completamente lleno.

Muchos sistemas limitan la cantidad de archivos abiertos por proceso para obligar a cerrarlos correctamente.

Lectura y escritura

Read
Lee datos desde el archivo.

- normalmente desde la **posición actual.**
- el proceso indica cuántos bytes desea,
- Los datos se colocan en un **búfer** provisto por el llamador.

Write
Escribe datos en el archivo.

- normalmente en la **posición actual**,
- si se escribe al final del archivo, el archivo crece,

- si se escribe en el medio, los datos existentes se sobrescriben y se pierden.

Append
Es una forma restringida de write.

- **sólo permite agregar datos al final del archivo**,
- Algunos sistemas la incluyen, otros no, porque puede simularse con write.

Posicionamiento

Seek
Se utiliza en archivos de acceso aleatorio.

- reposiciona el apuntador del archivo en una posición específica,
- después se puede leer o escribir secuencialmente desde allí.

Este mecanismo es el utilizado por UNIX y Windows.

Atributos y nombres

Get attributes
Permite leer los atributos del archivo.
Es esencial para programas como **make**, que comparan tiempos de modificación para decidir qué recompilar.

Set attributes
Permite modificar ciertos atributos después de creado el archivo.

- protección,
- banderas,
- otros metadatos permitidos por el sistema.

Rename
Cambia el nombre de un archivo existente.
No es estrictamente indispensable, pero simplifica muchas operaciones habituales del usuario.

4.1.7 Un programa de ejemplo que utiliza llamadas al sistema de archivos

El programa **copyfile** ilustra cómo se usan las llamadas al sistema para copiar un archivo en UNIX.

La idea general es simple:

1. abrir el archivo fuente,
2. crear el archivo destino,
3. leer bloques del archivo fuente,
4. escribirlos en el archivo destino,

5. cerrar ambos archivos.

Argumentos y descriptores de archivo

El sistema operativo pasa a `main`:

- `argc`: cantidad de argumentos,
- `argv`: arreglo con los nombres.

Los archivos abiertos se identifican mediante **descriptores de archivo**, que son **enteros pequeños** devueltos por `open` y `creat`.
Todas las operaciones posteriores usan estos enteros para identificar el archivo.

Ciclo de copia

El programa:

- lee bloques de 4096 bytes usando `read`,
- escribe exactamente la cantidad leída usando `write`.

Cuando `read` devuelve:

- un valor positivo: se leyeron bytes,
- 0: fin de archivo,
- negativo: error.

Al llegar al fin del archivo, el ciclo termina.

Cierre y finalización

Después de copiar:

- se cierran ambos archivos con `close`,
- El programa termina indicando éxito o error mediante el **código de estado**.

4.2 Directorios

Para **llevar el registro de los archivos**, los sistemas de archivos utilizan **directorios** (o carpetas).
En muchos sistemas, los directorios **también son archivos**, pero con una función especial: **organizar y localizar otros archivos**.

En esta sección se analiza:

- cómo se organizan los directorios,
- cómo se nombran los archivos dentro de ellos,
- qué operaciones se pueden realizar.

4.2.1 Sistemas de directorios de un solo nivel

La forma más simple de organización es un sistema con **un solo directorio** que contiene todos los archivos.
A este directorio a veces se lo llama **directorio raíz**, aunque en este caso es el único.

Este esquema fue común:

- en las primeras computadoras personales,
- en sistemas con **un solo usuario**,
- incluso en sistemas grandes, como la CDC 6600, para simplificar el software.

Ventajas

- extrema **simpleza**,
- **localización rápida** de archivos, ya que sólo hay un lugar donde buscar.

Desventajas

- no escala: con muchos archivos se vuelve imposible organizar o encontrar información.

Por eso hoy sólo se usa en **sistemas muy simples**, como dispositivos incrustados (cámaras, teléfonos, reproductores).

4.2.2 Sistemas de directorios jerárquicos

Un solo nivel es insuficiente para usuarios modernos que manejan **miles de archivos**.
La solución es organizar los archivos en una **jerarquía**, es decir, un **árbol de directorios**.

En este esquema:

- los directorios pueden contener archivos y **subdirectorios**,
- cada usuario puede tener su **propio directorio raíz**,
- los archivos se agrupan de forma **natural y lógica**.

Este modelo permite:

- separar proyectos,
- organizar por temas,
- compartir un mismo servidor entre múltiples usuarios sin mezclar archivos.

Por esta razón, **casi todos los sistemas de archivos modernos** usan directorios jerárquicos.

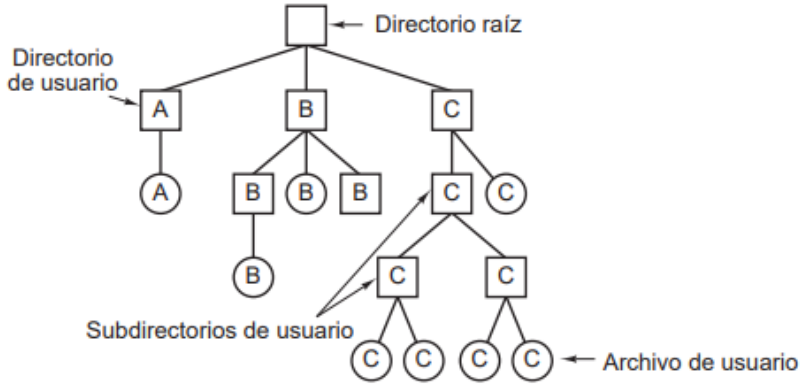


Figura 4-7. Un sistema de directorios jerárquico.

Entradas especiales: `.` y `..`

En los sistemas jerárquicos, cada directorio contiene dos entradas especiales:

- `.` → el directorio actual,
- `..` → el directorio padre.

Estas entradas permiten **navegar por el árbol** sin escribir rutas completas.

Ejemplo conceptual:

- subir un nivel usando `..`,
- referirse al directorio actual usando `.`

Esto simplifica muchas operaciones, como copias entre directorios relacionados.

4.2.4 Operaciones de directorios

Las operaciones sobre directorios varían más entre sistemas que las de archivos, pero las ideas básicas son comunes.

Creación y eliminación

Create
Crea un directorio nuevo.
El sistema coloca automáticamente las entradas `.` y `..`.

Delete
Elimina un directorio, **sólo si está vacío**.
Un directorio que contiene únicamente `.` y `..` se considera vacío.

Lectura del contenido

Opendir
Abre un directorio para poder leer su contenido.

Readdir
Devuelve la **siguiente entrada** del directorio en un formato estándar.
Evita que el programador tenga que conocer la estructura interna del directorio.

Closedir
Cierra el directorio y libera las estructuras internas.

Modificación de nombres y vínculos

Rename
Permite cambiar el nombre de un directorio, de forma similar a un archivo.

Link (vínculo duro) - Hard Link
Permite que **un mismo archivo aparezca en varios directorios**.
Internamente:

- se incrementa el contador del nodo-i,
- no se duplican los datos.

Unlink
Elimina una entrada de directorio.

- si es la última referencia, el archivo se elimina,
- si hay más vínculos, sólo se elimina ese nombre.

Un link (vínculo) es una forma de hacer que un archivo aparezca con más de un nombre o desde otro lugar del sistema de archivos.

Es decir, un directorio puede tener una entrada que apunte a algo que está almacenado en otro lugar.

En UNIX, **eliminar un archivo es en realidad hacer unlink**.

Vínculos simbólicos

Un **vínculo simbólico** no apunta directamente al archivo, sino a un **archivo pequeño que contiene un nombre de ruta**.

Características:

- pueden cruzar discos,
- pueden apuntar a archivos remotos,
- son más flexibles que los vínculos duros.

Desventaja:

- son **menos eficientes**, porque el sistema debe resolver el nombre adicional.

4.3 IMPLEMENTACIÓN DE SISTEMAS DE ARCHIVOS

En esta sección se cambia el **punto de vista del usuario** al **punto de vista del implementador**.
El usuario se preocupa por **nombres, operaciones y jerarquías**; el implementador por **cómo se almacenan archivos y directorios**, **cómo se administra el espacio en disco** y **cómo lograr eficiencia y confiabilidad**.

4.3.1 Distribución del sistema de archivos

Los sistemas de archivos se almacenan en **discos**, que pueden dividirse en **particiones**, cada una con su propio sistema de archivos.

- El **sector 0** del disco es el **MBR (Master Boot Record)**, que contiene la **tabla de particiones** y el código de arranque.
- El **BIOS** ejecuta el MBR, que localiza la **partición activa**, lee su **bloque de arranque** y carga el sistema operativo.

Dentro de una partición, la organización típica incluye:

- **Bloque de arranque**
- **Superbloque**: contiene parámetros clave del sistema de archivos (tipo, tamaño, información administrativa).
- **Información de bloques libres** (mapa de bits o lista).
- **Nodos-i** (uno por archivo).
- **Directorio raíz**
- **Archivos y subdirectorios**

El **superbloque** se lee en memoria al arrancar o al montar el sistema de archivos.

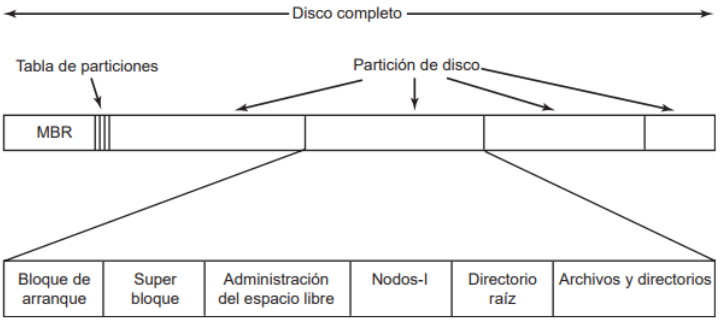


Figura 4-9. Una posible distribución del sistema de archivos.

4.3.2 Implementación de archivos

El problema central es **cómo registrar qué bloques de disco pertenecen a cada archivo**.

Asignación contigua

Cada archivo ocupa una **serie contigua de bloques**.

Ventajas

- Implementación simple (primer bloque + cantidad de bloques).
- **Excelente rendimiento de lectura**: una sola búsqueda y luego lectura secuencial.

Desventaja clave

- **Fragmentación externa**.
- Es necesario conocer el **tamaño final del archivo** al crearlo.

Uso típico

- Medios de solo escritura como **CD-ROM** y **DVD**.

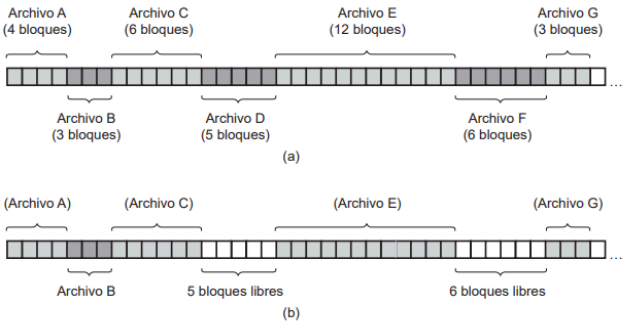


Figura 4-10. (a) Asignación contigua de espacio de disco para siete archivos. (b) El estado del disco después de haber removido los archivos *D* y *F*.

Asignación de lista enlazada

Cada archivo es una **lista enlazada de bloques**, donde cada bloque apunta al siguiente.

Ventajas

- No hay fragmentación externa.
- Se puede usar cualquier bloque libre.

Desventajas

- **Acceso aleatorio muy lento** (hay que recorrer la lista).
- Se pierde espacio en cada bloque por el apuntador.
- Tamaños de bloque no alineados a potencias de dos.

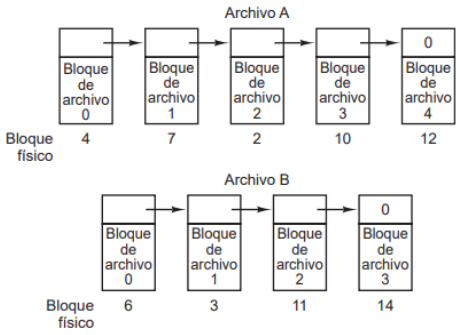


Figura 4-11. Almacenamiento de un archivo como una lista enlazada de bloques de disco.

Lista enlazada con tabla en memoria (FAT)

Los apuntadores se sacan de los bloques y se colocan en una **tabla en memoria** llamada **FAT**.

Mejoras

- El bloque completo queda disponible para datos.
- El acceso aleatorio es más rápido porque la cadena está en memoria.

Problema grave

- La **tabla FAT debe estar completa en memoria**.
- No escala bien en discos grandes (consume cientos de MB).

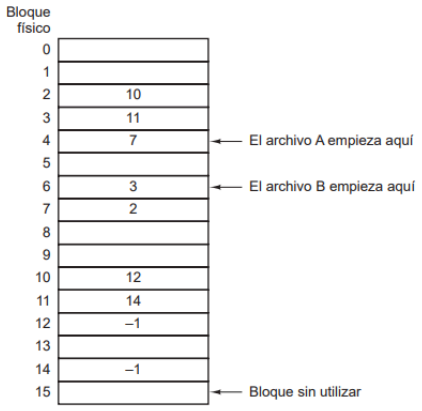


Figura 4-12. Asignación de lista enlazada que utiliza una tabla de asignación de archivos en la memoria principal.

Nodos-i

Otro método para llevar un registro de qué bloques pertenecen a cuál archivo es asociar con cada archivo una estructura de datos conocida como nodo-i (nodo-índice), Esta lista los atributos y las direcciones de disco de los bloques del archivo. Dado el nodo-i, entonces es posible encontrar todos los bloques del archivo.

Cada archivo tiene un **nodo-i** que contiene:

- **Atributos del archivo**
- **Direcciones de los bloques de disco**

El i-nodo NO guarda el nombre del archivo.

El nombre está en el directorio.

Un hard link crea otro nombre que apunta al mismo i-nodo.

Qué guarda el i-nodo para manejar esto

El i-nodo tiene un campo llamado:

contador de enlaces (link count)

El archivo **solo se elimina cuando llega a 0**.

Directorio → nombre del archivo → i-nodo → direcciones de bloques → bloques del disco → datos del archivo

Ventaja clave

- Sólo los nodos-i de archivos **abiertos** deben estar en memoria.
- La memoria requerida depende del número de archivos abiertos, no del tamaño del disco.

Archivos grandes

- Se usan **bloques indirectos** que contienen más direcciones de bloques.

Este esquema es el utilizado por **UNIX**.

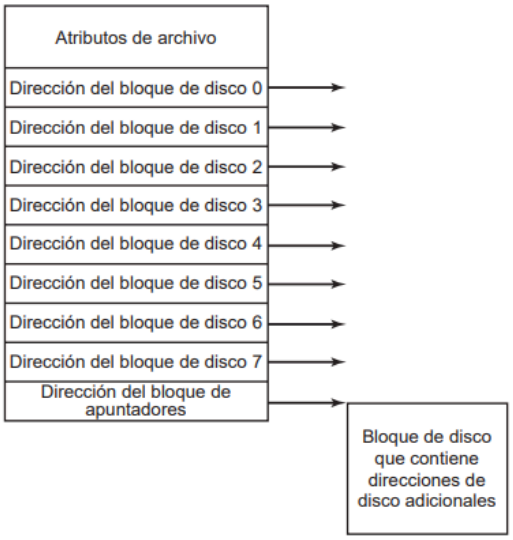


Figura 4-13. Un nodo-i de ejemplo.

4.3.3 Implementación de directorios

La función principal del directorio es **asociar el nombre ASCII del archivo con la información necesaria para localizar sus bloques**.

Dos enfoques:

- **Atributos en la entrada de directorio** (ej. Windows).
- **Entrada con nombre + número de nodo-i**, atributos en el nodo-i (ej. UNIX).

Búsqueda en directorios grandes

- **Búsqueda lineal** (simple pero lenta).
- **Tablas de hash** (rápidas pero complejas).
- **Caché de búsquedas** (efectiva si pocos archivos se usan mucho).

4.3.4 Archivos compartidos

Compartir archivos convierte el sistema de archivos en un **grafo acíclico dirigido (DAG)**.

Vínculo duro - Hard Link

- Varias entradas apuntan al **mismo nodo-i**.
- Problemas al eliminar archivos compartidos.
- El propietario original sigue siendo responsable del archivo.

Qué pasa al borrar un hard link

- Cada i-nodo tiene un **contador de enlaces (link count)**.
- Cada **hard link** incrementa ese contador.

Cuando borrás un nombre (unlink):

1. **Se elimina la entrada del directorio.**
2. **El contador del i-nodo disminuye en 1.**

Si el contador llega a:

0

entonces:

- se elimina el **i-nodo**
- se liberan **los bloques del archivo**

Vínculo simbólico - Soft Link

- Es un archivo que contiene una **ruta** a otro archivo.
- No comparte nodo-i.

- Más flexible (puede cruzar discos o máquinas).
- Más lento y consume más recursos.

Qué pasa al borrar un soft link

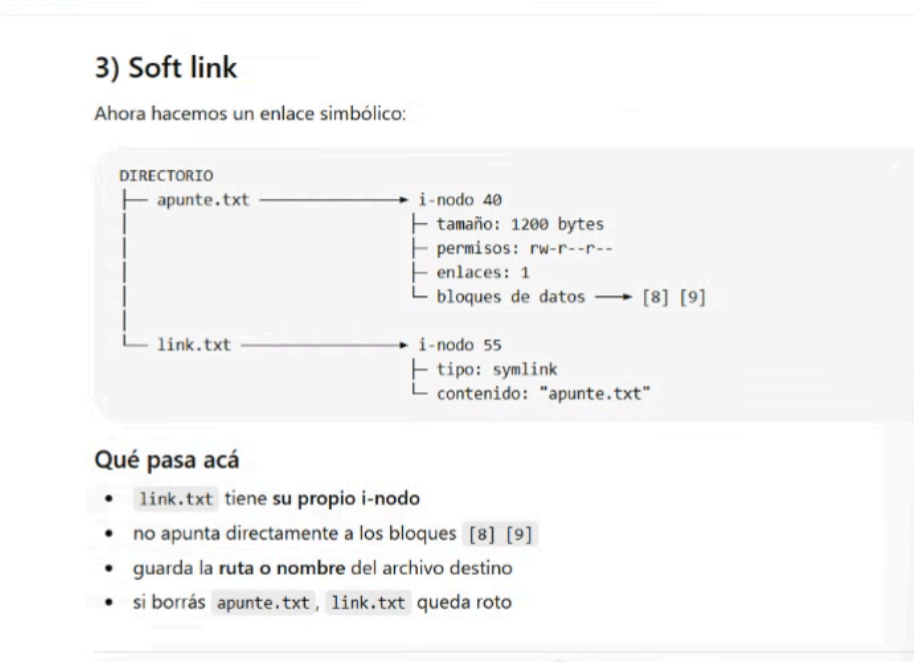
Un **soft link es un archivo independiente**.

Si borrás el soft link:

- solo se borra **ese archivo link**
- **el archivo original no se afecta**

Si borrás el archivo original:

- el **soft link sigue existiendo**
- pero queda **roto (broken link)** porque apunta a una ruta que ya no existe.

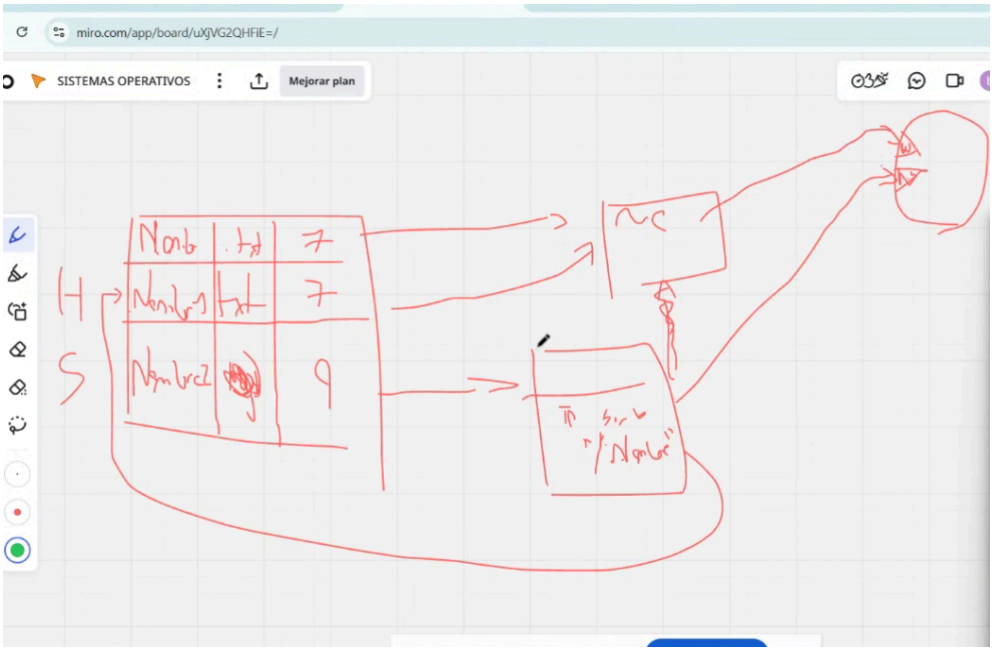


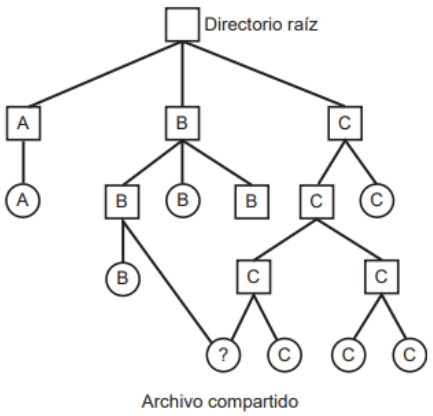
Otra forma todavía más simple

```
ARCHIVO NORMAL
apunte.txt ──► i-nodo 40 ──► datos

HARD LINK
apunte.txt ─┐
            ├──► i-nodo 40 ──► datos
resumen.txt ─┘

SOFT LINK
apunte.txt ──────► i-nodo 40 ──► datos
link.txt   ──────► i-nodo 55 ──► "apunte.txt"
```





Archivo compartido

4.3.5 Sistemas de archivos estructurados por registro (LFS)

Idea central: **el disco se trata como un registro secuencial**.

- Las escrituras pequeñas se acumulan en memoria.
- Se escriben juntas como **segmentos grandes y contiguos**.
- Mejora drásticamente el rendimiento de escrituras pequeñas.

Incluye un **hilo limpiador** que:

- Elimina bloques obsoletos.
- Reescribe los bloques aún válidos.
- Mantiene el disco como un **búfer circular**.

4.4 ADMINISTRACIÓN Y OPTIMIZACIÓN DE SISTEMAS DE ARCHIVOS

Que un sistema de archivos funcione es sólo el primer paso. En la práctica, debe hacerlo de manera **eficiente**, **robusta** y **confiable**, aun frente a fallas, grandes volúmenes de datos y usos reales. Esta sección analiza los principales problemas de **administración de disco**, **respaldo**, **consistencia** y **rendimiento**.

4.4.1 Administración del espacio en disco

Los archivos se almacenan en **disco**, por lo que administrar ese espacio es una cuestión central.

En teoría, un archivo de *n* bytes podría almacenarse como una secuencia contigua, pero esto genera problemas cuando el archivo crece, ya que habría que moverlo. Como mover archivos en disco es muy costoso, **casi todos los sistemas de archivos dividen los archivos en bloques de tamaño fijo**, que no necesitan ser contiguos.

Tamaño de bloque

Elegir el tamaño del bloque es un **compromiso entre espacio y tiempo**:

- **Bloques grandes**
    - Menos búsquedas y retrasos rotacionales
    - Mejor rendimiento
    - Mucho desperdicio de espacio en archivos pequeños
- **Bloques pequeños**
    - Mejor aprovechamiento del espacio
    - Peor rendimiento por múltiples accesos al disco

Los estudios empíricos citados por Tanenbaum muestran que **la mayoría de los archivos son pequeños**, pero **la mayor parte del espacio del disco está ocupado por pocos archivos grandes**.
Esto implica que el desperdicio de espacio en archivos pequeños **no es tan grave**, mientras que el rendimiento sí lo es.

Históricamente se eligieron tamaños entre **1 KB y 4 KB**, pero con discos modernos de varios terabytes, **bloques más grandes (por ejemplo 64 KB)** pueden ser razonables, aceptando el desperdicio de espacio.

Registro de bloques libres

Una vez elegido el tamaño de bloque, el sistema debe llevar registro de **qué bloques están libres**.

Los dos métodos principales son:

- **Lista enlazada de bloques libres**
  Cada bloque contiene direcciones de otros bloques libres.
  Es simple, pero puede generar muchas operaciones de E/S cuando se crean y eliminan archivos pequeños y temporales.
- **Mapa de bits**
  Un bit por bloque indica si está libre u ocupado.
  Usa mucho menos espacio y facilita asignar bloques cercanos, reduciendo el movimiento del brazo del disco.

Ambos métodos tienen ventajas y desventajas, y la elección depende del uso real del sistema. Este punto ilustra un problema clásico del diseño de sistemas operativos: **las decisiones deben tomarse sin conocer exactamente el comportamiento futuro de los usuarios**.

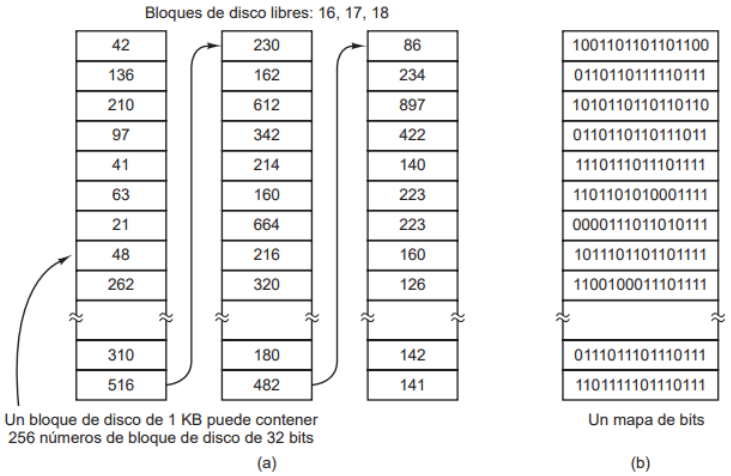


Figura 4-22. (a) Almacenamiento de la lista de bloques libres en una lista enlazada. (b) Un mapa de bits.

Unidad 5. ENTRADA/SALIDA

Además de procesos, memoria y archivos, el sistema operativo **controla todos los dispositivos de entrada/salida**.
Sus responsabilidades principales son:

- emitir comandos a los dispositivos
- captar y manejar interrupciones
- detectar y tratar errores
- ofrecer una **interfaz uniforme**, independiente del dispositivo

El objetivo central es **ocultar la complejidad del hardware** y presentar mecanismos simples y consistentes al resto del sistema.

5.1 Principios del hardware de E/S

El enfoque del capítulo es **cómo se programa el hardware de E/S**, no su construcción física.
Sin embargo, la programación está fuertemente ligada a la forma en que el hardware opera.

5.1.1 Dispositivos de E/S

Los dispositivos de E/S se clasifican en **dos grandes categorías**.

Dispositivos de bloque

- Almacenan información en **bloques de tamaño fijo**
- Cada bloque tiene una **dirección**
- Permiten lectura y escritura **independiente** de cada bloque
- Transferencias en uno o más bloques completos

Ejemplos: discos duros, CD-ROM, memorias USB.

La propiedad clave es el **acceso aleatorio**.

Dispositivos de carácter

- Transfieren un **flujo de caracteres**
- No son direccionables
- No poseen operación de búsqueda

Ejemplos: teclados, impresoras, interfaces de red, ratones.

Este modelo no cubre perfectamente todos los dispositivos:

- los relojes sólo generan interrupciones
- las pantallas por memoria mapeada no encajan bien

Aun así, la distinción **bloque / carácter** permite construir software de E/S **independiente del dispositivo**.

5.1.2 Controladores de dispositivos

Un dispositivo de E/S suele dividirse en:

- **componente mecánico** (el dispositivo físico)
- **controlador de dispositivo** (componente electrónico).

El controlador:

- traduce señales físicas en datos utilizables
- realiza corrección de errores
- ofrece una interfaz uniforme al sistema operativo

El sistema operativo **no controla directamente el hardware**, sino que configura el controlador y delega el trabajo fino.

5.1.3 E/S por asignación de memoria

La CPU se comunica con los dispositivos mediante **registros de control** y **búferes de datos**.
Existen dos esquemas para acceder a ellos.

Puertos de E/S separados

- Espacio de direcciones distinto al de memoria
- Instrucciones especiales: `IN`, `OUT`
- Usado en sistemas clásicos y mainframes

Memoria y E/S son **espacios independientes**.

Solo el sistema operativo puede acceder, los procesos de usuario no pueden.

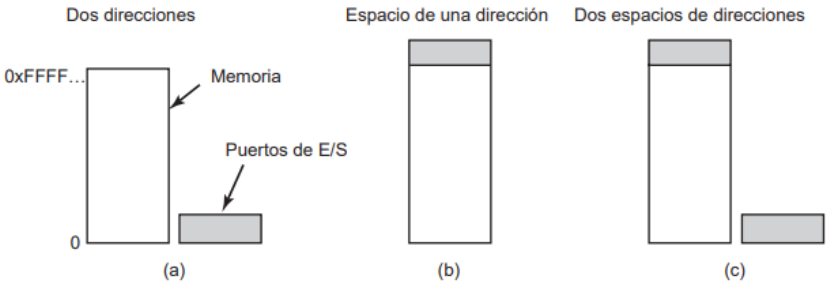


Figura 5-2. (a) Espacio separado de E/S y memoria. (b) E/S por asignación de memoria. (c) Híbrido.

E/S por asignación de memoria (memory-mapped I/O)

- Los registros de control se asignan a direcciones de memoria
- Se acceden como variables comunes
- Introducido con la PDP-11

Ventajas:

- controladores escritos completamente en C

- no requiere instrucciones especiales
- permite protección por páginas
- cualquier instrucción de memoria puede usarse sobre E/S

Desventajas de la E/S por asignación de memoria

- riesgo de **caché incorrecta** sobre registros de control
- requiere soporte de hardware para deshabilitar **caché selectivamente**
- mayor complejidad en arquitecturas con **buses separados**

Por eso existen arquitecturas:

- de un solo bus
- de buses múltiples
- híbridas (como Pentium)

No hay una solución perfecta: todo implica **concesiones de diseño**.

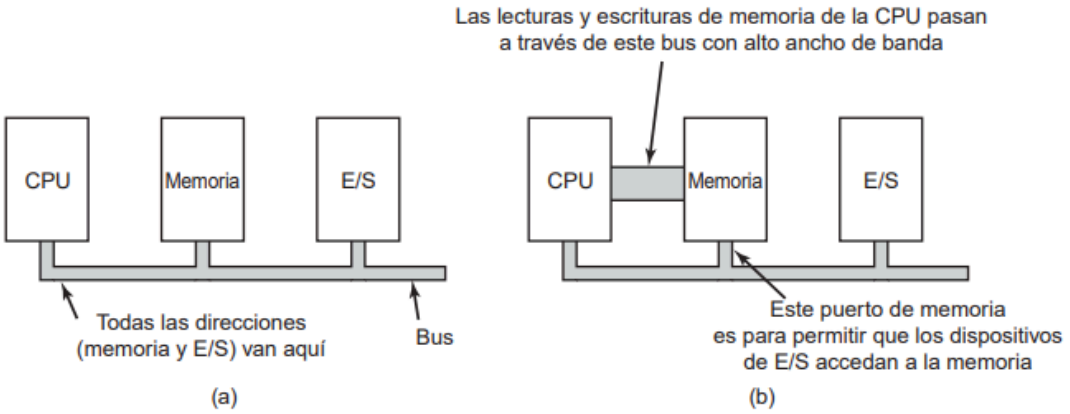


Figura 5-3. (a) Arquitectura con un solo bus. (b) Arquitectura con bus dual.

5.1.4 Acceso Directo a Memoria (DMA)

DMA permite transferir datos entre dispositivos y memoria **sin intervención constante de la CPU**.

Componentes:

- controlador de DMA
- registros de dirección
- contador de bytes
- registros de control

Funcionamiento del DMA

1. La CPU programa el controlador DMA
2. El dispositivo coloca datos en su búfer interno

3. El DMA transfiere datos directamente a memoria
4. Al finalizar, el DMA interrumpe a la CPU

La CPU **no copia los datos**, sólo coordina.

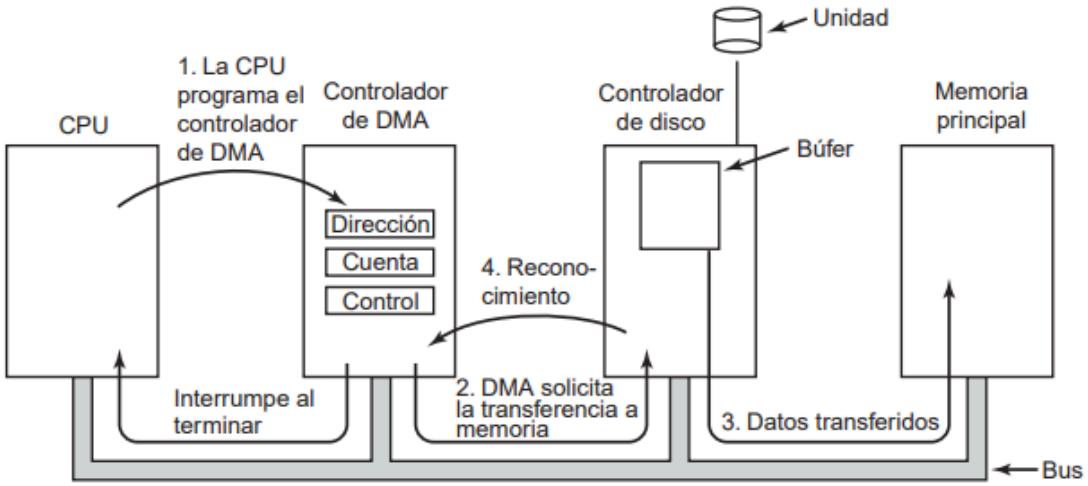


Figura 5-4. Operación de una transferencia de DMA.

Modos de DMA

- **robo de ciclo**: el DMA toma el bus una palabra a la vez
- **modo de ráfaga**: transfiere bloques completos

El modo de ráfaga es más eficiente, pero puede bloquear la CPU por más tiempo.

Consideraciones adicionales

- DMA suele usar **direcciones físicas**
- el SO debe traducir direcciones virtuales
- el búfer interno del controlador permite:
    - verificar errores antes de transferir
    - desacoplar el ritmo del dispositivo del bus

DMA vs CPU

DMA no siempre es conveniente:

- la CPU puede ser más rápida
- en sistemas simples se evita DMA por costo y complejidad
- frecuente en sistemas embebidos

5.1.5 Repaso de las interrupciones

Una **interrupción** ocurre cuando un dispositivo de E/S termina una operación y necesita atención del sistema operativo.

Si las interrupciones están habilitadas, el dispositivo **impone una señal en el bus**, que es detectada por el **controlador de interrupciones** de la tarjeta principal.

El controlador de interrupciones:

- decide si la interrupción puede atenderse de inmediato
- maneja prioridades entre dispositivos
- coloca un número identificador del dispositivo en las líneas de dirección
- envía la señal de interrupción a la CPU

Si hay otra interrupción en progreso o una de mayor prioridad, la petición se **pospone**, y el dispositivo continúa solicitando atención.

Atención de una interrupción por la CPU

Cuando la CPU recibe la señal:

1. **detiene la ejecución actual**
2. usa el número recibido como índice en el **vector de interrupciones**
3. obtiene la dirección del **procedimiento de servicio de interrupciones**
4. comienza a ejecutarlo

Las interrupciones y las trampas suelen usar el mismo mecanismo y, en muchos sistemas, el mismo vector.

Al iniciar el procedimiento de servicio, el sistema operativo **reconoce la interrupción** escribiendo en el controlador de interrupciones. Esto le indica que puede aceptar nuevas interrupciones y evita condiciones de competencia.

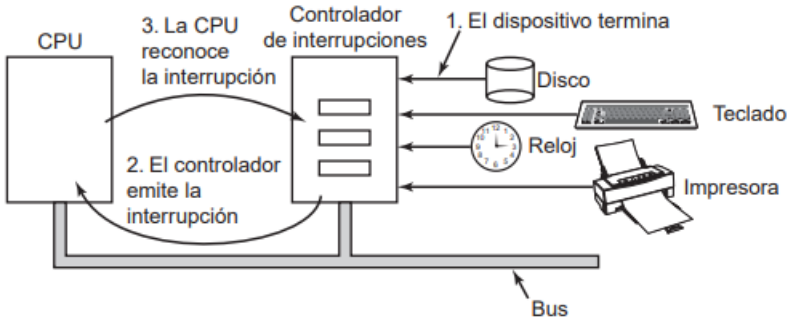


Figura 5-5. Cómo ocurre una interrupción. Las conexiones entre los dispositivos y el controlador de interrupciones en realidad utilizan líneas de interrupción en el bus, en vez de cables dedicados.

Guardado del estado de la CPU

Antes de ejecutar el procedimiento de servicio, el hardware **guarda el estado** del proceso interrumpido.

Como mínimo se guarda el **contador del programa (PC)**, y en algunos sistemas también registros visibles e internos.

Este estado debe almacenarse en algún lugar:

- **registros internos**: impiden reconocer nuevas interrupciones hasta leer todo el estado
- **pila**: es el método más común

Usar la pila del proceso de usuario es peligroso, ya que:

- el apuntador puede ser inválido
- puede generar fallos de página durante la interrupción

Por eso, normalmente se utiliza la **pila del kernel**, aunque esto puede requerir cambios de contexto de la MMU y afectar la caché y el TLB, aumentando el tiempo de atención de la interrupción.

Interrupciones precisas e imprecisas

En sistemas antiguos, las interrupciones ocurrían **entre instrucciones**, lo que garantizaba un estado bien definido.
En CPUs modernas con **líneas de tuberías** y **ejecución superescalar**, esto deja de ser cierto.

Al ocurrir una interrupción:

- varias instrucciones pueden estar en ejecución parcial
- el contador del programa puede no reflejar el límite real entre lo ejecutado y lo no ejecutado
- las instrucciones pueden haberse ejecutado fuera de orden

Interrupciones precisas

Una **interrupción precisa** deja a la máquina en un estado completamente definido y cumple cuatro propiedades:

1. El contador del programa se guarda en un lugar conocido.
2. Todas las instrucciones anteriores al PC se han ejecutado por completo.
3. Ninguna instrucción posterior al PC ha tenido efecto.
4. El estado de ejecución de la instrucción apuntada por el PC es conocido.

Esto permite:

- reiniciar correctamente el proceso
- manejar fallos de página y trampas con seguridad

Interrupciones imprecisas

Una **interrupción imprecisa** no garantiza un estado claro:

- instrucciones cercanas al PC pueden estar en distintos grados de avance
- instrucciones más nuevas pueden estar más completas que las antiguas
- el sistema operativo debe reconstruir el estado real

Para hacerlo posible, la CPU suele guardar **gran cantidad de estado interno**, lo que:

- complica enormemente el sistema operativo
- ralentiza la atención de interrupciones
- reduce el rendimiento global

Esto genera una paradoja: CPUs muy rápidas que se ven limitadas por interrupciones lentas.

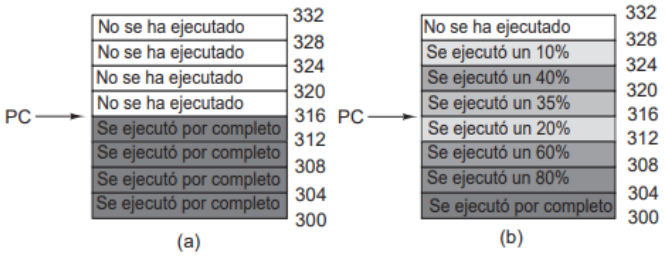


Figura 5-6. (a) Una interrupción precisa. (b) Una interrupción imprecisa.

Decisiones de diseño

Algunas arquitecturas:

- hacen precisas las interrupciones de E/S
- permiten interrupciones imprecisas para errores fatales

Otras ofrecen un bit para forzar **todas las interrupciones precisas**, lo cual:

- simplifica el sistema operativo
- incrementa mucho la complejidad del hardware
- impacta negativamente el rendimiento

La familia **Pentium** implementa interrupciones precisas para mantener compatibilidad con software antiguo, pagando el costo en complejidad del chip.

5.2 Fundamentos del software de E/S

Luego de analizar el hardware de E/S, el foco pasa al **software de E/S**, es decir, a cómo el sistema operativo organiza, abstrae y controla el uso de los dispositivos.
Primero se definen los **objetivos** del software de E/S y luego las **formas en que se realiza la E/S** desde el punto de vista del sistema operativo.

5.2.1 Objetivos del software de E/S

El objetivo central del software de E/S es lograr la **independencia de dispositivos**.
Esto significa que los programas no deben depender del tipo concreto de dispositivo que se utiliza. Un mismo programa debe poder leer datos desde un disco duro, un CD-ROM, una memoria USB o el teclado sin ser modificado. El sistema operativo es el encargado de ocultar las diferencias reales entre los dispositivos, aunque internamente requieran comandos y secuencias distintas.

Un objetivo estrechamente relacionado es la **denominación uniforme**.
Los archivos y dispositivos deben identificarse únicamente por un nombre (una cadena o un entero), sin que ese nombre revele el tipo de dispositivo. En UNIX, los dispositivos se integran a la jerarquía del sistema de archivos, por lo que acceder a un archivo en un disco o a una memoria USB se hace de la misma forma: mediante un nombre de ruta. El usuario no necesita saber qué dispositivo físico está detrás.

Otro objetivo fundamental es el **manejo de errores**.
Los errores deben resolverse lo más cerca posible del hardware. Si el controlador detecta un error de lectura, debe intentar corregirlo. Si no puede, el software del dispositivo puede reintentar la operación. Muchos errores son transitorios y desaparecen al repetir la operación. Sólo cuando los niveles inferiores no pueden resolver el problema, se informa a los niveles superiores. En muchos casos, la recuperación ocurre de manera transparente, sin que el resto del sistema se entere.

También es clave la distinción entre **E/S síncrona (de bloqueo)** y **E/S asíncrona (controlada por interrupciones)**.
Aunque la E/S real suele ser asíncrona, los programas de usuario son más simples de escribir si las operaciones parecen de bloqueo. Por ejemplo, una llamada a `read` suspende al proceso hasta que los datos están disponibles. El sistema operativo debe hacer que una operación basada en interrupciones se comporte como una operación de bloqueo desde el punto de vista del usuario.

El **uso de búferes** es otro aspecto central.
Muchas veces los datos no pueden colocarse directamente en su destino final. Por ejemplo, al recibir un paquete de red, el sistema operativo necesita almacenarlo primero para analizarlo. Además, algunos dispositivos tienen restricciones temporales estrictas, por lo que los datos deben almacenarse en búferes para desacoplar la velocidad de producción y consumo. El uso de búferes implica copias adicionales y tiene un impacto importante en el rendimiento de la E/S.

Un buffer es un dispositivo de almacenamiento temporal.

Por último, el software de E/S debe manejar **dispositivos compartidos y dispositivos dedicados**.
Los dispositivos compartidos, como los discos, pueden ser usados por múltiples procesos al mismo tiempo sin conflictos. Los dispositivos dedicados, como las unidades de cinta, deben ser asignados a un solo usuario hasta que termine su uso. Esto introduce problemas adicionales, como interbloqueos, que el sistema operativo debe evitar.

5.2.2 E/S programada

La **E/S programada** es la forma más simple de realizar E/S. En este método, la **CPU hace todo el trabajo**.

Un proceso de usuario prepara los datos en un búfer y solicita al sistema operativo que realice la operación. El sistema operativo copia los datos al espacio de kernel y luego entra en un ciclo en el que:

- escribe un dato en el dispositivo,
- verifica si el dispositivo está listo,
- espera activamente hasta que pueda continuar.

Este mecanismo se conoce como **sondeo** u **ocupado en espera**. La CPU verifica continuamente el estado del dispositivo hasta que esté listo para aceptar el siguiente dato.

La E/S programada es simple, pero **ineficiente**, ya que la CPU queda ocupada durante toda la operación de E/S. Sólo es razonable cuando el tiempo de espera es muy corto o en sistemas incrustados donde la CPU no tiene otro trabajo que realizar.

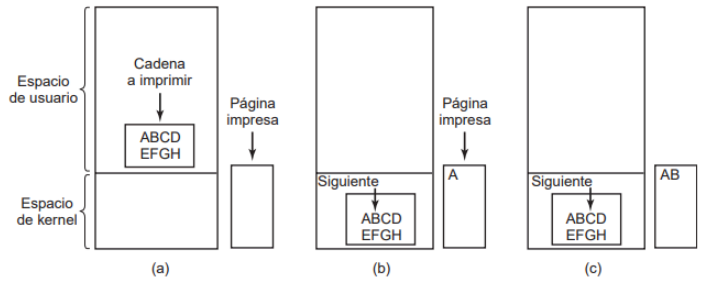


Figura 5-7. Pasos para imprimir una cadena.

5.2.3 E/S controlada por interrupciones

Para evitar que la CPU pierda tiempo esperando, se utiliza la **E/S controlada por interrupciones**.

En este esquema, el sistema operativo copia los datos al espacio de kernel y envía el primer dato al dispositivo. Luego, el proceso que solicitó la E/S se bloquea y la CPU puede ejecutar otros procesos.
Cuando el dispositivo termina de procesar el dato y está listo para continuar, genera una **interrupción**.

El **procedimiento de servicio de interrupciones** se encarga de:

- enviar el siguiente dato al dispositivo, si quedan datos por transferir,
- o desbloquear al proceso de usuario si la operación terminó.

De esta forma, la CPU sólo interviene cuando ocurre una interrupción y no pierde tiempo esperando activamente. El costo es que se produce una interrupción por cada unidad transferida.

5.2.4 E/S mediante el uso de DMA

La desventaja de la E/S por interrupciones es que genera **muchas interrupciones**.
Para reducir este costo, se utiliza **DMA (Acceso Directo a Memoria)**.

Con DMA, la CPU programa al controlador de DMA indicando:

- qué datos transferir,
- desde dónde,
- hacia dónde,
- y cuántos bytes.

Luego, el controlador de DMA realiza la transferencia sin intervención de la CPU. La CPU queda libre para ejecutar otros procesos y sólo se genera una interrupción cuando **toda la transferencia termina**, no por cada dato.

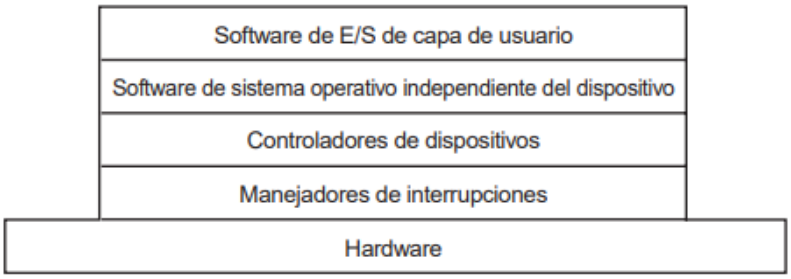
La ventaja principal es reducir drásticamente el número de interrupciones.
La desventaja es que el controlador de DMA puede ser más lento que la CPU. Si la CPU no tiene otro trabajo que realizar o el controlador de DMA no puede manejar la velocidad del dispositivo, puede ser preferible usar E/S por interrupciones o incluso E/S programada.

En la práctica, **la mayor parte del tiempo vale la pena usar DMA**.

5.3 Capas del software de E/S

El software de E/S se organiza en **capas**, cada una con una función bien definida y con interfaces claras hacia las capas vecinas.
Esta organización permite **ocultar la complejidad del hardware**, separar responsabilidades y facilitar la extensión del sistema con nuevos dispositivos.

De abajo hacia arriba, las capas son:
hardware, manejadores de interrupciones, controladores de dispositivos, software de E/S independiente del dispositivo y software de E/S en espacio de usuario.



5.3.1 Manejadores de interrupciones

Las interrupciones son inevitables en la mayoría de las operaciones de E/S y deben **ocultarse lo más posible del resto del sistema operativo**.

La idea central es que el controlador que inicia una operación de E/S **se bloquee** hasta que la operación termine y ocurra la interrupción.

Cuando el hardware genera una interrupción, el procedimiento de servicio de interrupciones:

- identifica qué dispositivo generó la interrupción,
- obtiene información del controlador del dispositivo,
- y **desbloquea** al controlador que estaba esperando.

Este modelo funciona mejor cuando los controladores están estructurados como **procesos del kernel**, con su propio estado, pila y contador del programa.

El procesamiento de una interrupción implica mucho más que “atenderla y volver”.
El sistema operativo debe guardar registros, establecer un contexto de ejecución, manejar la MMU y el TLB si existen, ejecutar el procedimiento de servicio, decidir qué proceso ejecutar a continuación y restaurar el estado del proceso elegido.

Por este motivo, **las interrupciones son costosas** en términos de tiempo de CPU, especialmente en sistemas con memoria virtual y hardware moderno.

5.3.2 Drivers de dispositivos

Cada dispositivo de E/S necesita **código específico** para ser controlado.
Este código es el **driver**, generalmente escrito por el fabricante del dispositivo y dependiente del sistema operativo.

Los drivers difieren enormemente entre sí porque los dispositivos son muy distintos.
Un driver de disco debe conocer sectores, pistas y cilindros; un driver de ratón maneja desplazamientos y botones. Por eso, **no existe un driver universal**.

En la mayoría de los sistemas operativos, los drivers se ejecutan **dentro del kernel**, porque necesitan acceso directo a los registros del hardware.
Aunque es posible ejecutar drivers en espacio de usuario (como en MINIX 3), el modelo habitual es que formen parte del kernel.

Para permitir que drivers de terceros se integren al sistema, el sistema operativo define una **arquitectura estándar de drivers**.
Cada clase de dispositivo (bloque o carácter) debe implementar un conjunto bien definido de funciones, como leer, escribir, inicializar o apagar el dispositivo. Estas funciones se acceden mediante una tabla de apuntadores, que actúa como interfaz entre el driver y el resto del sistema.

El funcionamiento típico de un driver es:

- validar los parámetros,
- traducir solicitudes abstractas a términos concretos del dispositivo,
- verificar si el dispositivo está ocupado,
- enviar una secuencia de comandos al hardware,
- bloquearse si es necesario,

- manejar la interrupción,
- verificar errores y devolver el resultado.

Los drivers deben ser **reentrantes**, ya que pueden ser interrumpidos mientras se están ejecutando.
Además, en sistemas con conexión en caliente, deben manejar la aparición o desaparición inesperada de dispositivos sin dañar las estructuras del kernel.

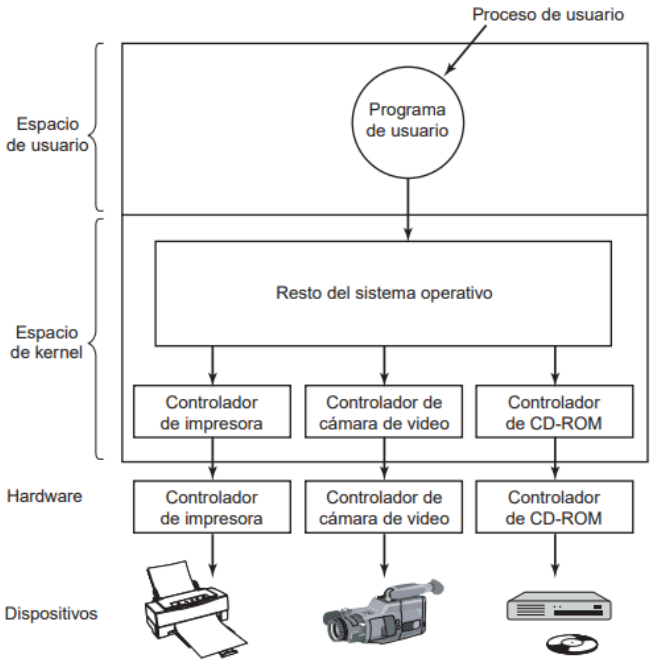


Figura 5-12. Posicionamiento lógico del software controlador de dispositivos. En realidad toda la comunicación entre el software controlador y los controladores de dispositivos pasa a través del bus.

5.3.3 Software de E/S independiente del dispositivo

Esta capa implementa **funciones comunes a todos los dispositivos**, ocultando diferencias específicas del hardware.

Su función principal es proporcionar una **interfaz uniforme** hacia los niveles superiores.
Gracias a esto, el sistema operativo no necesita cambiar cada vez que se agrega un nuevo dispositivo compatible con una clase existente.

También se encarga del **nombramiento y protección** de dispositivos.
Los dispositivos aparecen como archivos especiales, por lo que se aplican las reglas normales de permisos. El número mayor identifica al controlador y el número menor identifica al dispositivo concreto.

Otra responsabilidad clave es el **uso de búferes**.
El uso de búfer mejora el rendimiento y desacopla la velocidad del dispositivo de la del proceso.
Se utilizan técnicas como:

- búfer en kernel,
- doble búfer,
- búfer circular.

Estas técnicas evitan pérdidas de datos y reducen el número de activaciones de procesos, pero a costa de múltiples copias de datos, lo que puede degradar el rendimiento.

El **reporte de errores** también se coordina aquí.
Los errores simples se reportan directamente al proceso. Los errores de E/S reales pueden implicar reintentos, interacción con el usuario o, en casos graves, la detención del sistema.

Esta capa también gestiona la **asignación de dispositivos dedicados**, evitando que más de un proceso los use al mismo tiempo, ya sea bloqueando o rechazando solicitudes.

Por último, oculta las diferencias de **tamaño de bloque** entre dispositivos, ofreciendo un tamaño de bloque lógico uniforme a los niveles superiores.

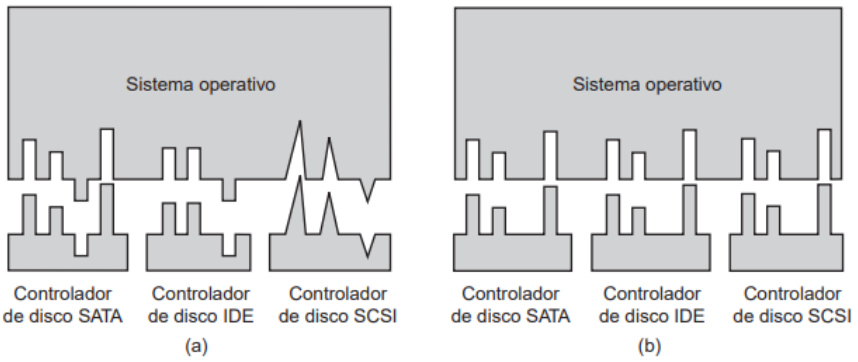


Figura 5-14. (a) Sin una interfaz de controlador estándar. (b) Con una interfaz de controlador estándar.

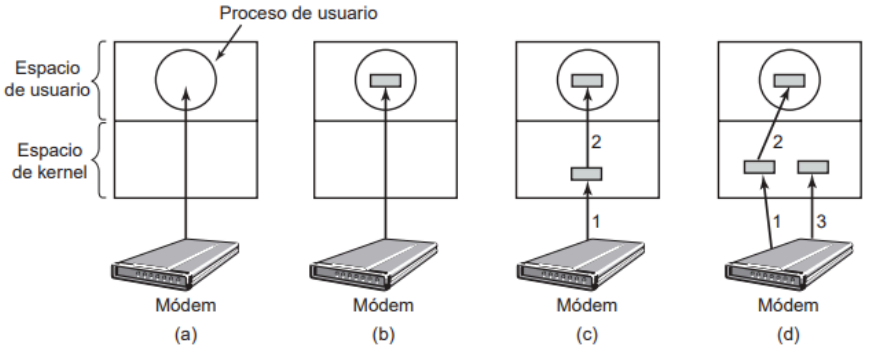


Figura 5-15. (a) Entrada sin búfer. (b) Uso de búfer en espacio de usuario. (c) Uso de búfer en el kernel, seguido de la acción de copiar al espacio de usuario. (d) Uso de doble búfer en el kernel.

5.3.4 Software de E/S en espacio de usuario

Una parte del software de E/S se ejecuta **fuera del kernel**, principalmente en forma de bibliotecas y procesos especiales.

Las llamadas al sistema de E/S suelen invocarse a través de **procedimientos de biblioteca**, como `write`, `read`, `printf` o `scanf`.
Estas bibliotecas se encargan del **formato de datos**, mientras que el kernel sólo maneja la transferencia.

Otra forma importante de E/S en espacio de usuario es el **uso de colas (spooling)**.
Se utiliza para dispositivos dedicados, como impresoras.
En lugar de permitir acceso directo al dispositivo, los procesos depositan su salida en un directorio de cola. Un proceso especial, llamado **demonio**, es el único que accede al dispositivo y procesa la cola.

Este esquema evita bloqueos innecesarios, mejora la equidad y permite manejar dispositivos lentos en sistemas multiprogramados.

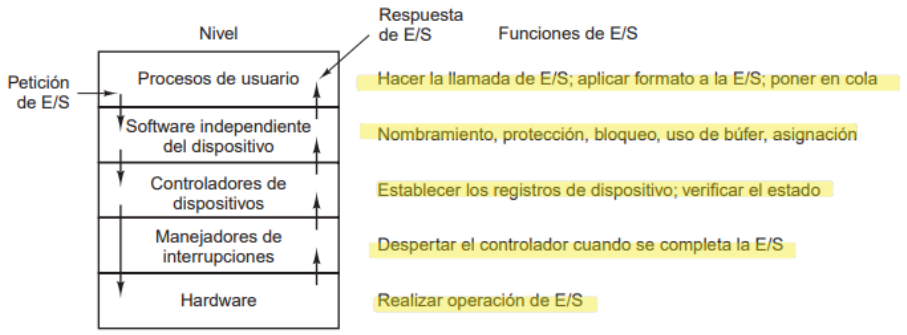


Figura 5-17. Niveles del sistema de E/S y las funciones principales de cada nivel.

5.4 Discos

Los discos son dispositivos de E/S fundamentales. Conceptualmente son simples, pero críticos, porque se utilizan como **memoria secundaria**, tanto para sistemas de archivos como para paginación. Existen discos magnéticos y discos ópticos, cada uno con características físicas y de rendimiento distintas.

5.4.1 Hardware de disco

Discos magnéticos

Los discos magnéticos se organizan en **cilindros**, **pistas** y **sectores**.
Un cilindro es el conjunto de pistas alineadas verticalmente; cada pista se divide en sectores, que son la unidad mínima de lectura y escritura.

En discos modernos (IDE, SATA), gran parte del trabajo lo realiza un **microcontrolador interno**, que maneja caché, reasignación de bloques defectuosos y comandos de alto nivel.

5.5 Relojes

Los **relojes (temporizadores)** son esenciales en sistemas de multiprogramación. Permiten **mantener la hora del día**, **evitar que un proceso monopolice la CPU** y **medir el uso del sistema**, entre otras funciones.

Aunque no son dispositivos de bloque ni de carácter, el **software de reloj** se implementa como un **controlador de dispositivo**.

Unidad 6 – Interbloqueos

Un **interbloqueo** ocurre cuando dos o más procesos quedan bloqueados de manera permanente porque cada uno posee recursos que los otros necesitan y ninguno puede continuar.

Los sistemas operativos administran recursos que **no pueden ser usados simultáneamente** por más de un proceso, por lo que deben otorgar **acceso exclusivo** de forma temporal.

Los interbloqueos pueden darse tanto con:

- **recursos de hardware** (impresoras, unidades de cinta, grabadores de CD),
- **recursos de software** (registros de bases de datos, entradas en tablas del sistema).

También pueden ocurrir en **sistemas distribuidos**, cuando los recursos se reservan a través de una red.

6.1 Recursos

Un **recurso** es cualquier objeto que un proceso debe:

1. solicitar,
2. utilizar,
3. liberar.

Puede ser un dispositivo físico o una estructura de información.
Puede haber **una o varias instancias** de un mismo tipo de recurso.

Si el recurso solicitado no está disponible, el proceso **debe esperar** (bloquearse o reintentar).

6.1.1 Recursos apropiativos y no apropiativos

Recursos apropiativos

Son aquellos que **pueden quitarse** a un proceso sin causar fallos.

Ejemplo: **memoria**.
El sistema operativo puede retirarla temporalmente (swap) y luego devolverla.

Los interbloqueos que involucran solo recursos apropiativos **pueden resolverse** reasignando recursos.

Recursos no apropiativos

Son aquellos que **no pueden quitarse** sin dañar el resultado del proceso.

Ejemplo: **grabador de CD** durante la grabación.
Quitar el recurso produce un fallo irreversible.

Los **interbloqueos reales** involucran principalmente **recursos no apropiativos**.

6.1.2 Adquisición de recursos

Muchos recursos se protegen usando **semáforos o mutexes**, inicializados en 1.

El uso típico es:

- `down()` para adquirir el recurso,
- usar el recurso,
- `up()` para liberarlo.

Cuando un proceso necesita **más de un recurso**, los adquiere **en forma secuencial**.

Orden de adquisición y aparición del interbloqueo

Si **todos los procesos solicitan los recursos en el mismo orden**, no hay interbloqueo.

Si **los procesos solicitan los recursos en distinto orden**, puede ocurrir interbloqueo.

Caso típico:

- Proceso A adquiere recurso 1 y espera recurso 2.
- Proceso B adquiere recurso 2 y espera recurso 1.
- Ninguno puede continuar.

Esto muestra que **el orden de adquisición de recursos es crítico** y que pequeñas diferencias en el código pueden provocar interbloqueos difíciles de detectar.

6.2 Introducción a los interbloqueos

Un **interbloqueo** existe cuando un conjunto de procesos está bloqueado de forma permanente porque **cada proceso espera un evento que sólo puede ser producido por otro proceso del mismo conjunto**.
Como todos esperan, **ninguno puede continuar**, liberar recursos ni despertar a los demás.

En este modelo se asume:

- procesos de **un solo hilo**,
- **sin interrupciones externas** que puedan despertar a un proceso bloqueado.

En la práctica, el evento esperado casi siempre es **la liberación de un recurso**.
Cada proceso del conjunto posee al menos un recurso y espera otro que está en manos de

otro proceso del mismo conjunto.
A esto se lo llama **interbloqueo de recursos**, el caso más común.

6.2.1 Condiciones necesarias para el interbloqueo (Coffman)

Para que ocurra un interbloqueo de recursos deben cumplirse **simultáneamente** cuatro condiciones:

1. **Exclusión mutua**
   Cada recurso puede ser asignado a **un solo proceso a la vez**.
2. **Contención y espera**
   Un proceso que ya posee recursos **puede solicitar recursos adicionales**.
3. **No apropiación**
   Los recursos **no pueden ser retirados por la fuerza**; sólo el proceso que los posee puede liberarlos.
4. **Espera circular**
   Existe una **cadena circular de procesos**, donde cada uno espera un recurso que posee el siguiente.

Si **alguna de estas condiciones no se cumple**, el interbloqueo **no es posible**.

Estas condiciones reflejan **políticas del sistema operativo**.
Más adelante se atacan los interbloqueos negando deliberadamente alguna de ellas.

6.2.2 Modelado de interbloqueos con gráficos de recursos

Los **gráficos de asignación de recursos** permiten representar interbloqueos de forma visual.

- **Procesos**: círculos
- **Recursos**: cuadros

Tipos de arcos:

- **Recurso → Proceso**: el recurso está asignado al proceso.
- **Proceso → Recurso**: el proceso está esperando ese recurso.

Un **ciclo en el gráfico** indica un interbloqueo (cuando hay una sola instancia de cada recurso).

Estos gráficos permiten analizar, paso a paso, si una secuencia de solicitudes y liberaciones **conduce o no a un interbloqueo**.

El sistema operativo **no está obligado** a conceder una solicitud inmediatamente.
Si al otorgarla se produciría un ciclo, puede **posponer la asignación** y ejecutar otros procesos hasta que sea seguro.

Estrategias generales frente a los interbloqueos

Existen cuatro enfoques clásicos:

1. **Ignorar el problema** (algoritmo de la avestruz).
2. **Detección y recuperación**.
3. **Evitación dinámica** mediante asignación cuidadosa.
4. **Prevención**, negando estructuralmente alguna condición.

6.4 Detección y recuperación de un interbloqueo

En este enfoque el sistema **no intenta evitar** los interbloqueos.
Permite que ocurran, **los detecta** y luego **actúa para recuperarse**.

6.4.1 Detección con un recurso de cada tipo

Se asume que existe **una sola instancia de cada recurso**.
En este caso se usa un **gráfico de asignación de recursos**.

- Si el gráfico **tiene un ciclo**, hay interbloqueo.
- Los procesos que forman parte del ciclo están interbloqueados.
- Si **no hay ciclos**, no existe interbloqueo.

La detección se reduce a **buscar ciclos en un gráfico dirigido**.
Existen algoritmos formales que recorren el gráfico y detectan ciclos mediante una **búsqueda en profundidad**.
Si durante el recorrido se vuelve a un nodo ya visitado, se detecta un ciclo y el algoritmo termina.

6.4.2 Detección con varios recursos de cada tipo

Cuando existen **varias instancias de un mismo recurso**, el gráfico ya no alcanza.
Se utiliza un método basado en **matrices y vectores**.

Estructuras usadas:

- **E**: recursos existentes.
- **A**: recursos disponibles.
- **C**: recursos actualmente asignados a cada proceso.
- **R**: recursos que cada proceso aún necesita.

Idea central del algoritmo:

- Se busca un proceso cuya **petición pendiente pueda satisfacerse** con los recursos disponibles.
- Ese proceso se considera ejecutable, se marca y **devuelve sus recursos**.
- Los recursos devueltos se suman a los disponibles.
- El proceso se elimina del análisis y se repite.

Resultado:

- Si **todos los procesos pueden marcarse**, no hay interbloqueo.
- Los procesos que **quedan sin marcar** están en interbloqueo.

El orden de evaluación no importa: el resultado final es siempre el mismo.

La detección puede hacerse:

- en cada solicitud de recurso (costosa),
- periódicamente,
- o cuando el uso de la CPU cae notablemente.

6.4.3 Recuperación de un interbloqueo

Una vez detectado el interbloqueo, el sistema debe **romperlo**.
No existe una solución ideal.

Apropiación de recursos

Se quita un recurso a un proceso y se le asigna a otro.
Sólo es posible si el recurso puede retirarse **sin dañar el cómputo**.
En muchos casos requiere intervención manual.

Retroceso (rollback)

Los procesos realizan **puntos de comprobación** periódicos.
Para recuperarse:

- se reinicia un proceso desde un punto anterior,
- se liberan los recursos adquiridos después de ese punto,
- el trabajo realizado desde entonces se pierde.

Eliminación de procesos

Se elimina uno o más procesos para liberar recursos.

- Puede eliminarse un proceso del ciclo.
- O eliminar otro proceso que tenga recursos necesarios.

Se prefiere eliminar procesos que **puedan reiniciarse sin efectos secundarios**.
Procesos que modifican datos persistentes pueden generar resultados incorrectos si se ejecutan nuevamente.

6.5 Cómo evitar interbloqueos

La evitación de interbloqueos consiste en **decidir si es seguro o no otorgar un recurso en el momento en que se solicita**.
A diferencia de la detección, el sistema **no deja que el interbloqueo ocurra**: analiza cada asignación antes de hacerla.

Esto sólo es posible si el sistema **conoce información de antemano**, en especial cuántos recursos puede llegar a necesitar cada proceso.

6.5.1 Trayectorias de los recursos

El concepto central es el de **regiones seguras e inseguras**.

El estado del sistema puede representarse como una trayectoria que muestra el avance de los procesos y el uso de los recursos.
Hay regiones a las que **no se debe entrar**, porque desde allí el sistema inevitablemente llegará a un interbloqueo.

Cuando un proceso solicita un recurso, el sistema debe decidir:

- si al otorgarlo permanece en una región segura, o
- si al otorgarlo entra en una región insegura.

Para evitar el interbloqueo, **no se otorgan solicitudes que lleven al sistema a una región insegura**, aunque en ese momento no exista todavía un interbloqueo.

6.5.2 Estados seguros e inseguros

Un **estado seguro** es aquel en el que existe **algún orden de ejecución** que permite que **todos los procesos se completen**, incluso si todos solicitan de inmediato su máximo número de recursos.

Un **estado inseguro** es aquel desde el cual **no se puede garantizar** que todos los procesos terminen.

Puntos clave:

- Un estado inseguro **no es lo mismo que un interbloqueo**.
- Desde un estado inseguro el sistema puede seguir funcionando un tiempo.
- La diferencia es que **ya no hay garantía** de que se evite el interbloqueo.

La evitación consiste justamente en **no permitir transiciones de estados seguros a estados inseguros**.

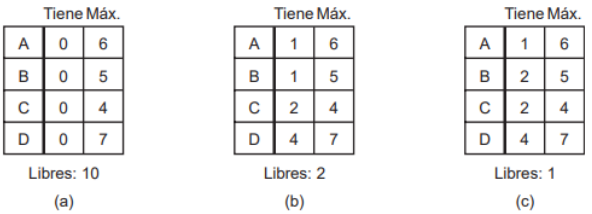
6.5.3 El algoritmo del banquero para un solo recurso

El **algoritmo del banquero**, propuesto por Dijkstra, decide si una solicitud se puede otorgar sin llevar al sistema a un estado inseguro.

La idea es simple:

- Antes de otorgar un recurso, el sistema **simula** la asignación.
- Si después de la simulación el estado sigue siendo seguro, la solicitud se concede.
- Si el estado resultante es inseguro, la solicitud se pospone.

El algoritmo se basa en que cada proceso **declare de antemano su necesidad máxima** de recursos.



	Tiene	Máx.
A	0	6
B	0	5
C	0	4
D	0	7

Libres: 10 (a)

	Tiene	Máx.
A	1	6
B	1	5
C	2	4
D	4	7

Libres: 2 (b)

	Tiene	Máx.
A	1	6
B	2	5
C	2	4
D	4	7

Libres: 1 (c)

Figura 6-11. Tres estados de asignación de recursos: (a) Seguro. (b) Seguro. (c) Inseguro.

6.5.4 El algoritmo del banquero para varios recursos

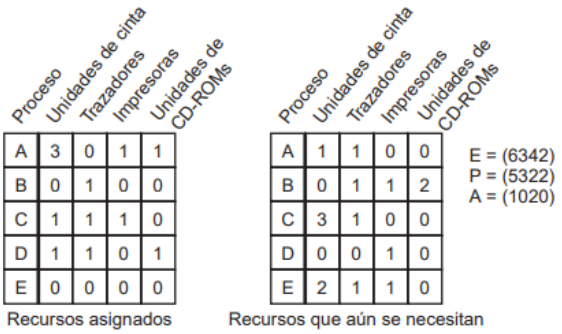
El algoritmo se generaliza usando las estructuras:

- **E**: recursos existentes.
- **A**: recursos disponibles.
- **C**: recursos actualmente asignados.
- **R**: recursos que aún necesita cada proceso.

Un estado es seguro si el sistema puede encontrar una secuencia en la que:

1. Algún proceso puede obtener todos los recursos que necesita con los recursos disponibles.
2. Ese proceso termina y libera sus recursos.
3. El procedimiento se puede repetir hasta que todos los procesos finalicen.

Si no existe ningún proceso cuyas necesidades puedan satisfacerse, el estado es inseguro y la asignación debe rechazarse.



Proceso	Unidades de cinta	Trazadores	Impresoras	Unidades de CD-ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Recursos asignados

Proceso	Unidades de cinta	Trazadores	Impresoras	Unidades de CD-ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Recursos que aún se necesitan

E = (6342)
P = (5322)
A = (1020)

Figura 6-12. El algoritmo del banquero con varios recursos.

6.6 Cómo prevenir interbloqueos

La **prevención** busca que el interbloqueo sea **estructuralmente imposible**.
La idea es simple: **si al menos una de las cuatro condiciones de Coffman nunca se cumple, el interbloqueo no puede ocurrir**.

6.6.1 Ataque a la exclusión mutua

Si los recursos no fueran exclusivos, no habría interbloqueos.
En la práctica, esto rara vez es posible.

Una técnica común es **virtualizar recursos**. Por ejemplo, la impresora no se asigna directamente a los procesos, sino a un demonio de impresión que recibe los trabajos en una cola. Así, los procesos no compiten por la impresora real.

Esto reduce interbloqueos, pero no los elimina por completo, ya que se traslada el problema a otros recursos, como el espacio en disco.

6.6.2 Ataque a la contención y espera

Esta condición se elimina si **un proceso no puede retener recursos mientras espera otros**.

Dos enfoques:

- Exigir que el proceso solicite **todos los recursos al inicio**.
- Obligar a que, si necesita un recurso adicional, **libere primero los que ya tiene**.

Ambos métodos evitan interbloqueos, pero **desperdician recursos** y requieren que los procesos conozcan sus necesidades de antemano, lo cual no siempre es posible.

6.6.3 Ataque a la no apropiación

Si los recursos pudieran quitarse a la fuerza, el interbloqueo se rompería.
Esto sólo funciona con **recursos apropiativos**.

Algunos recursos pueden virtualizarse (impresoras), pero otros no, como:

- registros de bases de datos,
- estructuras internas del sistema operativo.

Por eso, este enfoque tiene **aplicación limitada**.

6.6.4 Ataque a la espera circular

Es el método más usado conceptualmente.

La técnica consiste en **imponer un orden global a los recursos** y exigir que:

- los procesos sólo soliciten recursos en **orden creciente**,
- nunca pidan un recurso “menor” al que ya poseen.

Con esta regla, **no pueden formarse ciclos**, y sin ciclos no hay interbloqueo.

El problema es que, en sistemas reales, **definir un orden válido para todos los recursos suele ser impracticable**.

Condición	Método
Exclusión mutua	Poner todo en la cola de impresión
Contención y espera	Solicitar todos los recursos al principio
No apropiativa	Quitar los recursos
Espera circular	Ordenar los recursos en forma numérica